

第五章 神经网络与深度学习

提 纲

- 一、人工神经网络概述
- 二、前馈神经网络与参数优化
- 三、卷积神经网络
- 四、循环神经网络
- 五、注意力机制与正则化
- 六、自然语言和计算机视觉应用

一、人工神经网络概述

人工神经网络概述：定义

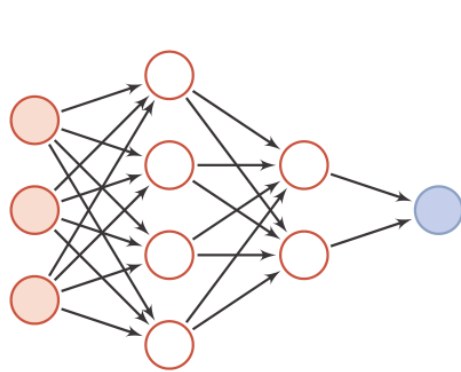
人工神经网络 (Artificial Neural Network, ANN) 的定义

- “人工神经网络是由具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的反应”
[Kohonen, 1988]
- ANN受到生物学的启发，生物的学习系统是由相互连接的神经元组成的异常复杂的网络。
- 人脑大约包含 10^{12} 个神经元，分成约1000种类型，每个神经元大约与 $10^2 \sim 10^4$ 个其他神经元相连接，形成极为错综复杂而又灵活多变的神经网络。

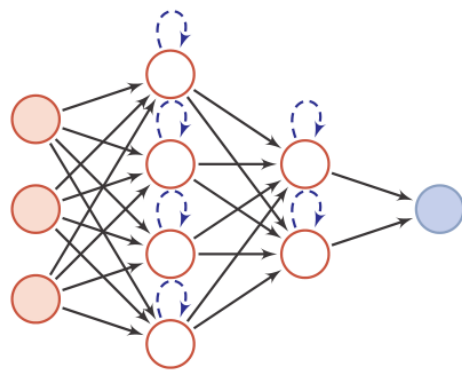
一、人工神经网络概述

人工神经网络概述：结构

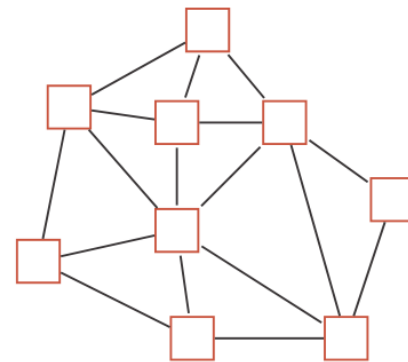
- ▶ 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络



(b) 记忆网络



(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

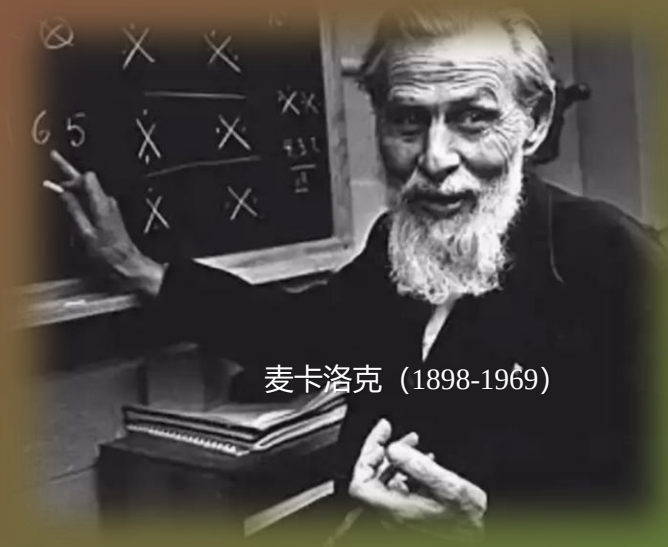
一、人工神经网络概述

人工神经网络概述：发展史

1943 年，神经科学家瓦伦·麦卡洛克 (Warren McCulloch) 和逻辑学家沃尔特·皮茨 (Walter Pitts) 合作在《数学生物物理期刊》上发表了**神经网络的开山之作**：

“A Logical Calculus of Ideas Immanent in Nervous Activity”，提出了第一个神经元数学模型 “**McCulloch–Pitts (MCP) neuron**”，是人类历史上第一次对大脑工作原理描述的尝试。

历史意义：我们在科学史上第一次知道了我们是怎么知道的 (for the first time in the history of science, we know how we know)”



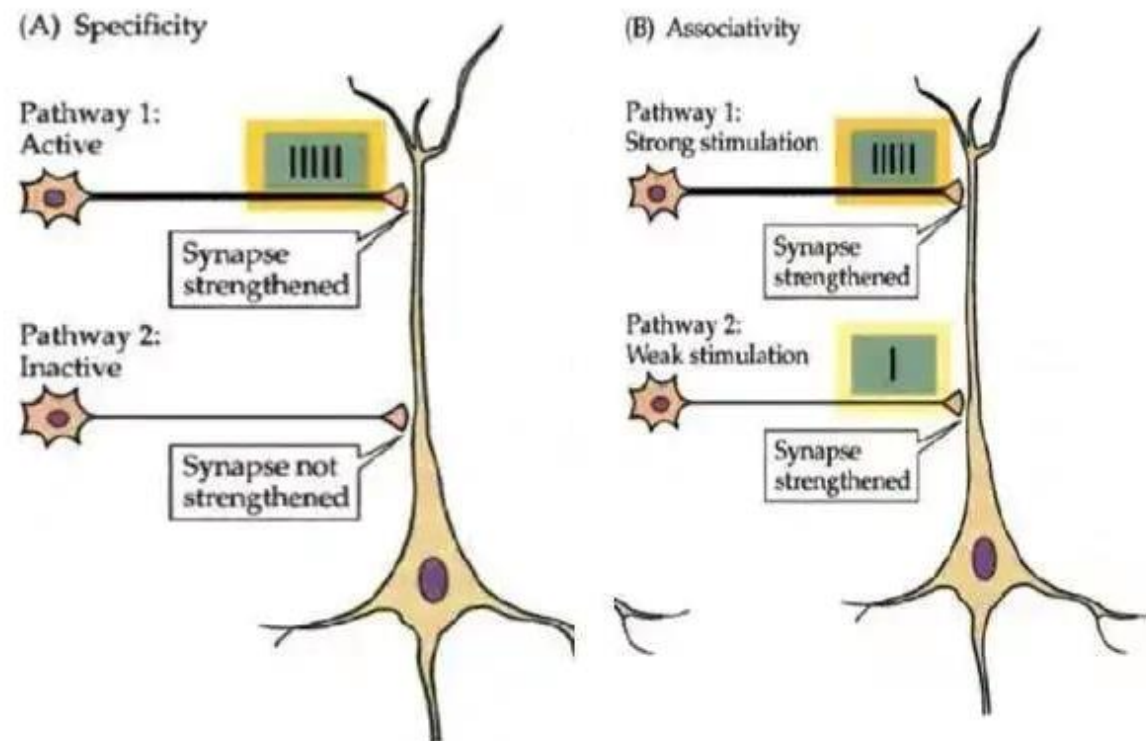
麦卡洛克 (1898-1969)

一、人工神经网络概述

人工神经网络概述：发展史

1949年，心理学家唐纳德·赫布（Donald Hebb）在《The Organization of Behavior》中对神经元之间连接强度的变化进行了分析，提出了赫布理论(Hebbian theory)：神经元之间持续**重复经验刺激**可导致突触传递效能增加(“Neurons that fire together, wire together”)。其主要思想是根据两个**神经元的激发状态来调整连接关系**，以此实现对简单神经活动的模拟。

历史意义：神经元之间突触的强弱变化是学习与记忆的生理学基础



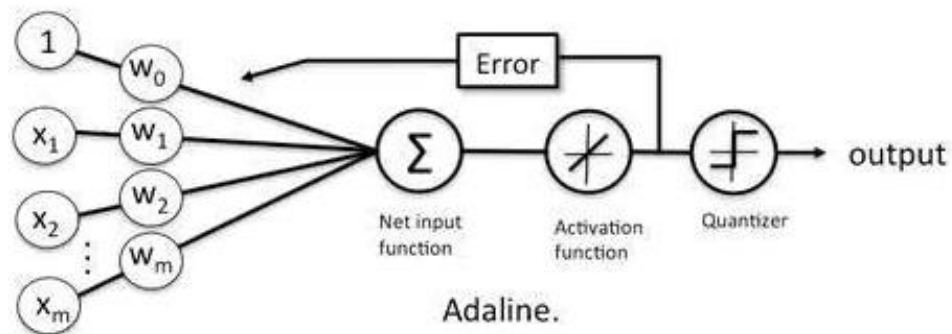
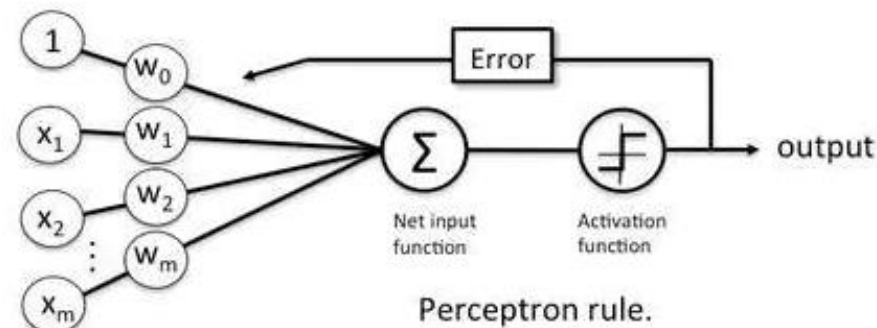
一、人工神经网络概述

人工神经网络概述：发展史

20世纪50年代就职于Cornell航空实验室的弗朗克·罗森布拉特（Frank Rosenblatt）提出“**感知机**（perceptron）”模型，其是一个仅包含输入层和输出层的两层神经网络，可完成两类问题的线性分类。**感知机是人工神经网络的第一个实际应用，标志着神经网络进入了新的发展阶段。**

1960年，斯坦福大学教授维德罗（Bernard Widrow）提出了Adaline自适应线性神经元。Adaline和感知机很相似，也是机器学习的鼻祖模型之一。Adaline网络和感知机的区别就是**将感知机的Step函数换为Linear线性函数。**

历史意义：感知机模型被誉为“Electronic 'Brain' Teaches Itself（能自学的电脑）”



1969年，Marvin Minsky 和 Seymour Papert 从数学的角度证明了单层神经网络具有有限的功能，甚至在面对简单的“异或”逻辑问题时也显得无能为力。此后，**神经网络的研究陷入了很长一段时期的低迷期。**

一、人工神经网络概述

人工神经网络概述：发展史

1974年，哈佛大学的一篇博士论文证明了在神经网络多加一层，并且利用“反向传播”（back-propagation）学习方法，**可以解决异或问题**。这篇论文的作者是沃波斯（Paul Werbos），他后来得到了IEEE神经网络学会的先驱奖。

神经网络在20世纪80年代的复兴归功于物理学家霍普菲尔德（John Hopfield）。1982年，那时在加州理工学院担任生物物理教授的霍普菲尔德，提出了一种新的神经网络，**可以解决一大类模式识别问题，还可以给出一类组合优化问题的近似解**。这种神经网络模型后来被称为霍普菲尔德网络。

历史意义：霍普菲尔德模型的提出振奋了神经网络领域，使得神经网络的发展从低迷期转入繁荣期



霍普菲尔德（1933-）

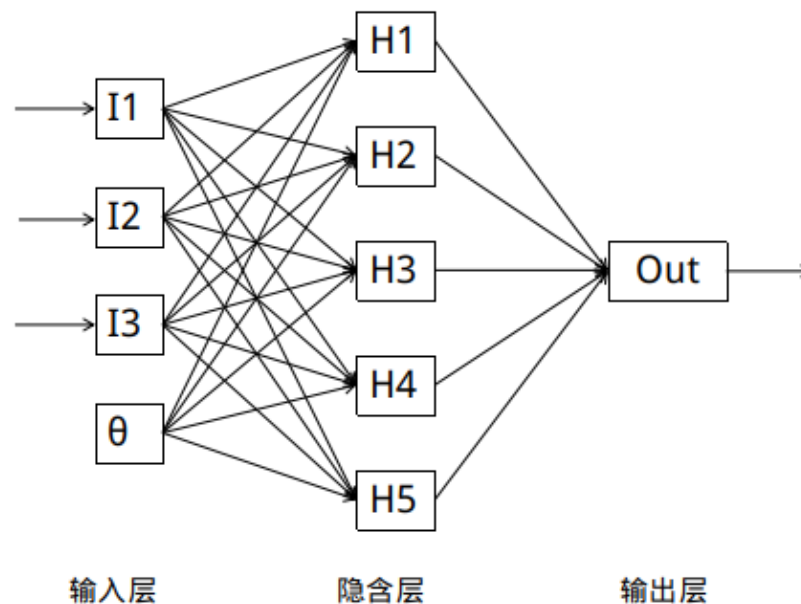
一、人工神经网络概述

人工神经网络概述：发展史

1986年神经网络之父辛顿（Hinton）**提出了适用于多层感知机MLP的BP算法**，并在层与层之间的传播过程中引入了 Sigmoid函数，为神经网络引入了非线性，改良了单层感知机无法解决异或问题的缺陷。

BP算法包括前向传播的计算与反向传播的计算。如果是已经训练好的神经网络模型，仅使用前向传播即可预测出结果。神经网络的训练还需要反向传播的参与。当前向传播预测值与真实值有出入的时候，利用**梯度下降**的方法，在反向传播中使模型权重改变，让模型权重更加贴合真实值。常见的梯度更新优化器有如下四种，它们分别是：SGD、Momentum、AdaGrad、Adam。

历史意义：克服了第一代神经网络的弱点，为多层神经网络的学习训练与实现提供了一种切实可行的解决途径。



BP神经网络

一、人工神经网络概述

人工神经网络概述：发展史

2006年，辛顿在Science等期刊上发表题为《Reducing the Dimensionality of Data with Neural Networks》的文章，并**首次提出“深度信念网络（deep belief network）”模型**（在相关分类任务中可取得性能超过传统浅层学习模型（BP神经网络）的结果）。

这篇文章是一个分水岭，拉开了深度学习大幕，标志着深度学习的诞生。从此，历史这样写就：从感知机提出，到BP算法应用以及2006年以前的历史被称为浅层学习，以后的历史被称为深度学习。

历史意义：有别于分段式学习，端到端(end to end)的深度学习框架建立



辛顿 (1947-)

一、神经网络概述

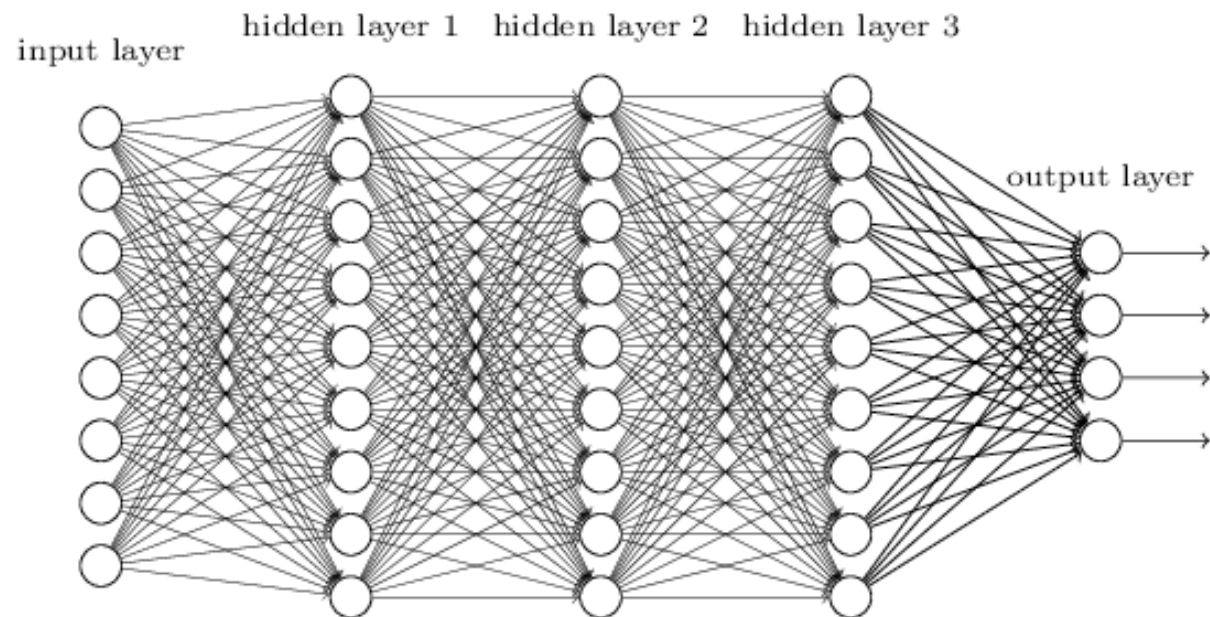
神经网络概述：发展史

第三代神经网络代表：

- RNN、LSTM
- CNN
- Word2vec
- BERT、Transformer、LLM

历史意义：从造人和造脑到更通用人工智能

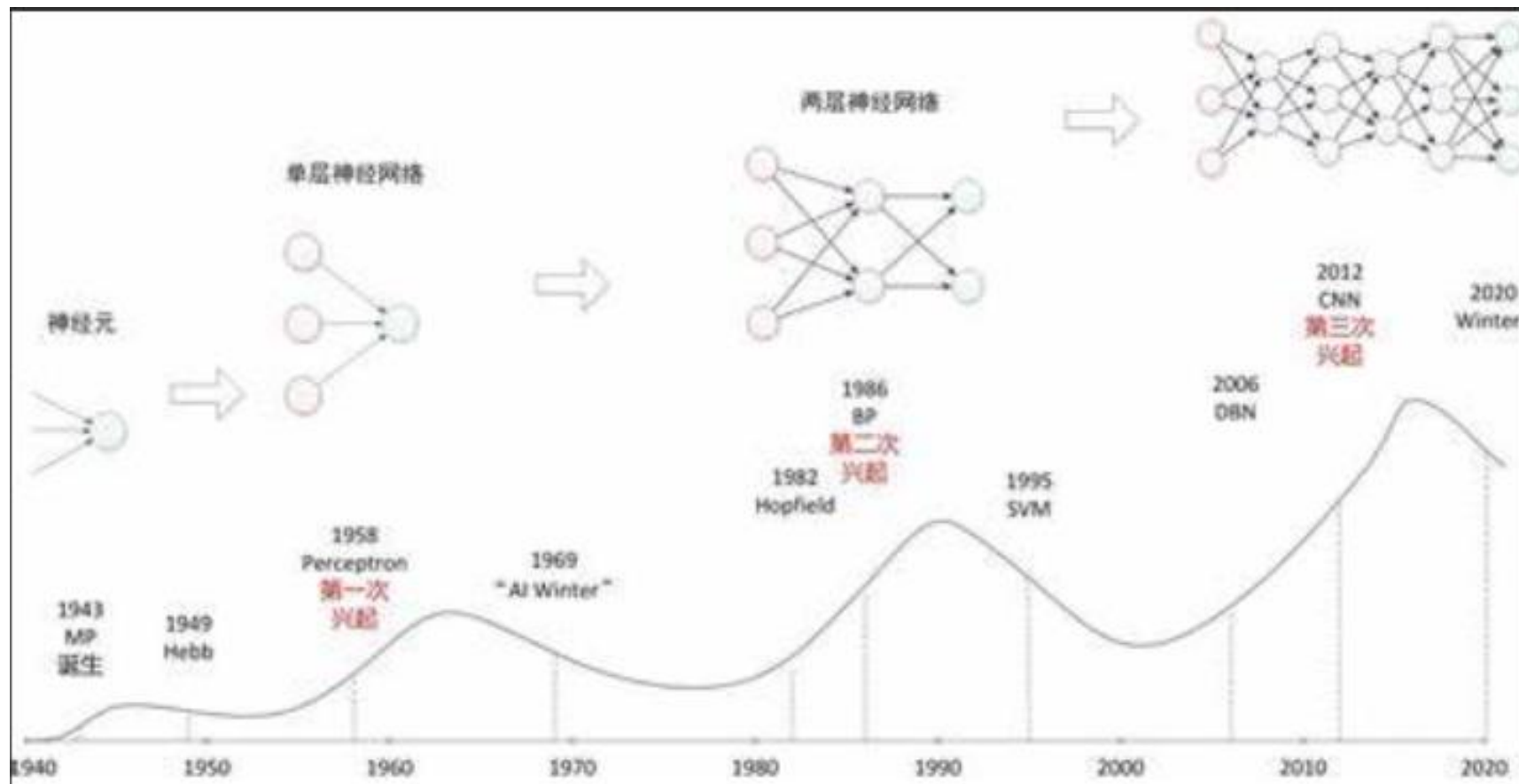
引发AI第三次崛起
语言大模型



深度神经网络 (DNN)

一、人工神经网络概述

人工神经网络概述：发展史



- 2012年, Hinton 组参加ImageNet 竞赛, 使用 CNN 模型以超过第二名10个百分点的成绩夺得当年竞赛的冠军

二、前馈神经网络与参数优化

前馈神经网络：神经元

- 在生物学中，神经元细胞有**兴奋与抑制**两种状态。大多数神经元细胞在正常情况下处于抑制状态，一旦某个神经元受到刺激并且电位超过一定的阈值后，这个神经元细胞就被激活，处于兴奋状态，并向其他神经元传递信息。
- 基于神经元细胞的结构特性与信息传递方式，神经科学家瓦伦·麦卡洛克和逻辑学家沃尔特·皮茨合作提出了MCP模型。在人工神经网络中，**MCP模型成为最基本的结构**
- 从图5.1可见，给定 n 个输入数据 $x_i (1 \leq i \leq n)$ 与连接参数 $w_i (1 \leq i \leq n)$ ，MCP神经元模型对输入数据线性加权求和，然后使用函数 $\Phi(\cdot)$ 将加权累加结果映射为0或1，以完成两类分类任务。

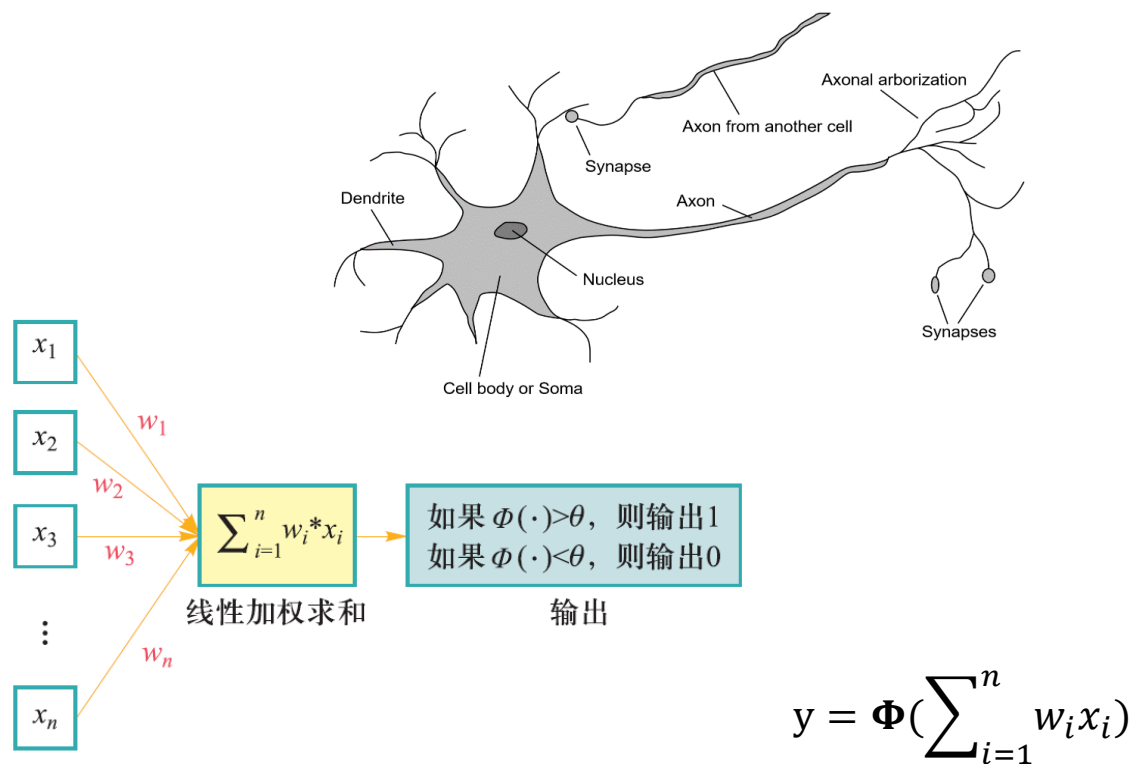


图 5.1 MCP 模型结构

其中 w_i 为预先设定的连接权重值（一般在0和1中一个值或者1和-1中取一个值），（即权重）。 $\Phi(\cdot)$ 将输入端数据与连接权重所得线性加权累加结果与预先设定阈值 θ 进行比较，根据比较结果输出1或0

二、前馈神经网络与参数优化

前馈神经网络：感知机

- 早期的感知机结构和MCP模型相似，由一个输入层和一个输出层构成，因此也被称为“**单层感知机**”。感知机的输入层负责接收实数值的输入向量，输出层则能输出1或-1两个值。
- 对于具有 n 个输入以及对应连接权重系数为 w_i 的感知机，首先通过线性加权得到输入数据的累加结果 z ： $z = w_1x_1 + w_2x_2 + \dots + b$ 。这里 $x_1, x_2 \dots x_n$ 为感知机的输入， b 为偏置项（bias）。然后将 z 作为激活函数 $\phi(\cdot)$ 的输入，激活函数会将 z 与某一阈值（此例中，阈值为0）进行比较，如果大于该阈值则感知器输出为1，否则输出为-1。
- **感知机中的连接权重参数是通过多次迭代训练得到的**。具体而言，单层感知机构建损失函数，来计算模型预测值与数据真实值之间的“误差”，通过最小化损失函数取值，来优化模型参数。

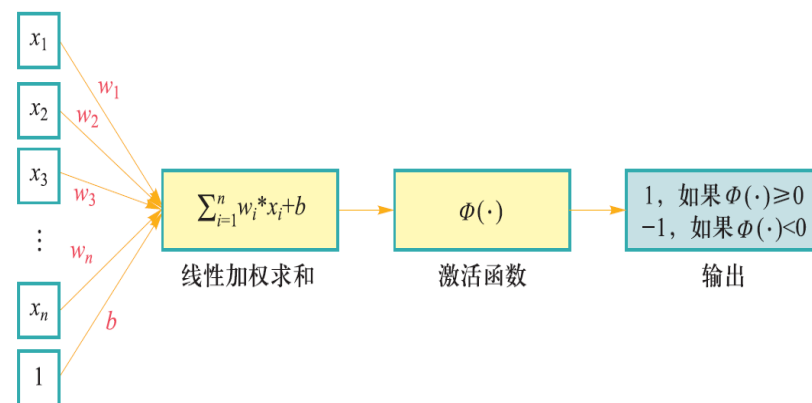


图 5.2 感知机模型

二、前馈神经网络与参数优化

前馈神经网络：感知机

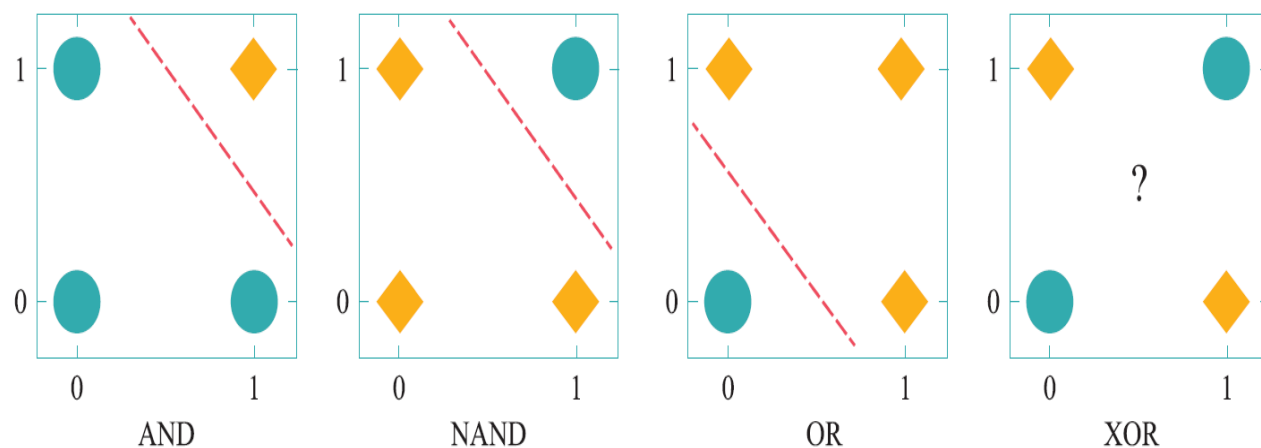


图 5.3 单层感知机模拟不同逻辑函数功能示意

感知机可模拟逻辑与(AND)、逻辑与非(NAND)和逻辑或(OR)等线性可分函数，但是无法完成逻辑异或 (XOR) 这一非线性可分逻辑函数任务

二、前馈神经网络与参数优化

前馈神经网络

- 在感知机模型中增加若干隐藏层，增强神经网络的非线性表达能力，就会让神经网络具有更强的拟合能力。因此，由多个隐藏层构成的**多层感知机（MLP）**被提出，这也被称为**前馈神经网络**。
- 如图5.4所示，**前馈神经网络由输入层、输出层和若干隐藏层构成。**
- 前馈神经网络中相邻层所包含的神经元之间通常使用“**全连接**”方式进行连接。但同一层内神经元之间没有连接。前馈神经网络可以模拟复杂非线性函数功能，所模拟函数的复杂性取决于网络隐藏层数目和各层中神经元数目。

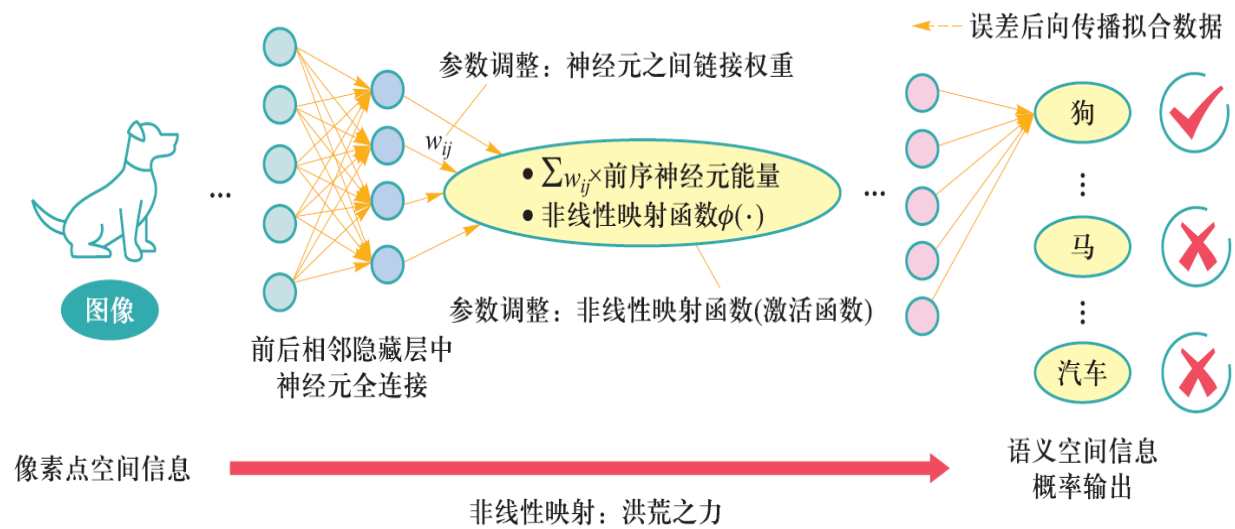
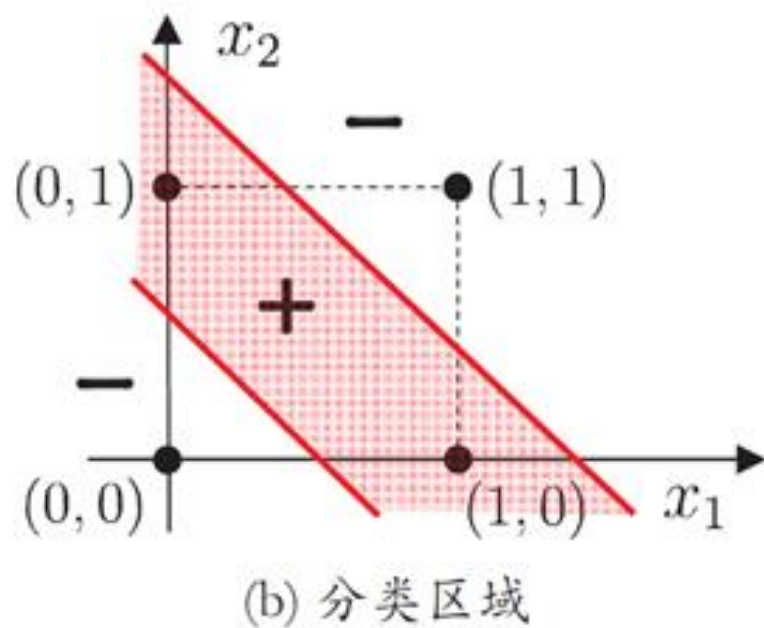
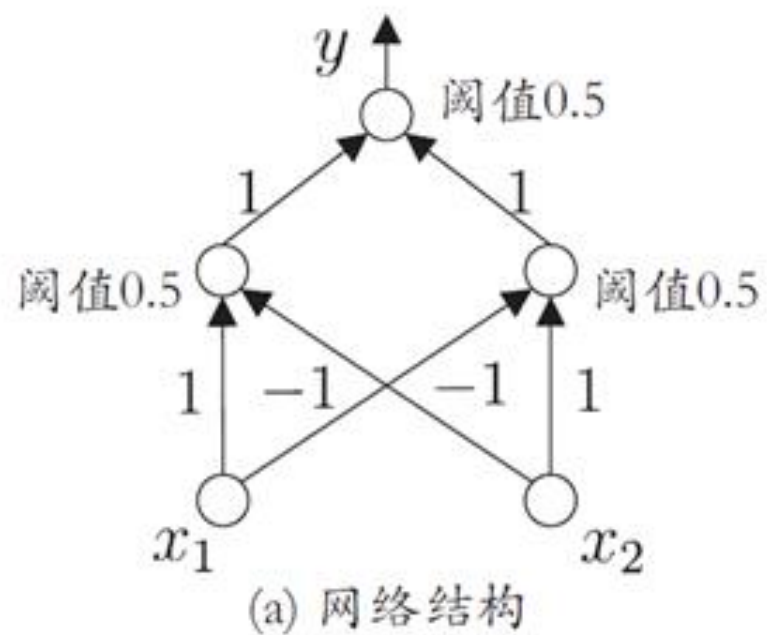


图 5.4 前馈神经网络模型

二、前馈神经网络与参数优化

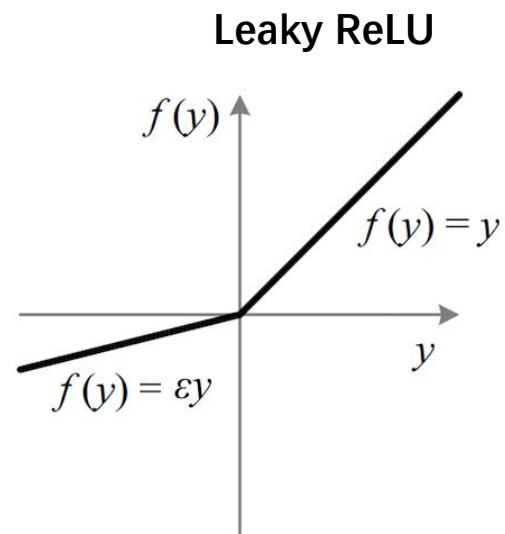
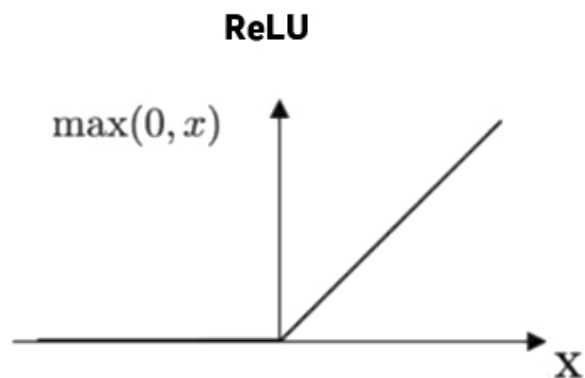
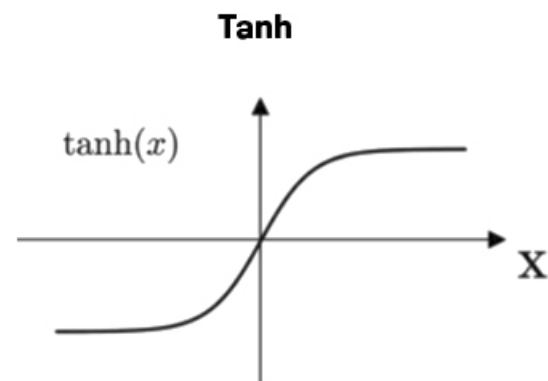
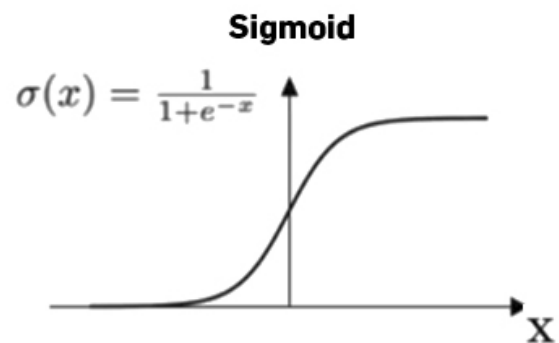
前馈神经网络



能解决异或问题的两层感知机

二、前馈神经网络与参数优化

前馈神经网络：执行非线性映射的激活函数



二、前馈神经网络与参数优化

前馈神经网络：执行非线性映射的激活函数

sigmoid函数

$$f(x) = \frac{1}{1 + e^{-x}}$$

优点：

- 1) **概率形式输出**； sigmoid函数值域为(0, 1)，因此Sigmoid函数的输出可视为概率值
- 2) **单调递增**。 Sigmoid函数对输入 x 取值范围没有限制，但当 x 大于一定值后，函数输出无限趋近于1，而小于一定数值后，函数输出无限趋近于0。特别地，当 $x=0$ 时，函数输出为0.5；
- 3) **非线性变化**。 x 取值在0附近时，函数输出值的变化幅度比较大（函数值变化陡峭），意味着函数在0附近容易被激活且是非线性变化。当 x 取值很大或很小时，函数输出值几乎不变，这是基于概率的一种认识与需要。

缺点： $f'(x) = \left(\frac{1}{1+e^{-x}}\right)' = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1+e^{-x}-1}{(1+e^{-x})^2} = \frac{1}{(1+e^{-x})} \left(1 - \frac{1}{(1+e^{-x})}\right) = f(x)(1 - f(x))$

- 1) **反向传播时，易出现梯度接近于0的情况，即梯度消失（vanishing gradient）的问题**
- 2) **函数指数形式，计算复杂度高**

二、前馈神经网络与参数优化

前馈神经网络：执行非线性映射的激活函数

ReLU函数

$$f(x) = \begin{cases} 0, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases} \quad \text{或者 } f(x) = \max(0, x)$$

ReLU函数的导数为：

$$f'(x) = \begin{cases} 0, & \text{for } x < 0 \\ 1, & \text{for } x \geq 0 \end{cases}$$

优点：1) 当输入 $x \geq 0$ 时，ReLU的导数为常数，不会出现梯度消失、梯度饱和问题。

2) 训练神经网络能加速收敛，计算简单，复杂度低，不需要进行指数运算。

3) 当 $x < 0$ 时，ReLU的梯度总是0，神经元“死亡”，参与分类等任务神经元数目稀缺，这种稀疏性可以在一定程度上克服过拟合现象。

缺点：极端情况会出现大量“死亡”神经元。

使用其他激活函数解决这个问题，如 Leaky ReLU 或 Parametric ReLU，它们在 $x < 0$ 时梯度取值非零。

ReLU函数---隐藏层常用的默认选择

二、前馈神经网络与参数优化

前馈神经网络：执行非线性映射的激活函数

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x)$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma_i \min(0, x)$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\text{softplus}(x) = \log(1 + \exp(x))$$

当前的大模型中用的什么激活函数？

SiLU, GeLU, SwiGLU

二、前馈神经网络与参数优化

前馈神经网络：执行非线性映射的激活函数

Softmax激活函数

$$y_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$$

Softmax函数一般用于多分类任务最后一层

Softmax函数将输入向量 x_i 映射到第 i 个类别的概率

$y_i, 0 < y_i < 1$ 。

图5.5给出了三类分类问题的Softmax输出示意图。。

由于Softmax输出结果的值累加起来为1，因此可将输出概率最大的作为分类目标（图5.5中被分类为第一类）。

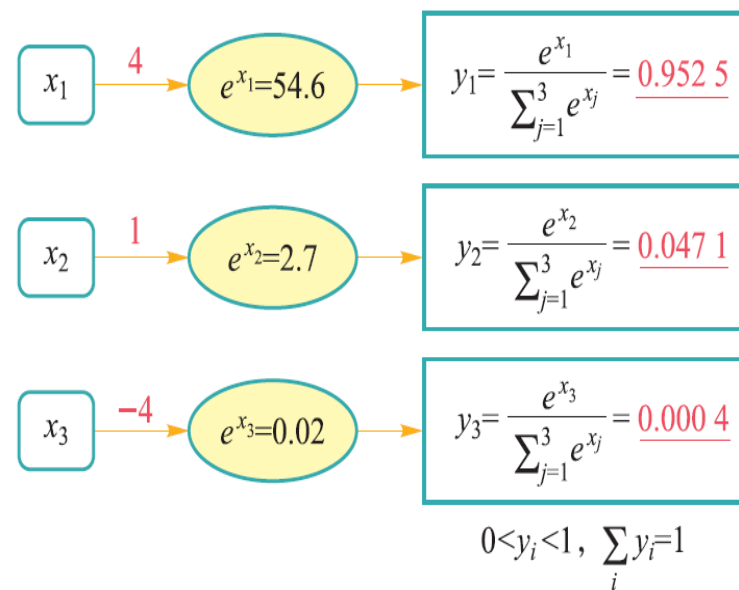


图 5.5 三分类问题的 softmax 输出示意

二、前馈神经网络与参数优化

前馈神经网络：神经网络参数优化

- 多层神经网络结构设计完成后（比如多少个隐藏层，每个隐藏层中包含的神经元数目以及神经元采用的非线性激活函数等），需要对神经网络中参数进行优化学习，以使得神经网络能够将给定输入数据映射到所期望的输出语义空间，完成分类识别等任务。
- 前馈神经网络参数优化是一个监督学习的过程，通过损失函数来计算模型预测值 $\hat{y}_i$ 与真实值 y_i 之间误差或损失，利用其来优化模型参数，以便让神经网络按照 x_i 的真实信息 y_i 输出。
- 模型利用反向传播算法将损失函数计算所得误差从输出端出发、由后向前传递给神经网络中每个单元，然后通过梯度下降算法对神经网络中参数进行更新。随着迭代的进行，模型预测所得 $\hat{y}_i$ 与真实值 y_i 之间的误差逐步缩小，模型预测准确率得以提升，预测的能力得以增强，当迭代达到一定轮次或准确率达到一定水平时，则可认为模型参数已被优化完毕。

二、前馈神经网络与参数优化

神经网络参数优化：损失函数

- **损失函数 (loss function) 又称为代价函数 (cost function)**，用来计算模型预测值与真实值之间的误差。损失函数是神经网络设计中的一个重要组成部分。通过定义与任务相关的良好损失函数，在训练过程中可根据损失函数来计算神经网络的误差大小，进而优化神经网络参数。
- 两种最常用的损失函数：均方误差损失函数和交叉熵损失函数。

均方误差损失函数

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

交叉熵损失函数

$$H(y_i, \hat{y}_i) = -y_i * \log \hat{y}_i$$

二、前馈神经网络与参数优化

神经网络参数优化：损失函数

交叉熵损失函数

$$H(y_i, \hat{y}_i) = -y_i * \log \hat{y}_i$$

- “熵”被用来表示热力学系统所呈现的无序程度。香农将这一概念引入信息论领域，提出了“信息熵”概念，通过对数函数来测量信息的不确定性。
- 交叉熵（cross entropy）是信息论中的重要概念，主要用来度量两个概率分布间的差异。
 y_i 是样本 x_i 分类的真实概率、 $\hat{y}_i$ 是模型预测概率。
- 交叉熵刻画了两个概率分布之间的距离，旨在描绘通过概率分布 $\hat{y}_i$ 来表达概率分布 y_i 的困难程度，两个概率分布 y_i 和 $\hat{y}_i$ 越接近，交叉熵越小。即，预测类别分布概率与实际类别分布概率之间的差距越小，交叉熵就越小。

二、前馈神经网络与参数优化

神经网络参数优化：损失函数

这里以一个例子来说明这个事实。假设模型 1 和模型 2 分别对三个分别属于类别 1、类别 2 和类别 3 的样本进行分类，得到如表 5.2 所示的分类结果。

表 5.2 两个模型的分类结果

样本	真实值	模型 1 预测结果	模型 2 预测结果
样本 1（类别 1）	0 0 1	0.3 0.3 0.4	0.1 0.2 0.7
样本 2（类别 2）	0 1 0	0.3 0.4 0.3	0.1 0.7 0.2
样本 3（类别 3）	1 0 0	0.1 0.2 0.7	0.3 0.4 0.3

模型 1 在三个样本上的交叉熵损失平均值为

$$\frac{- (0 \times \log 0.3 + 0 \times \log 0.3 + 1 \times \log 0.4 + 0 \times \log 0.3 + 1 \times \log 0.4 + 0 \times \log 0.3 + 1 \times \log 0.1 + 0 \times \log 0.2 + 0 \times \log 0.7)}{3}$$

=1.37

模型 2 在三个样本上的交叉熵损失平均值为

$$\frac{- (0 \times \log 0.1 + 0 \times \log 0.2 + 1 \times \log 0.7 + 0 \times \log 0.1 + 1 \times \log 0.7 + 0 \times \log 0.2 + 1 \times \log 0.3 + 0 \times \log 0.4 + 0 \times \log 0.3)}{3}$$

=0.63

可见，交叉熵损失函数准确给出了模型 2 预测性能优于模型 1 预测性能的事实。如果模型预测值与真实值一致，则交叉熵损失为零。

二、前馈神经网络与参数优化

神经网络参数优化：梯度下降

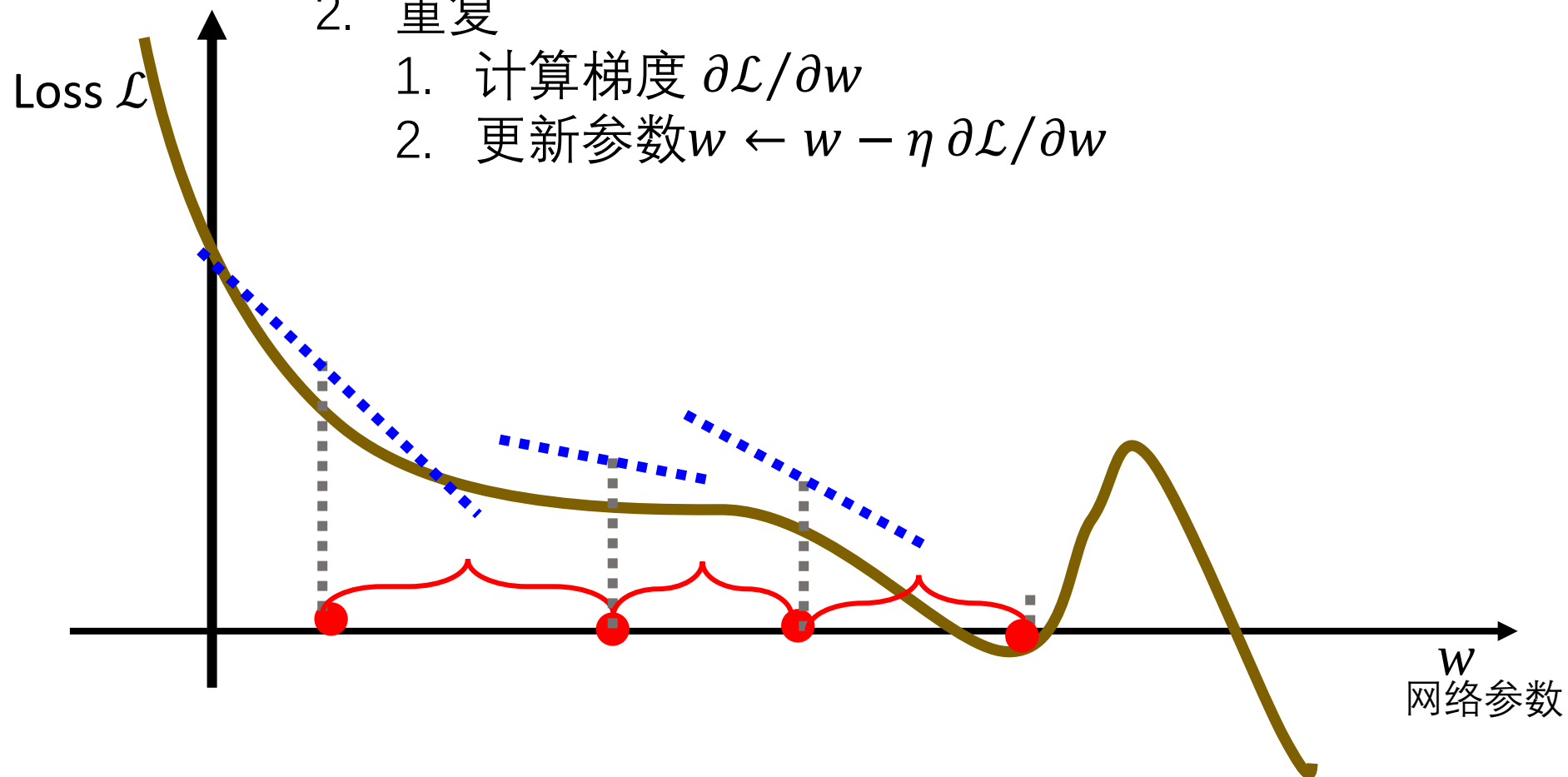
1. 初始化 w

$$\text{梯度: } \frac{\partial f(w)}{\partial w} = \lim_{\Delta w \rightarrow 0} \frac{f(w + \Delta w) - f(w)}{\Delta w}$$

2. 重复

1. 计算梯度 $\partial \mathcal{L} / \partial w$

2. 更新参数 $w \leftarrow w - \eta \partial \mathcal{L} / \partial w$



二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播

- 误差反向传播算法 (error back propagation) 主要利用损失函数来计算模型预测结果与真实结果之间误差以优化调整模型参数, 这一优化调整机制是从输出端向输入端, 由后向前递进进行。
- 损失函数梯度反方向是损失误差下降最快方向。在求取损失函数梯度时需要各个变量求偏导, 这一求取偏导过程要用到链式法则 (chain rule)。
- 在图5.6中, out 是一个神经元, 其在神经网络中完成两个任务: 1) 将相邻前序神经元传递而来的信息线性加权累加, 得到结果 x , 即 $x = \sum_i w_i \times out_i (1 \leq i \leq 3)$ 。这里 out_i 和 w_i 分别是与 out 前序相连的第 i 个神经元后向输递信息及第 i 个神经元与 out 之间的连接权重系数; 2) 使用激活函数对加权累加结果进行非线性变换。图中使用的激活函数为 sigmoid, 即 $O = \frac{1}{1+e^{-x}}$ 。可见, O 的大小就是神经网络对输入样本数据的预测输出。

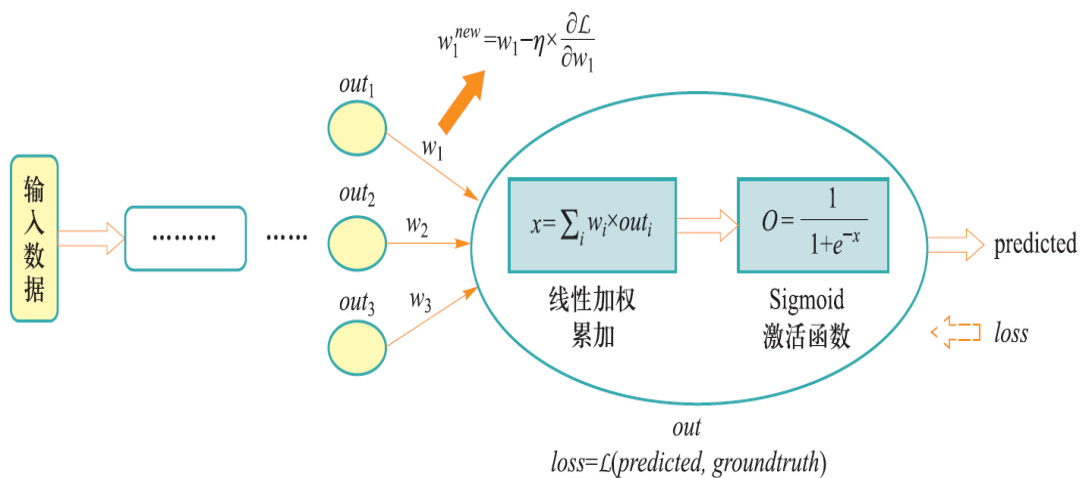


图 5.6 链式求导与模型参数更新示意

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播

模型预测结果 $\hat{O}$ 与真实结果之间的损失误差 $loss$ 由所定义的损失函数 $\mathcal{L}$ 计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数 w_i （未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

如何调整 w_1

为了使损失函数 $\mathcal{L}$ 取值减少（从而保证模型预测结果与实际结果之间的差距越来越小），需要求取损失函数 $\mathcal{L}$ 相对于 w_1 的偏导，然后按照损失函数梯度的反方向选取一个微小的增量，来调整 w_1 的取值，就能够保证损失函数取值减少。即将 w_1 变为 $w_1 - \eta \frac{d\mathcal{L}}{dw_1}$ 后，能使得损失误差减少。这里 $\frac{d\mathcal{L}}{dw_1}$ 为损失函数 $\mathcal{L}$ 对 w_1 的偏导。于是，需要求取损失函数 $\mathcal{L}$ 对变量参数 w_1 的偏导。

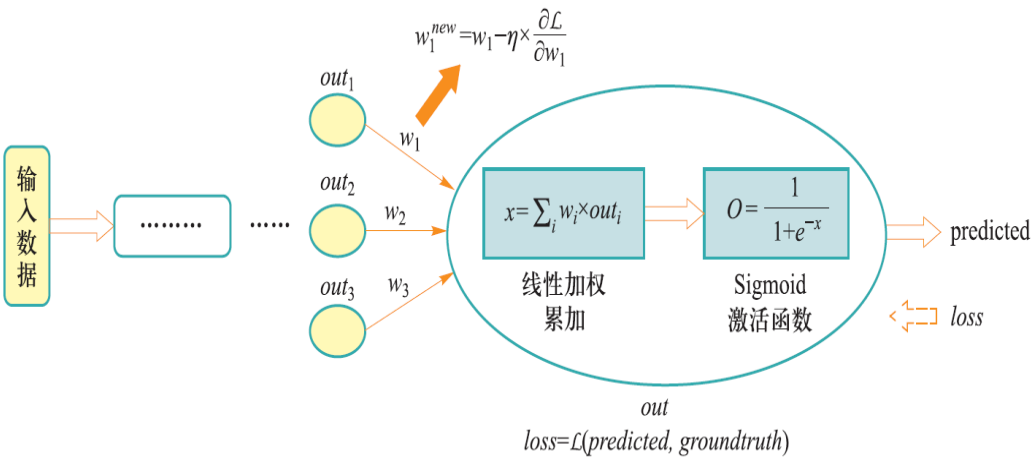


图 5.6 链式求导与模型参数更新示意

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播

模型预测结果 $\mathcal{O}$ 与真实结果之间的损失误差 $loss$ 由所定义的损失函数 $\mathcal{L}$ 计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数 w_i （未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

如何调整 w_1

由于 w_1 与加权累加函数 $\mathcal{X}$ 和sigmoid函数均有关，因此 $\frac{d\mathcal{L}}{dw_1} = \frac{d\mathcal{L}}{d\mathcal{O}} \frac{d\mathcal{O}}{d\mathcal{X}} \frac{d\mathcal{X}}{dw_1}$ 。在这个链式求导中： $\frac{d\mathcal{L}}{d\mathcal{O}}$ 与损失函数的定义有关； $\frac{d\mathcal{O}}{d\mathcal{X}}$ 是对sigmoid函数求导，结果为 $\frac{1}{1+e^{-x}} \times (1 - \frac{1}{1+e^{-x}})$ ； $\frac{d\mathcal{X}}{dw_1}$ 是加权累加函数 $w_1 \times out_1 + w_2 \times out_2 + w_3 \times out_3$ 对 w_1 求导，结果为 out_1 。

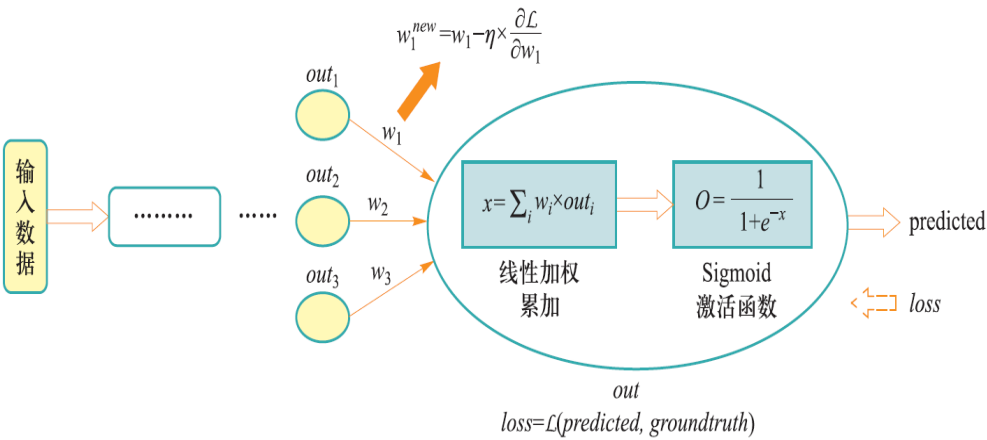


图 5.6 链式求导与模型参数更新示意

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播

模型预测结果 O 与真实结果之间的损失误差 $loss$ 由所定义的损失函数 $\mathcal{L}$ 计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数 w_i （未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

链式求导

- 链式求导实现了损失函数对某个自变量求偏导，好比将损失误差从输出端向输入端逐层传播，通过这个传播过程来更新该自变量取值。梯度下降法告诉我们，只要沿着损失函数梯度的反方向来更新参数，就可使得损失函数下降最快。
- 因此，假设损失函数 $\mathcal{L}(\theta)$ （ θ 为参数），可使用如下梯度下降算法对参数 θ 进行更新： $\theta^{new} = \theta - \eta \times \nabla_{\theta} \mathcal{L}(\theta)$ 。其中 η 也被称学习速率（learning rate），其大小会影响模型参数收敛速度， $\nabla_{\theta} \mathcal{L}(\theta)$ 是损失函数对参数 θ 求偏导结果神经网络中任意一个参数相对于损失函数的导数都可通过链式求导得到（这一求导过程可视为误差后向传播过程）。

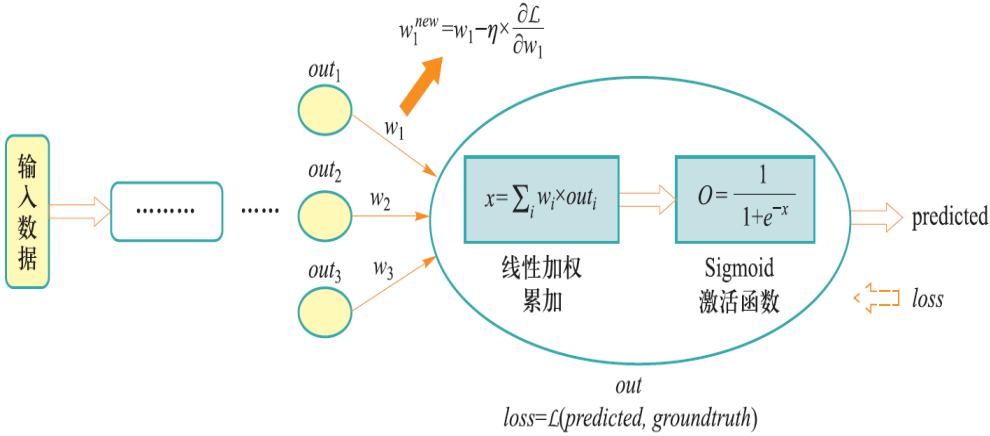


图 5.6 链式求导与模型参数更新示意

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播

模型预测结果 O 与真实结果之间的损失误差 $loss$ 由所定义的损失函数 $\mathcal{L}$ 计算得到。现在的问题是如何根据这个误差来优化调整连接权重系数 w_i （未出现的神经网络其它参数调整雷同），使得训练得到的模型尽可能输出预期结果。

通过链式求导调整调整 w_1

- 在图5.6中，参数 w_1 在下一轮迭代中的取值被调整为：
$$w_1^{new} = w_1 - \eta \times \frac{d\mathcal{L}}{dw_1} = w_1 - \eta \times \frac{d\mathcal{L}}{dO} \frac{dO}{dX} \frac{dX}{dw_1} = w_1 - \eta \times \frac{d\mathcal{L}}{dO} \times \frac{1}{1+e^{-x}} \times \left(1 - \frac{1}{1+e^{-x}}\right) \times out_1。$$
- 按照同样的方法，可调整 w_2 、 w_3 以及其它图中未显示参数的取值。经过这样的调整，在模型参数新的取值作用下，损失函数 $\mathcal{L}(\theta)$ 会以最快下降方式逐渐减少，直至减少到最小值（即全局最小值）。

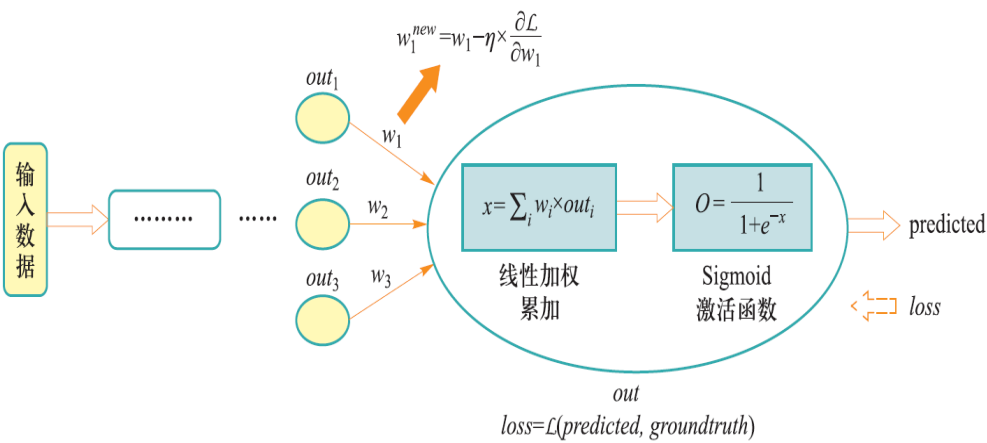


图 5.6 链式求导与模型参数更新示意

二、前馈神经网络与参数优化

神经网络参数优化：梯度下降的方法

梯度下降算法可以分为**批量梯度下降算法** (batch gradient descent)、**随机梯度下降算法** (stochastic gradient descent) 和**小批量梯度下降算法** (mini-batch gradient descent) 等类型。

- ◆ **批量梯度下降**：在整个训练集上计算损失误差 $\mathcal{L}(\theta)$ 。如果数据集较大，则会因内存容量不足而无法完成梯度计算。
- **随机梯度下降**：使用训练集中每个训练样本计算 $\mathcal{L}(\theta)$ 来分别更新参数。因为随机梯度下降每次迭代只使用一个样本，导致梯度方向具有很大的波动，因此其收敛速度通常较慢。尽管如此，随机梯度下降的随机性有助于跳出局部最优解，有时可能找到更好的解。
- **小批量梯度下降**：选取训练集中小批量样本计算 $\mathcal{L}(\theta)$ ，根据每一批量样本所得到的累加误差来更新参数，这样可保证训练过程更稳定，而且采用批量训练方法也可利用矩阵计算优势，因此其成为目前最常用的梯度下降算法

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播与参数更新例子

- 通过一个三类分类的具体例子来介绍神经网络中参数更新过程。给定一个包含输入层、一层隐藏层和输出层的多层感知机，其中隐藏层由两个神经元构成。这里为了简化介绍过程，因此只使用了一层隐藏层。网络使用 Sigmoid 函数作为神经元的激活函数，使用均方损失函数来计算网络输出值与实际值之间的误差。
- 在图 5.7 中，每个神经元完成如下两项任务：1) 对相邻前序层所传递信息进行线性加权累加；2) 对加权累加结果进行非线性变换。假设样本数据输入为 (x_1, x_2, x_3) ，其标注信息为 (y_1, y_2, y_3) 。在三类分类问题中，对于输入数据 (x_1, x_2, x_3) ， y_1 、 y_2 和 y_3 中只有一个取值为 1、其余两个取值为 0。

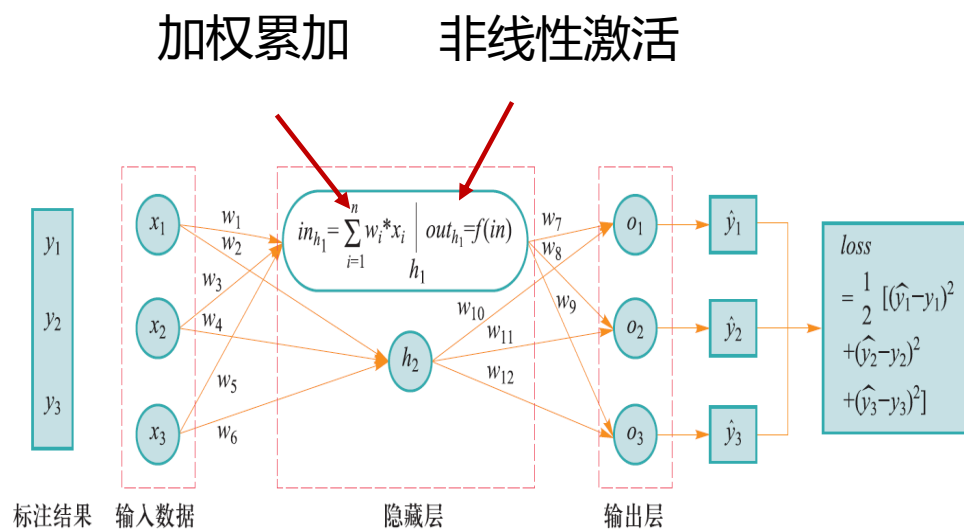


图 5.7 包含一个隐藏层的前馈神经网络

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播与参数更新例子

表 5.3 图 5.7 中神经网络各单元的计算结果

单元	相邻前序神经元所传递信息的线性累加	非线性变换
h_1	$In_{h_1} = w_1 \times x_1 + w_3 \times x_2 + w_5 \times x_3$	$Out_{h_1} = \text{sigmoid}(In_{h_1})$
h_2	$In_{h_2} = w_2 \times x_1 + w_4 \times x_2 + w_6 \times x_3$	$Out_{h_2} = \text{sigmoid}(In_{h_2})$
o_1	$In_{o_1} = w_7 \times Out_{h_1} + w_{10} \times Out_{h_2}$	$\hat{y}_1 = Out_{o_1} = \text{sigmoid}(In_{o_1})$
o_2	$In_{o_2} = w_8 \times Out_{h_1} + w_{11} \times Out_{h_2}$	$\hat{y}_2 = Out_{o_2} = \text{sigmoid}(In_{o_2})$
o_3	$In_{o_3} = w_9 \times Out_{h_1} + w_{12} \times Out_{h_2}$	$\hat{y}_3 = Out_{o_3} = \text{sigmoid}(In_{o_3})$

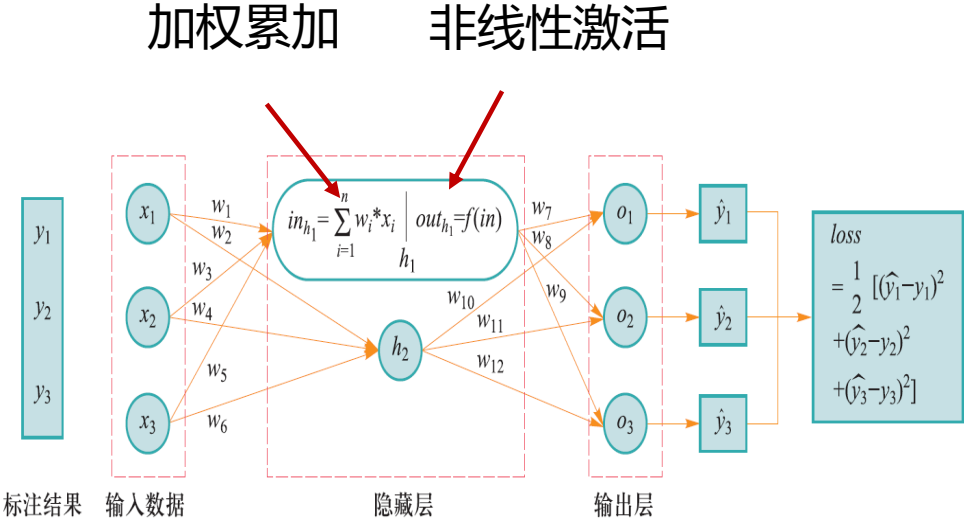


图 5.7 包含一个隐藏层的前馈神经网络

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播与参数更新例子---计算误差

- 一旦神经网络在当前参数下给出了预测结果 $(\hat{y}_1, \hat{y}_2, \hat{y}_3)$ 后，通过均方误差损失函数来计算模型预测值与真实值 (y_1, y_2, y_3) 之间误差，记为 $loss = \frac{1}{3} \sum_{i=1}^3 (\hat{y}_i - y_i)^2$ 。
- 接下来通过梯度下降和误差反向传播方法，沿着损失函数梯度的反方向来更改参数变量取值，使得损失函数快速下降到其最小值。由于损失函数对 $w_7 \sim w_{12}$ 的偏导计算相似，在此以 w_7 为例来介绍如何更新 w_7 这一参数取值。

记损失函数为 $\mathcal{L}(w)$ ， $\mathcal{L}(w) = \frac{1}{3} \sum_{i=1}^3 (\hat{y}_i - y_i)^2$ 。

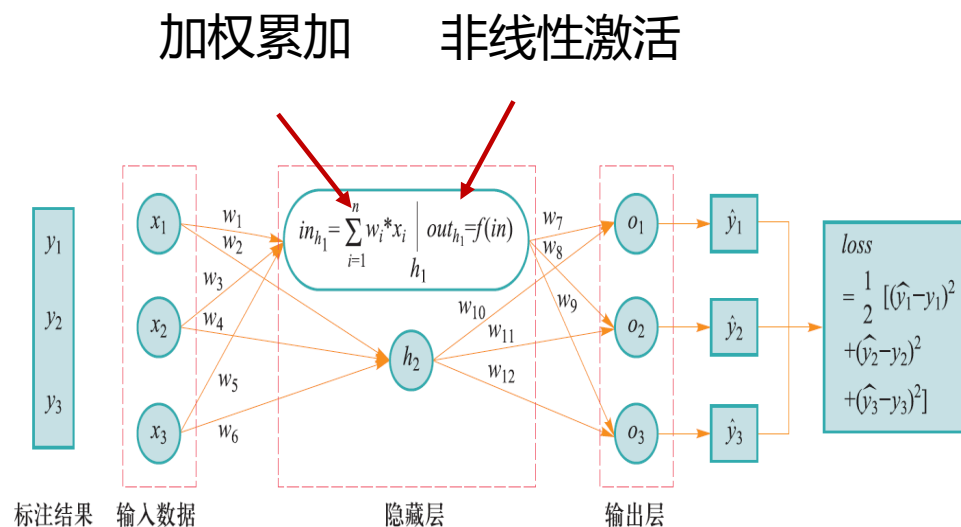


图 5.7 包含一个隐藏层的前馈神经网络

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播与参数更新例子---计算误差

损失函数 $\mathcal{L}$ 对参数 w_7 的偏导可如下计算：

$$\delta_7 = \frac{\partial \mathcal{L}}{\partial w_7} = \frac{\partial \mathcal{L}}{\partial \hat{y}_1} * \frac{\partial \hat{y}_1}{\partial In_{o_1}} * \frac{\partial In_{o_1}}{\partial w_7}$$

上述公式中每一项结果如下：

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_1} = \hat{y}_1 - y_1$$

$$\frac{\partial \hat{y}_1}{\partial In_{o_1}} = \hat{y}_1 * (1 - \hat{y}_1)$$

$$\frac{\partial In_{o_1}}{\partial w_7} = Out_{h_1}$$

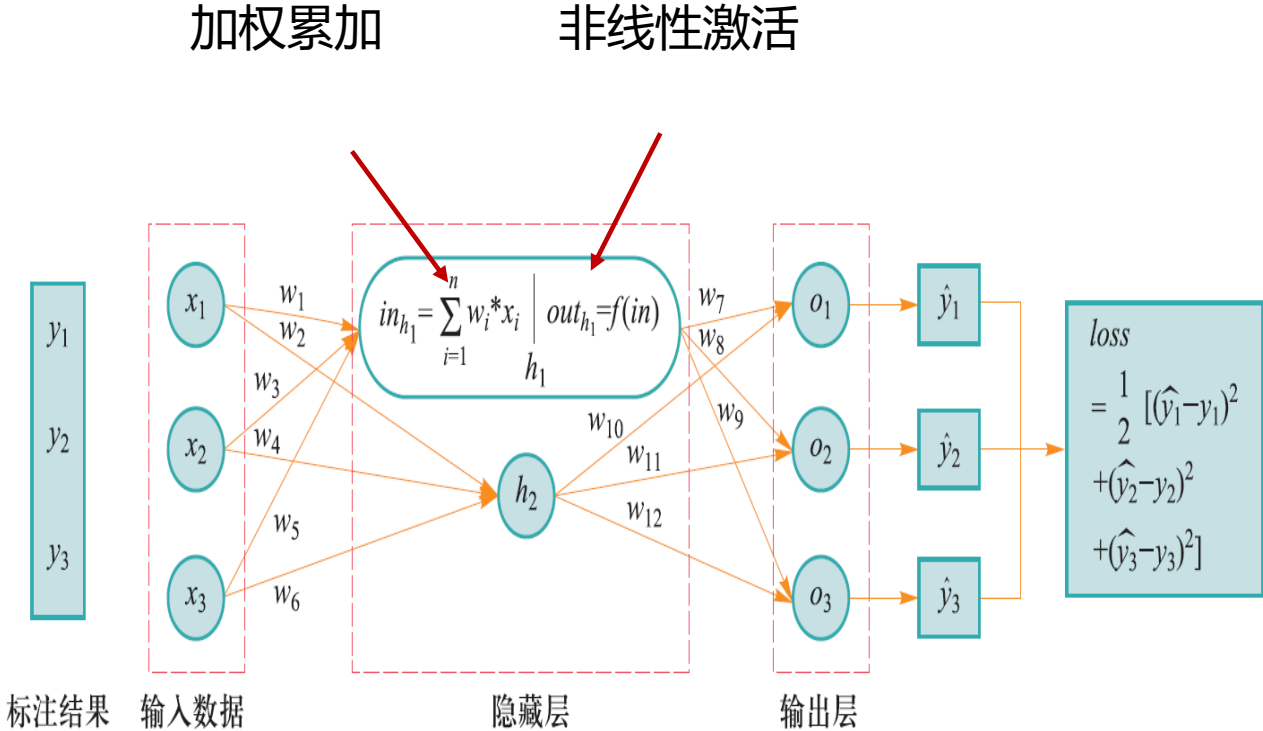


图 5.7 包含一个隐藏层的前馈神经网络

二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播与参数更新例子---计算误差

在得到 $w_7 \sim w_{12}$ 相对于损失函数的偏导结果后，利用链式求导法则，可计算得到损失函数相对于 $w_1 \sim w_6$ 的偏导结果。下面以 w_1 为例介绍损失函数 $\mathcal{L}$ 对 w_1 的偏导 δ_1 。

加权累加 非线性激活

$$\begin{aligned} \delta_1 &= \frac{\partial \mathcal{L}}{\partial w_1} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{y}_1} * \frac{\partial \hat{y}_1}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \hat{y}_2} * \frac{\partial \hat{y}_2}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \hat{y}_3} * \frac{\partial \hat{y}_3}{\partial w_1} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{y}_1} * \frac{\partial \hat{y}_1}{\partial \text{In}_{o_1}} * \frac{\partial \text{Out}_{h_1}}{\partial \text{In}_{h_1}} * \frac{\partial \text{In}_{h_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \hat{y}_2} * \frac{\partial \hat{y}_2}{\partial \text{In}_{o_2}} * \frac{\partial \text{Out}_{h_1}}{\partial \text{In}_{h_1}} * \frac{\partial \text{In}_{h_1}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial \hat{y}_3} * \frac{\partial \hat{y}_3}{\partial \text{In}_{o_3}} \\ &\quad * \frac{\partial \text{Out}_{h_1}}{\partial \text{In}_{h_1}} * \frac{\partial \text{In}_{h_1}}{\partial w_1} \\ &= \left(\frac{\partial \mathcal{L}}{\partial \hat{y}_1} * \frac{\partial \hat{y}_1}{\partial \text{In}_{o_1}} * \frac{\partial \text{In}_{o_1}}{\partial \text{Out}_{h_1}} + \frac{\partial \mathcal{L}}{\partial \hat{y}_2} * \frac{\partial \hat{y}_2}{\partial \text{In}_{o_2}} * \frac{\partial \text{In}_{o_2}}{\partial \text{Out}_{h_1}} + \frac{\partial \mathcal{L}}{\partial \hat{y}_3} * \frac{\partial \hat{y}_3}{\partial \text{In}_{o_3}} * \frac{\partial \text{In}_{o_3}}{\partial \text{Out}_{h_1}} \right) * \frac{\partial \text{Out}_{h_1}}{\partial \text{In}_{h_1}} * \frac{\partial \text{In}_{h_1}}{\partial w_1} \\ &= (\delta_7 + \delta_8 + \delta_9) * \frac{\partial \text{Out}_{h_1}}{\partial \text{In}_{h_1}} * \frac{\partial \text{In}_{h_1}}{\partial w_1} \end{aligned}$$

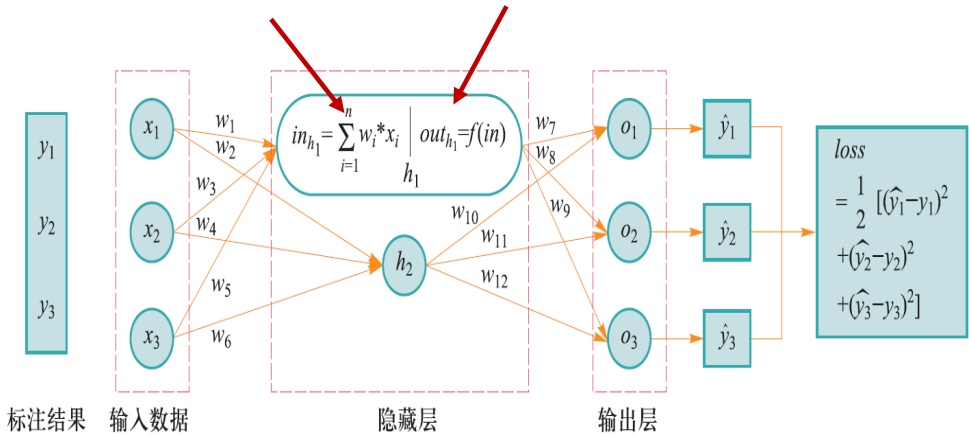
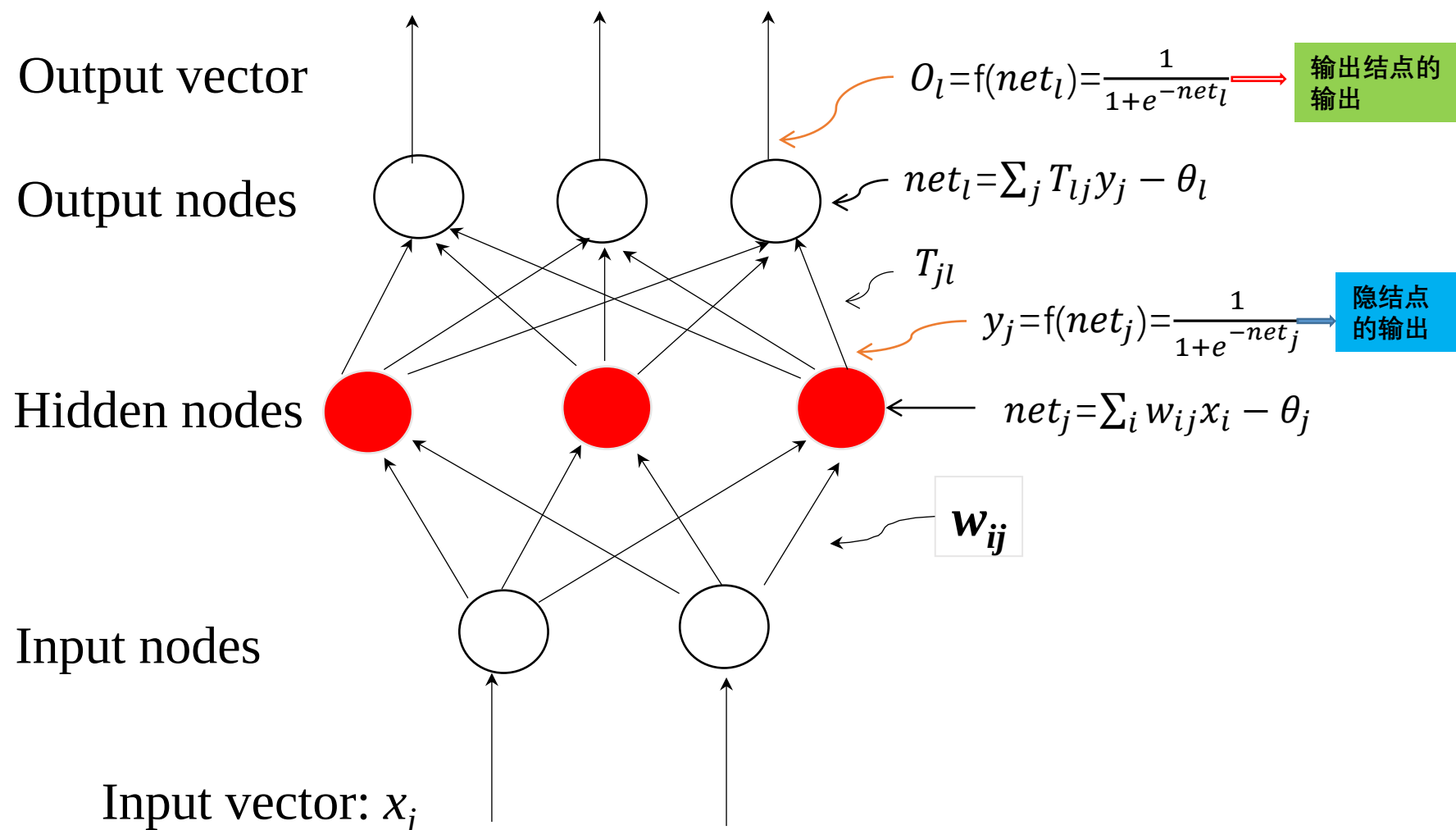


图 5.7 包含一个隐藏层的前馈神经网络

在计算得到所有参数相对于损失函数 $\mathcal{L}$ 的偏导结果后，利用梯度下降算法，通过 $w_i^{new} = w_i - \eta * \delta_i$ 来更新参数取值。然后不断迭代，直至模型参数收敛，此时损失函数减少到其最小值。

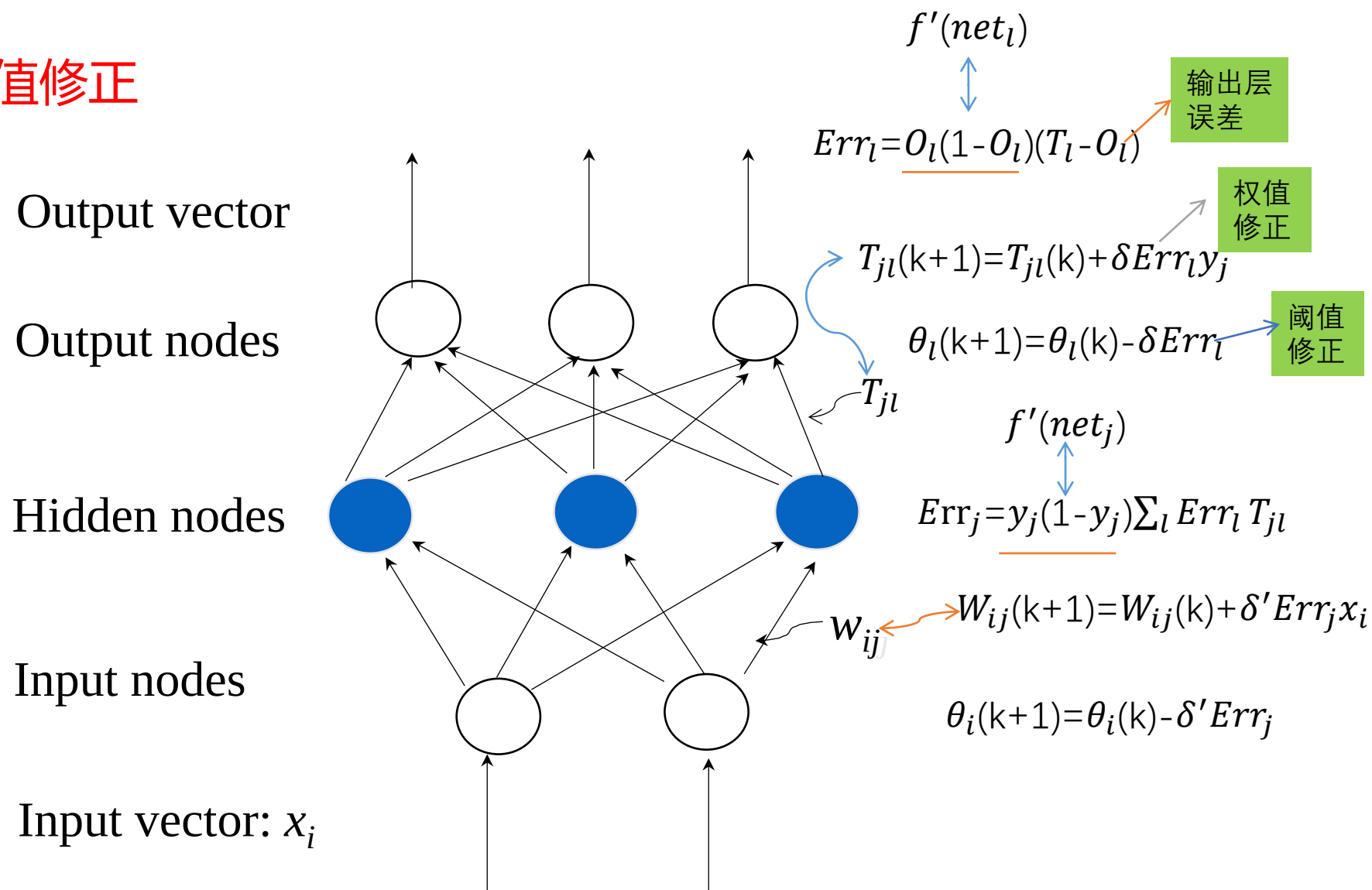
二、前馈神经网络与参数优化

➤ 信号正向传播



二、前馈神经网络与参数优化

➤ 反向传播权值修正



二、前馈神经网络与参数优化

神经网络参数优化：误差反向传播算法

- 1) 初始化网络及学习参数，即将隐含层和输出层各节点的连接权值、神经元阈值赋予 $[-1, 1]$ 或 $[-0.5, 0.5]$ 区间的一个小的随机数。
- 2) 提供训练模式，即从训练模式集合中选出一个训练模式，将其输入模式和期望输出送入网络。
- 3) 正向传播过程，即对给定的输入模式，从第一隐含层开始，**计算网络的输出模式**，并把得到的输出模式与期望模式比较，若有误差，则执行第4)步；否则，返回第2)步，提供下一个训练模式；
- 4) 反向传播过程，即从输出层反向计算到第一隐含层，**逐层修正各单元的连接权值**。
- 5) 返回第2)步，对训练模式集中的每一个训练模式重复第2)到第3)步，直到满足结束条件为止。

卷积神经网络---概论

- ▶ 卷积神经网络 (Convolutional Neural Networks, CNN)
- ▶ 一种前馈神经网络
- ▶ 受生物学上感受野 (Receptive Field) 的机制而提出的
 - ▶ 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。
- ▶ 卷积神经网络有三个结构上的特性：
 - ▶ 局部连接
 - ▶ 权重共享
 - ▶ 空间或时间上的次采样

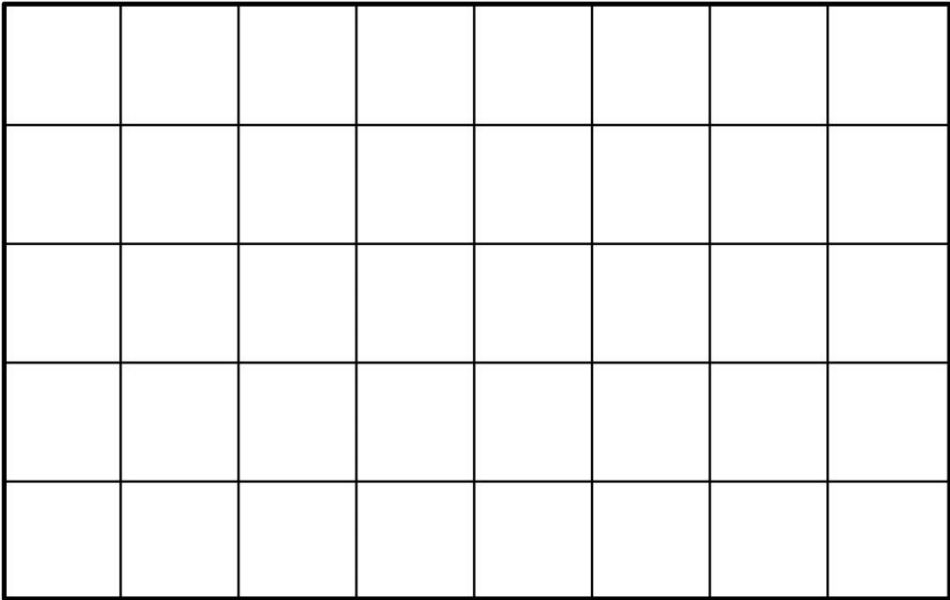
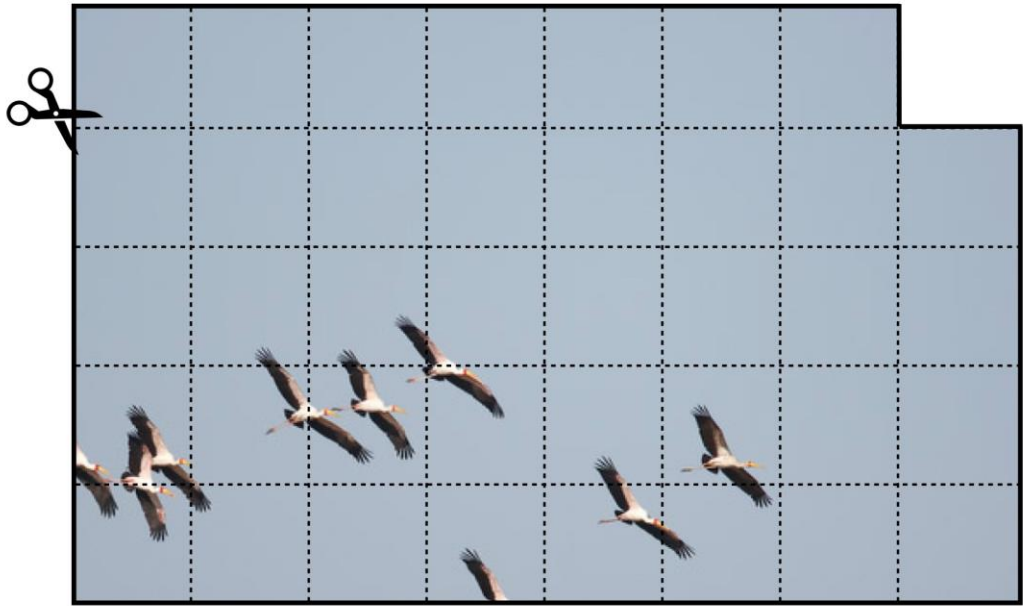
卷积神经网络---概论

一个例子



卷积神经网络---概论

一个例子



卷积神经网络---概论

一个例子



							Bird



卷积神经网络---概论

一个例子

- 简单且理想的情况
 - 图片中的每个物体都恰好包在一个块中
 - 物体大小和块的大小接近

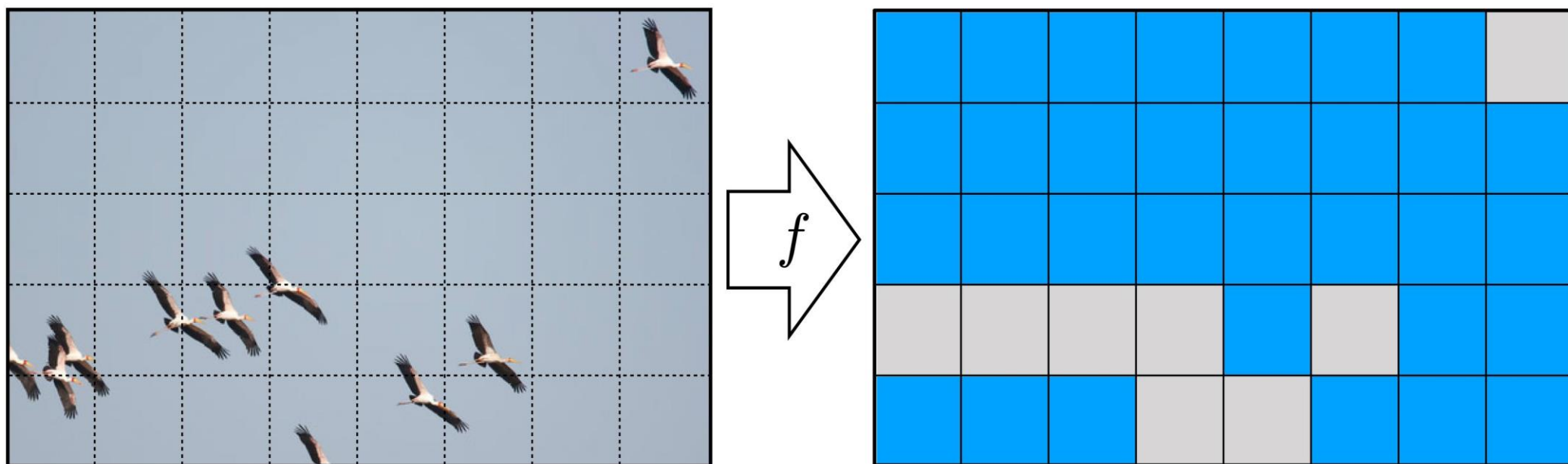


Sky	Sky	Sky	Sky	Sky	Sky	Sky	Bird
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Bird	Bird	Bird	Sky	Bird	Sky	Sky	Sky
Sky	Sky	Sky	Bird	Sky	Sky	Sky	Sky

卷积神经网络---概论

问题

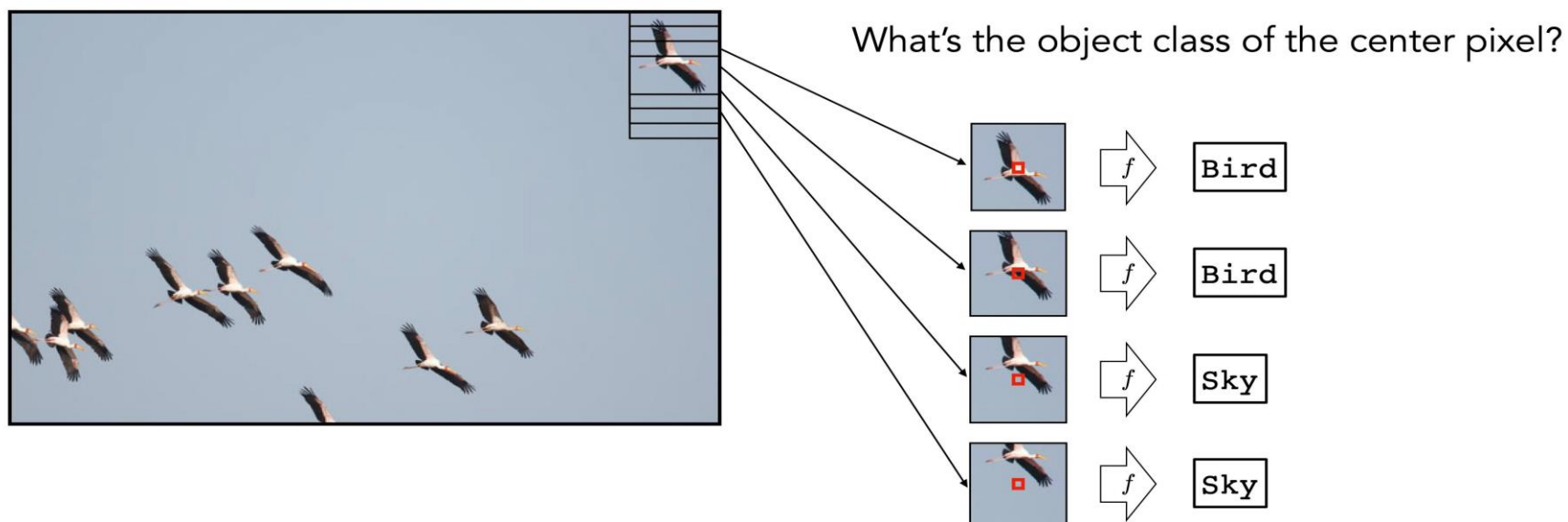
- 如果图片中的物体比块大得多该怎么办?
- 如果一个物体处于多个块中怎么办?



卷积神经网络---概论

如何更加有效地提取局部特征?

- 改进小块的划分方式:
 - 以每个像素为中心划分出一系列相互重叠的稠密块
 - 相邻块之间的偏移均为一个像素
 - 将每个块的预测结果作为这个块中心像素点的预测结果



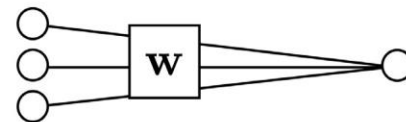
卷积神经网络---卷积

卷积

- 如何处理每个小块?
 - 卷积核: 权重矩阵 W
 - 卷积操作: 加权求和
- 权重共享
- 为何叫做“卷积” ?
 - 无需翻转, 本质是“互相关”
 - 早期受信号处理领域启发



W computes a weighted sum of all pixels in the patch



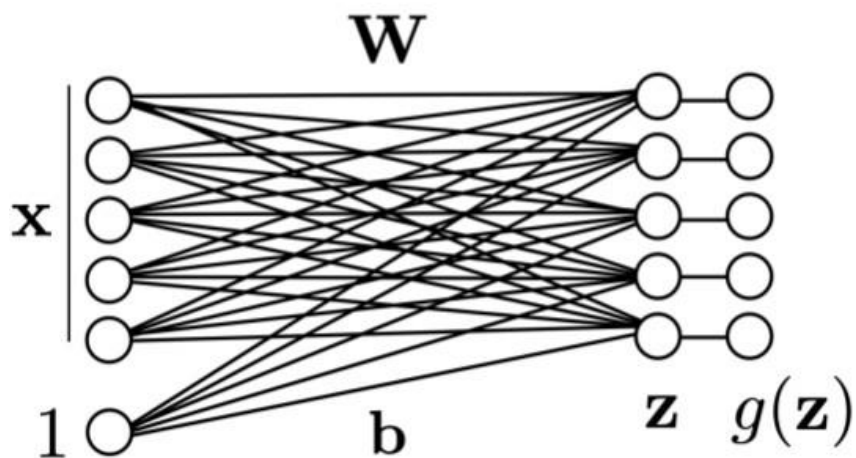
W is a **convolutional kernel** applied to the full image!

卷积神经网络---卷积

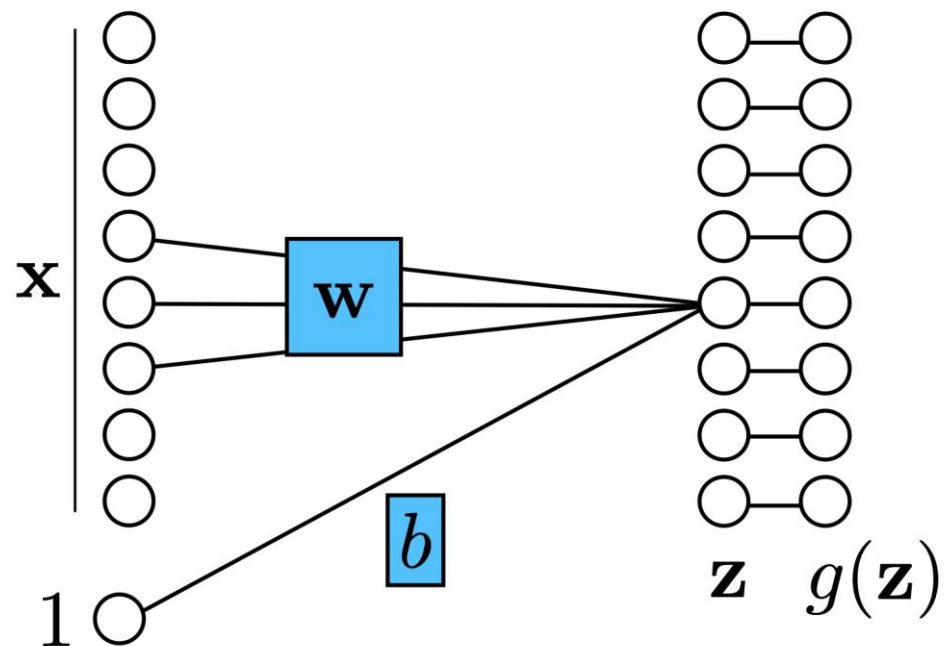
全连接层 vs. 局部连接层

全连接： 相关每个神经元都与前一层所有神经元相关。

Fully-connected (fc) layer

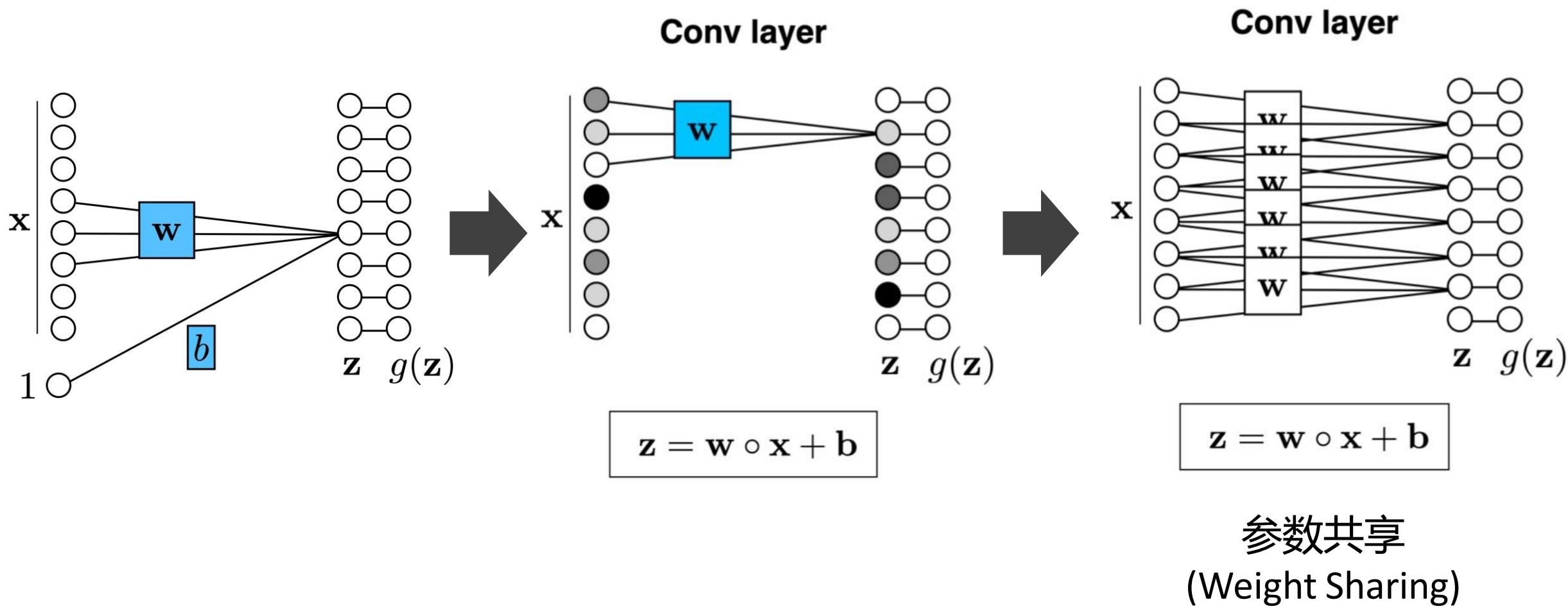


局部连接： 输出仅和个别输入分量相关权重/参数共享：局部参数 $\mathbf{w}$ 是共享的



卷积神经网络---卷积

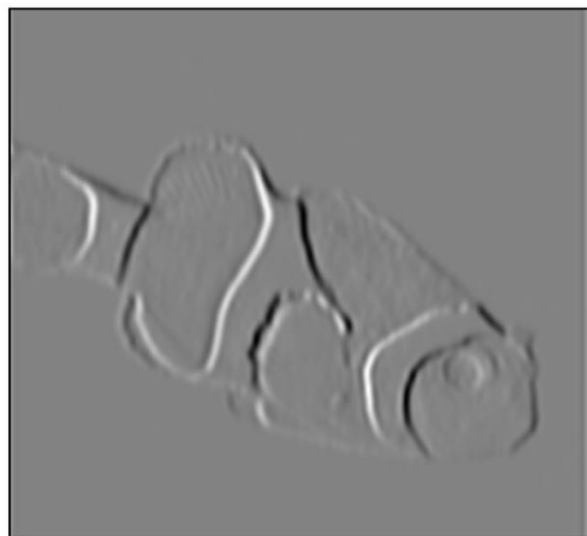
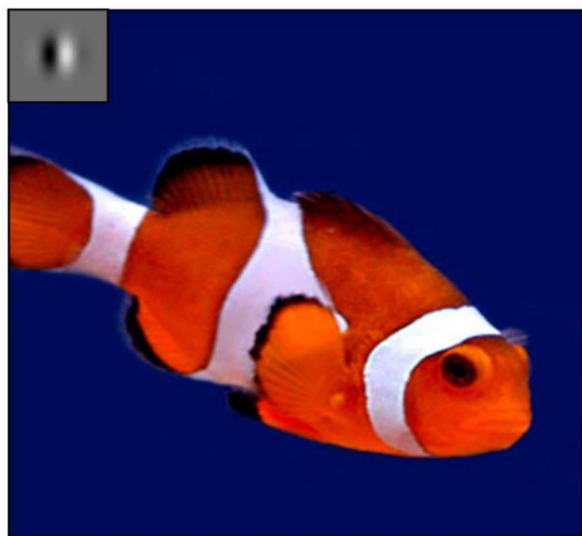
从局部连接到卷积层



卷积神经网络---卷积

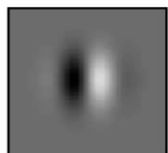
卷积核: Conv2D

- 线性、平移不变的计算操作 (算子, Operator) , 也成为滤波器filter



示例: 一个提取边缘信息的卷积核

filter

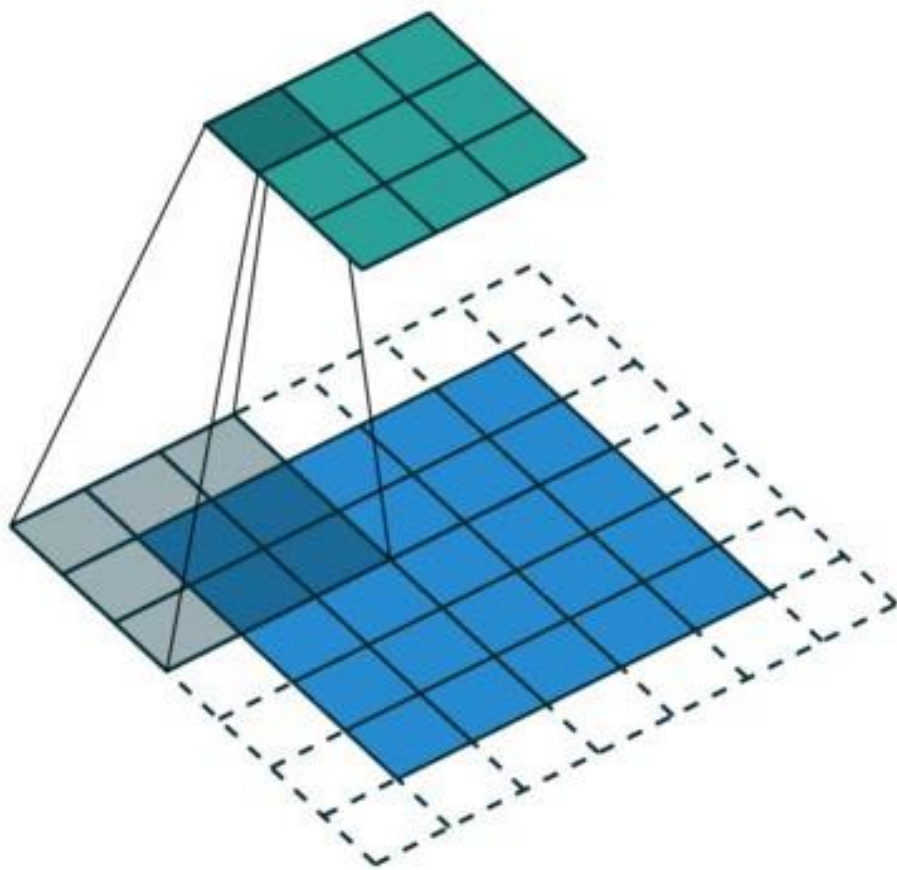


$$\mathbf{x}_{\text{out}}[n, m] = b + \sum_{k, l=-N}^N \mathbf{w}[k, l] \mathbf{x}_{\text{in}}[n - k, m - l]$$

计算公式

卷积神经网络---卷积

卷积计算过程



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

卷积神经网络---卷积

卷积：填充 (Padding)

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix		
0	-1	0
-1	5	-1
0	-1	0

320				

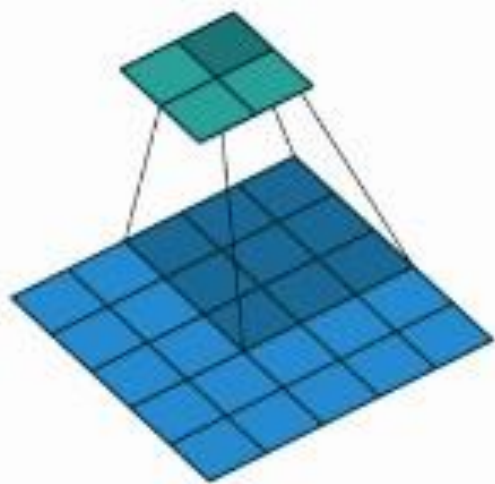
Output Matrix

$$\begin{aligned} &0 * 0 + 0 * -1 + 0 * 0 \\ &+ 0 * -1 + 105 * 5 + 102 * -1 \\ &+ 0 * 0 + 103 * -1 + 99 * 0 = 320 \end{aligned}$$

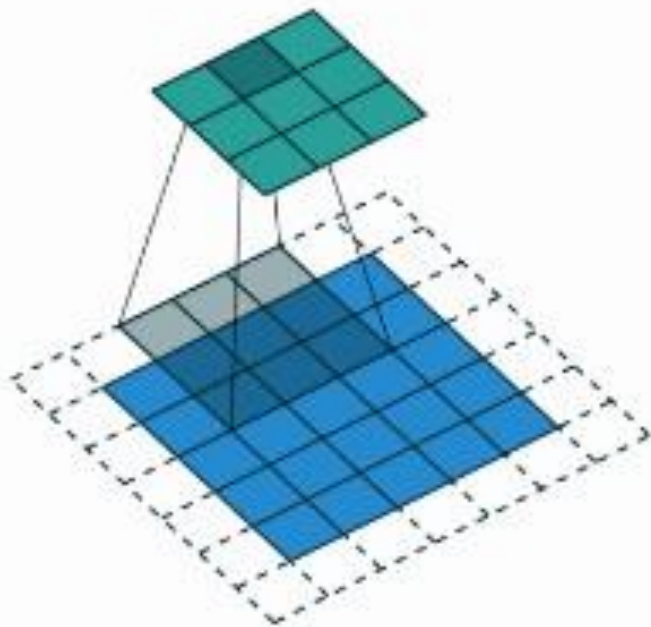
Convolution with horizontal and
vertical strides = 1

卷积神经网络---卷积

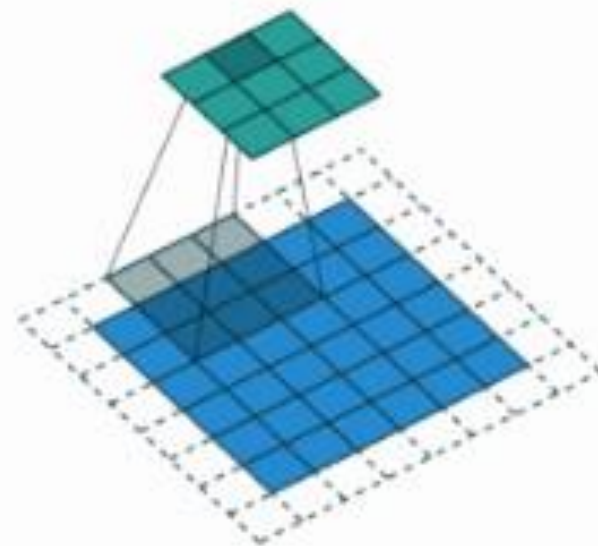
卷积：步长 (Stride)



No padding, strides



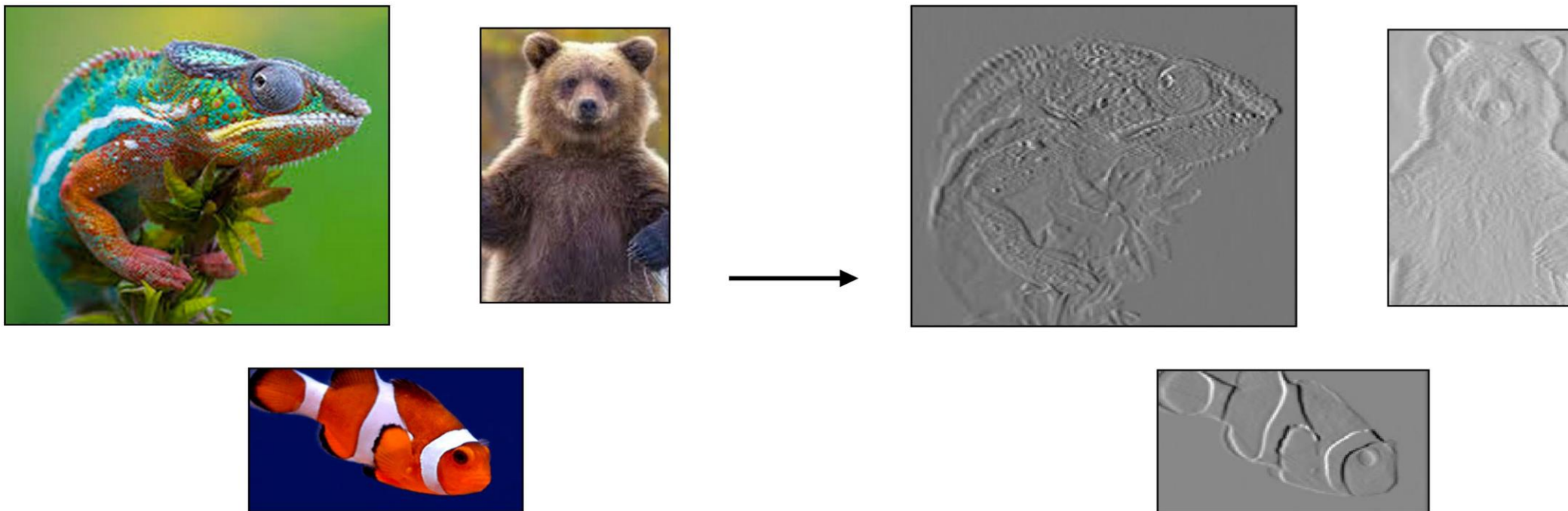
Padding, strides



Padding, strides (odd)

卷积神经网络---卷积

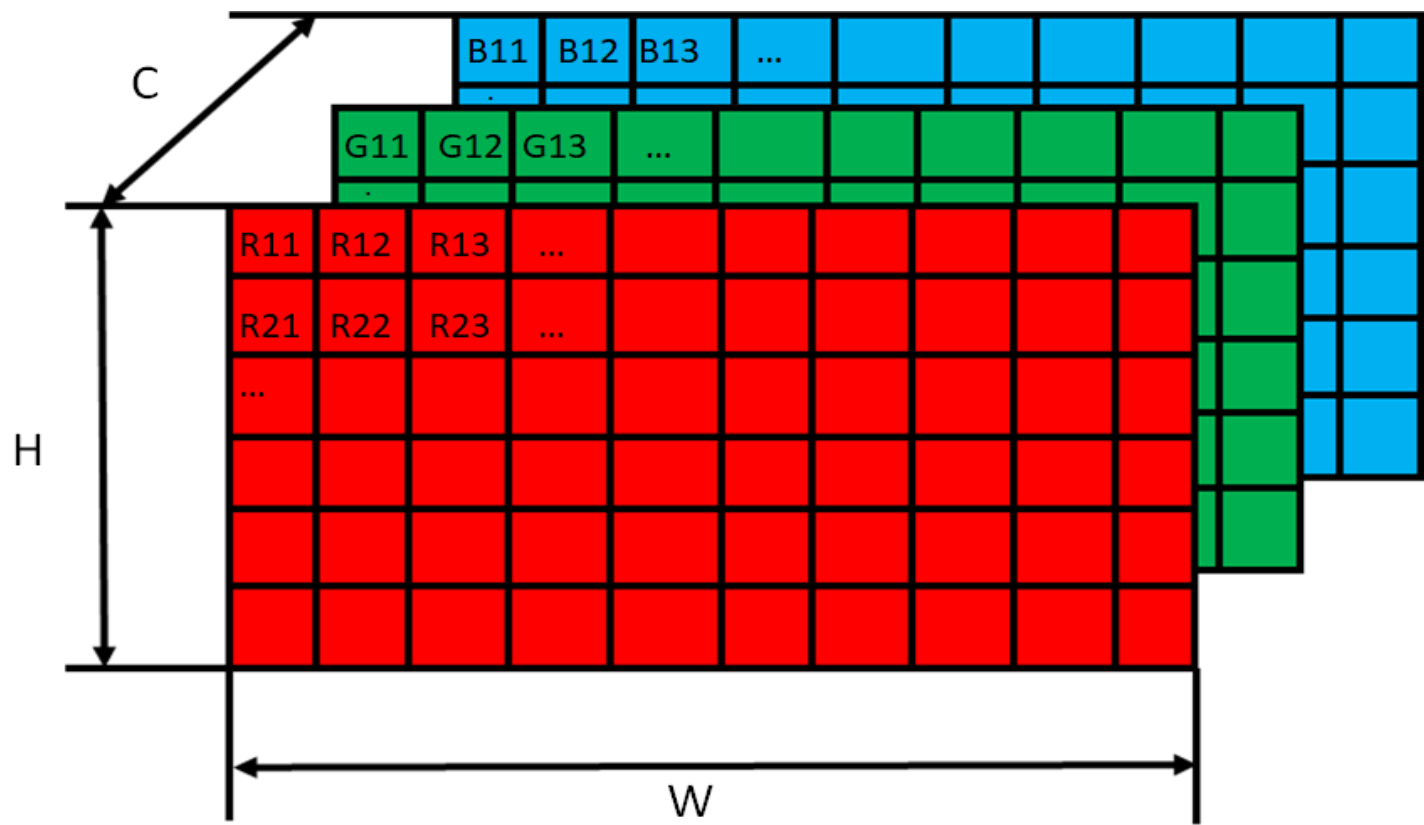
同样大小的卷积核可以应用于不同尺寸图像



Conv layers can be applied to arbitrarily-sized inputs

卷积神经网络---卷积

RGB图像格式



HWC: [R11 G11 B11 R12 G12 B21 ...]

CHW: [R11 R12 R13 ...R21 R22 R23 ... G11 G12 G13 ... G21 G22 G23 ... B11 B12 ...]

卷积神经网络---卷积

卷积算子：多输入通道&单输出通道

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+

+

+ 1 = -25



Bias = 1

-25				...
				...
				...
				...
...	...	...	...	...

卷积神经网络---池化

- ◆ **最大池化 (max pooling)**：从输入特征图的某个区域子块中选择值最大的像素点作为最大池化结果 (见图5.12)。
- ◆ **平均池化 (average pooling)**：计算区域子块所包含所有像素点的均值，将均值作为平均池化结果。
- ◆ **k-max池化 (k-max pooling)**：对输入特征图区域子块中的像素点取前k个最大值。如图5.13所示，从包含4个取值的每一列中选取前2个最大值就得到了k-max池化结果。

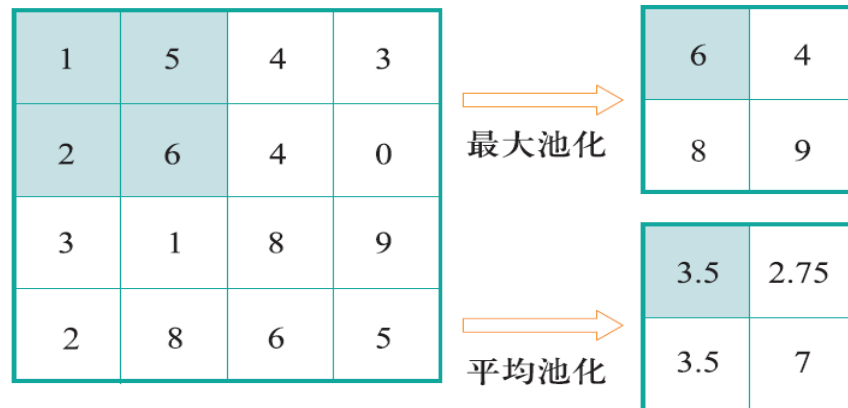


图 5.12 最大池化与平均池化操作示意

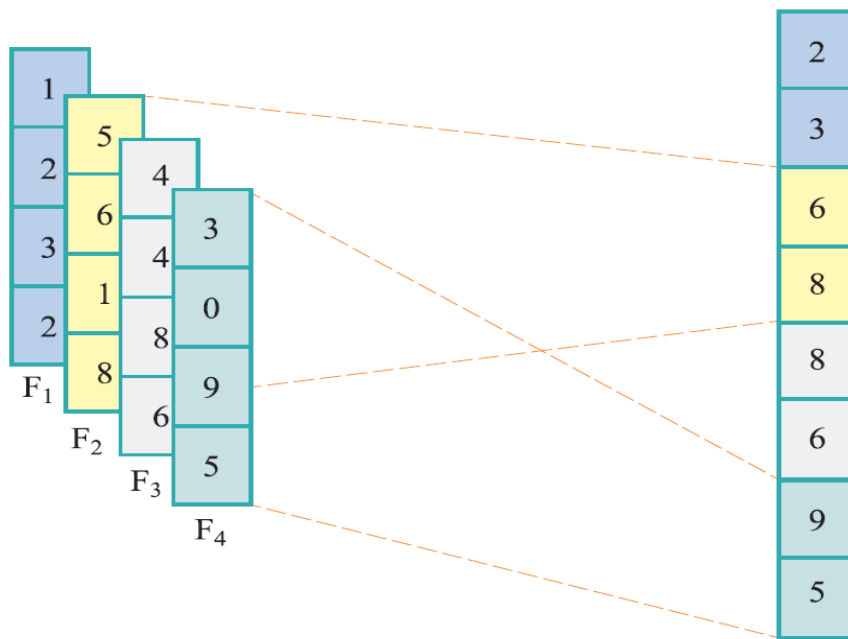


图 5.13 k-max 池化操作示意

卷积神经网络---网络结构

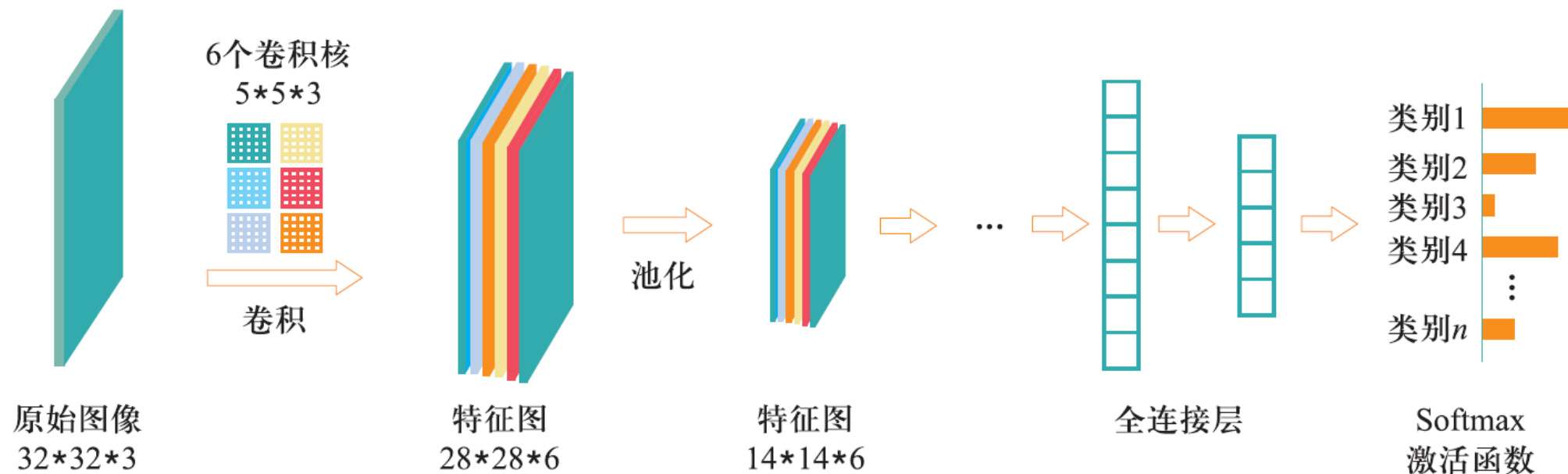
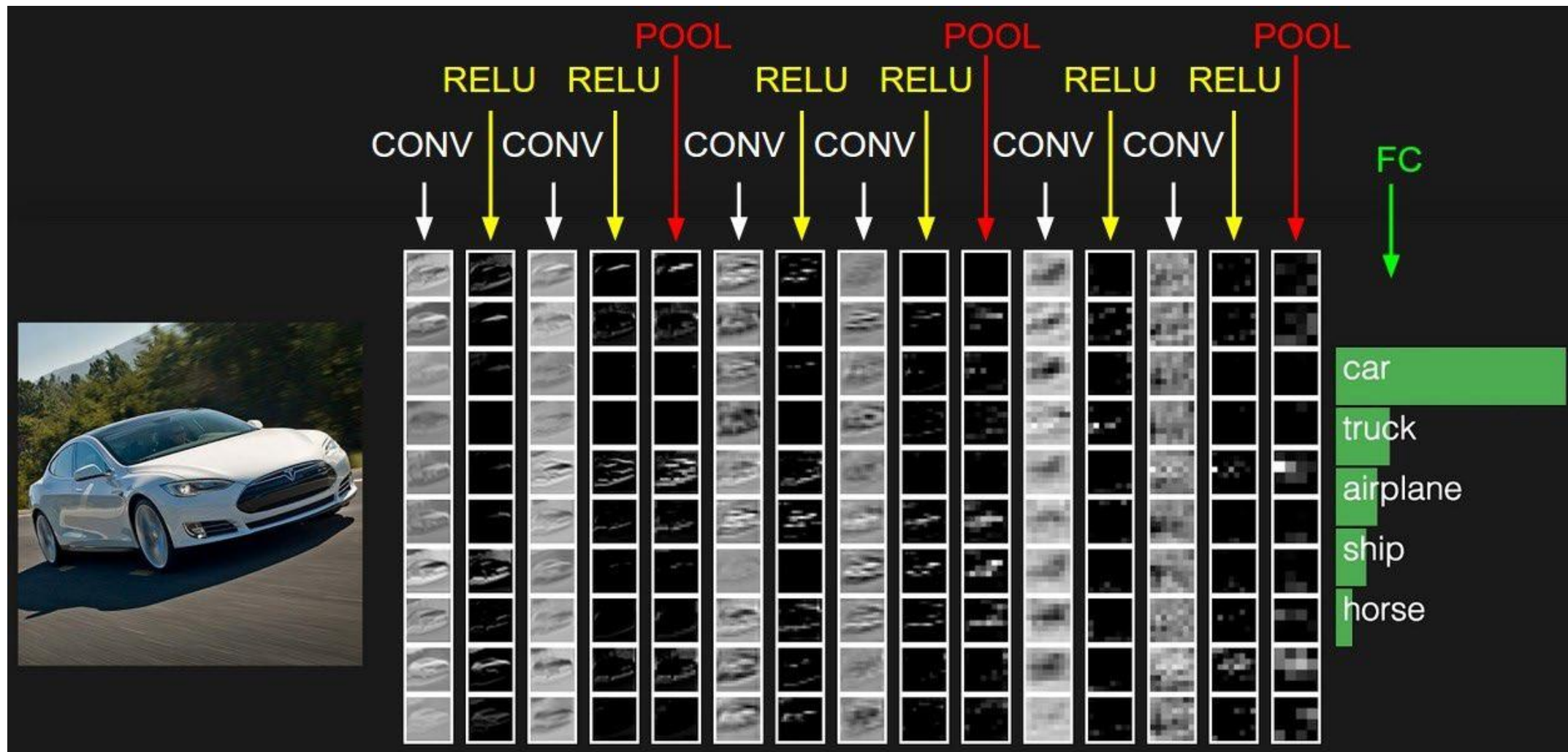


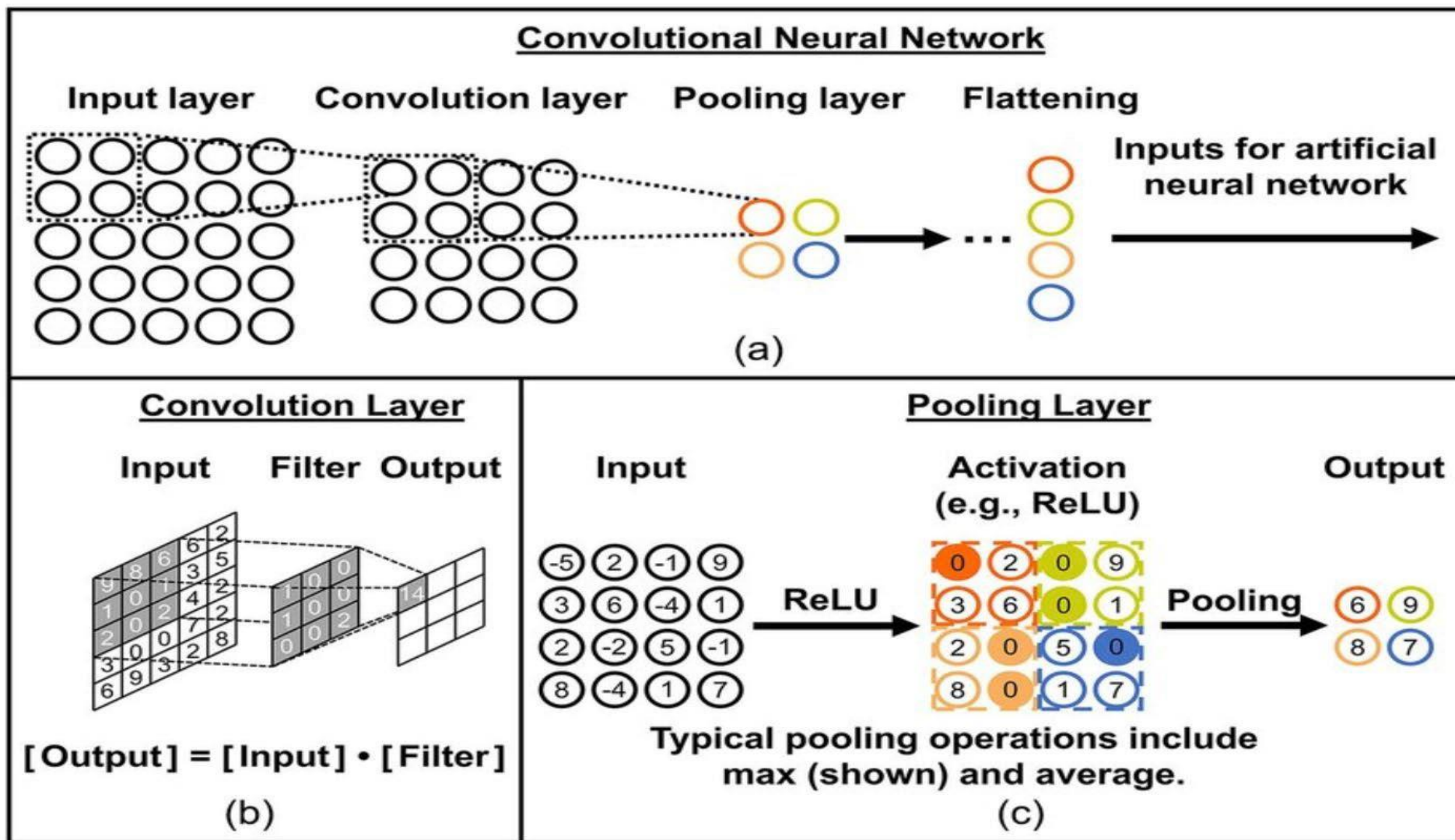
图 5.14 基于卷积神经网络的图像分类示意

如图5.14所示，对于 $32 \times 32 \times 3$ 的彩色图像，先用6个 $5 \times 5 \times 3$ 的卷积核对其进行卷积操作（步长为1，无填充），得到6个 28×28 的卷积结果（特征图），然后以 2×2 大小进行池化，得到6个 14×14 大小的特征图。最后通过若干个全连接层得到原始图像的向量表达，再对这个向量表达通过softmax函数进行分类识别，得到输入图像隶属于 n 个类别的概率。

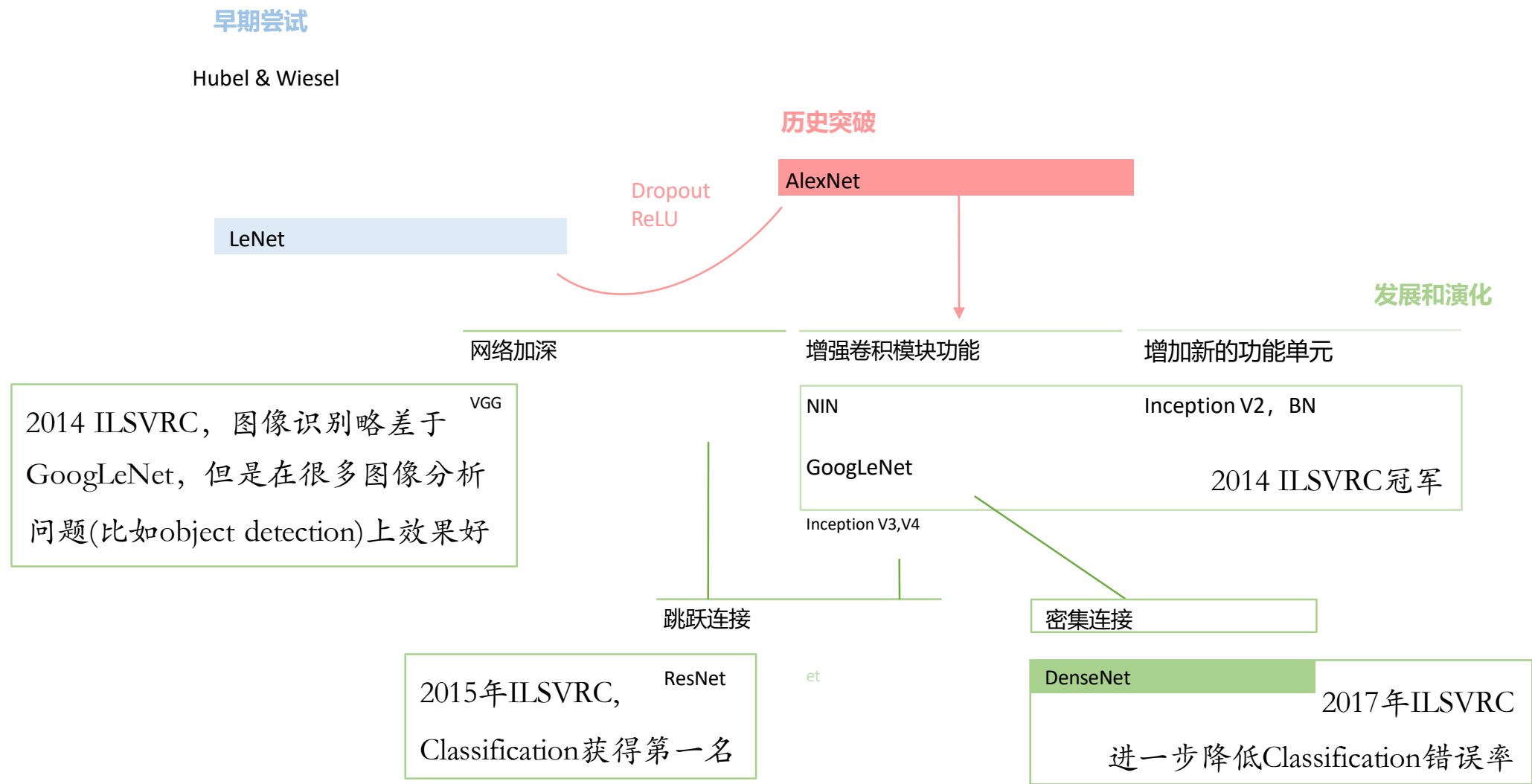
卷积神经网络---网络结构



卷积神经网络---网络结构

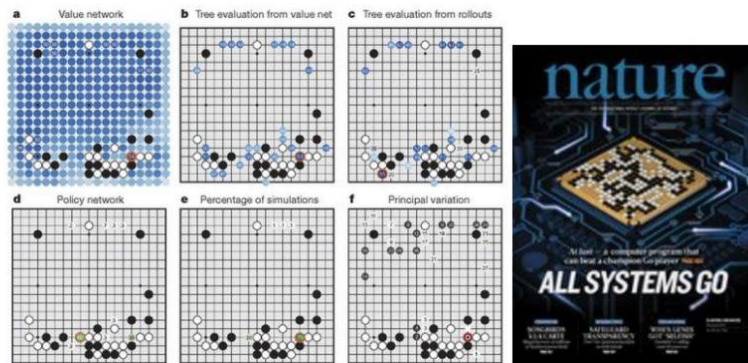


卷积神经网络---发展



卷积神经网络---应用

AlphaGo



The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

policy network:

[19x19x48] Input

CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]

CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)

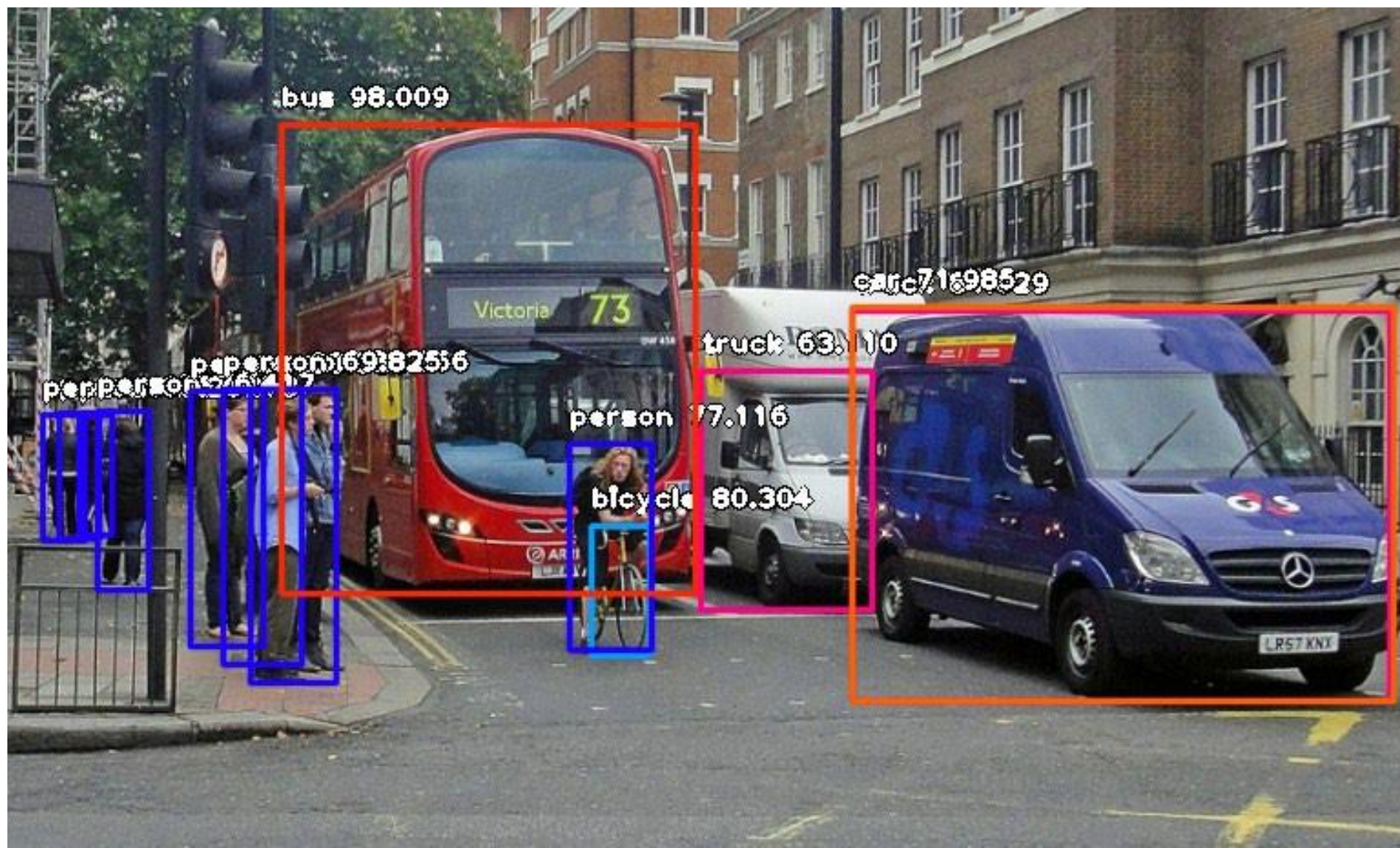
分布式系统: 1202 个CPU 和176 块GPU

单机版: 48 个CPU 和8 块GPU

走子速度: 3 毫秒-2 微秒

卷积神经网络---应用

目标检测 (Object Detection)



卷积神经网络---应用

Mask RCNN实例分割



Figure 4. More results of **Mask R-CNN** on COCO test images, using ResNet-101-FPN and running at 5 fps, with 35.7 mask AP (Table 1).

卷积神经网络---应用

OCR文字识别



卷积神经网络---应用

图像生成



卷积神经网络---应用

Deep Dream超现实的视觉效果



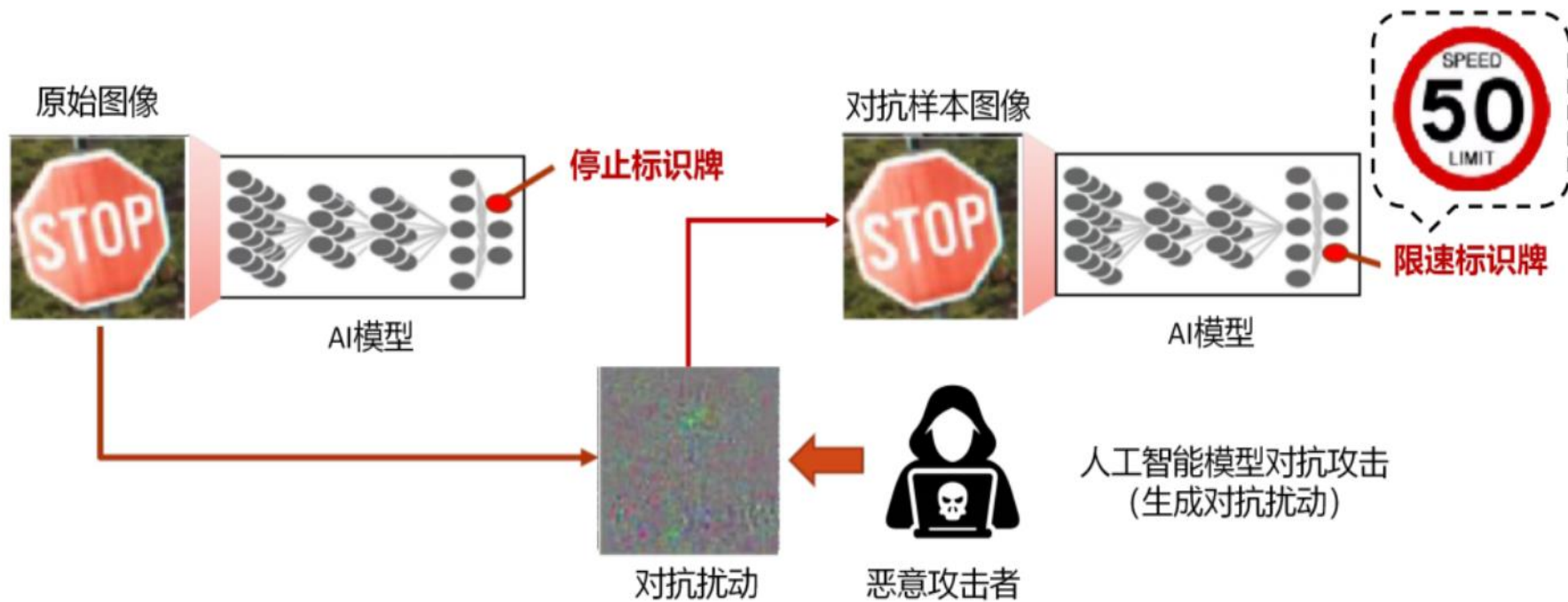
卷积神经网络---应用

画风迁移



卷积神经网络---应用

对抗样本



循环神经网络---概念

- **循环神经网络** (Recurrent Neural Network, RNN) 是处理序列数据（如文本句子、视频帧等）时所采用的网络结构。前面的前馈神经网络或卷积神经网络所需要处理的输入数据一次性给定，难以处理存在前后依赖关系的数据。
- 1990年，Jeffrey L.Elman提出循环神经网络的最初概念框架，循环神经网络的本质是希望模拟人所具有的**记忆能力**，在学习过程中记住部分已经出现的信息，并利用所记住的信息影响后续结点输出。
- 循环神经网络在自然语言处理，例如语音识别、情感分析、机器翻译等领域有重要应用。

循环神经网络---概念

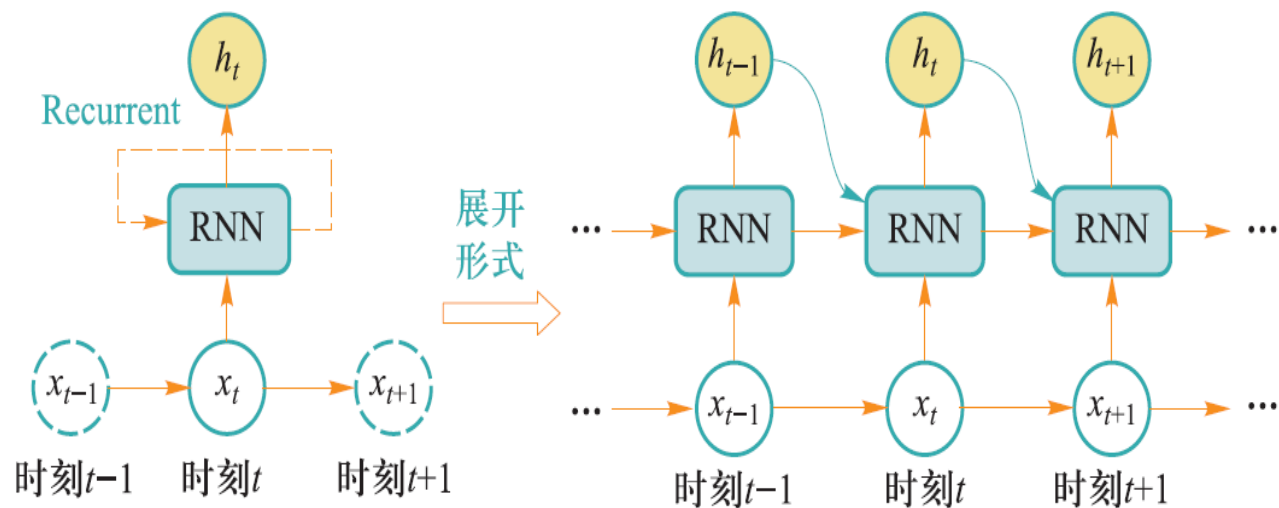


图 5.15 循环神经网络结构及其展开形式示意

图5.15 循环神经网络的结构图。对于一个序列数据，在每一时刻 t ，循环神经网络单元会读取当前输入数据 x_t 和前一时刻输入数据 x_{t-1} 所对应的隐式编码结果 h_{t-1} ，一起生成 t 时刻的隐式编码结果 h_t 。接着将 h_t 后传，去参与生成 $t + 1$ 时刻输入数据 x_{t+1} 的隐式编码 h_{t+1} 。如此循环处理，直至该序列数据被处理完毕。

隐式编码 h_{t-1} 已经“记忆”了 t 时刻之前的时序信息

循环神经网络---概念

在时刻 t ，一旦得到当前输入数据 x_t ，循环神经网络会结合前一时刻 $t - 1$ 得到的隐式编码 h_{t-1} ，如下产生当前时刻隐式编码 h_t ：

$$h_t = \Phi(U \times x_t + W \times h_{t-1})$$

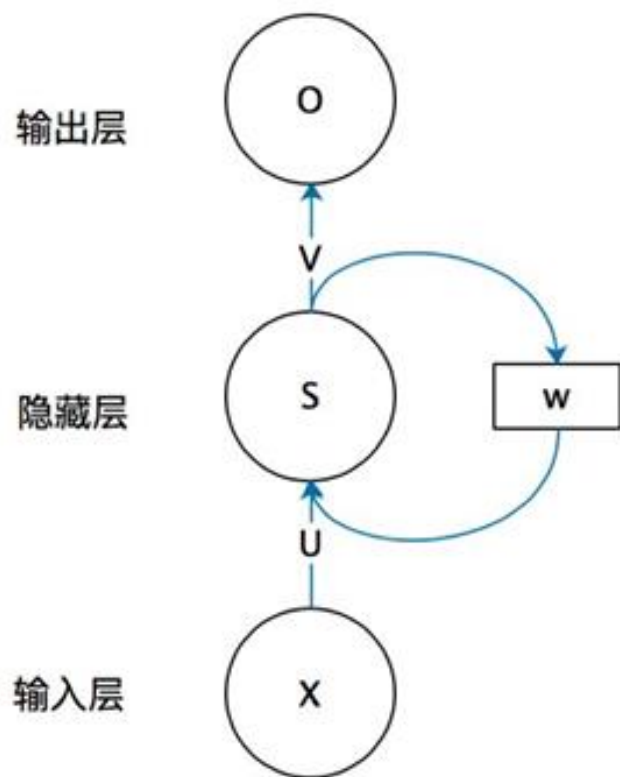
这里 $\Phi(\cdot)$ 是激活函数，一般可为Sigmoid或者Tanh激活函数， U 与 W 为模型参数。从这里可看出，当前时刻的隐式编码输出 h_t 不仅仅与当前输入数据 x_t 相关，与网络已有的“记忆” h_{t-1} 也有着密不可分的联系。

$$h_t = \Phi(U \times x_t + W \times h_{t-1}) = \Phi(U \times x_t + W \times \Phi(U \times x_{t-1} + W \times h_{t-2}))$$

$$= \Phi \left(U \times \underbrace{x_t}_{t \text{ 时刻输入}} + W \times \Phi \left(U \times \underbrace{x_{t-1}}_{t-1 \text{ 时刻输入}} + W \times \Phi \left(U \times \underbrace{x_{t-2}}_{t-2 \text{ 时刻输入}} + \cdots \right) \right) \right)$$

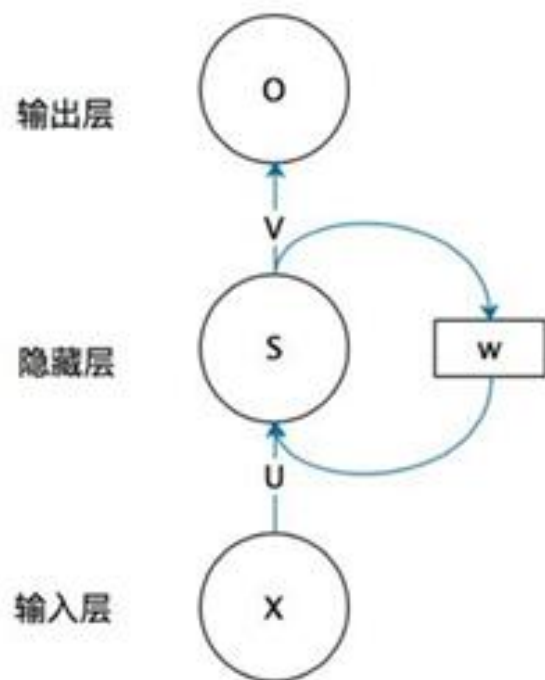
循环神经网络---计算流程

一个简单的循环神经网络由输入层、一个隐藏层和一个输出层组成。

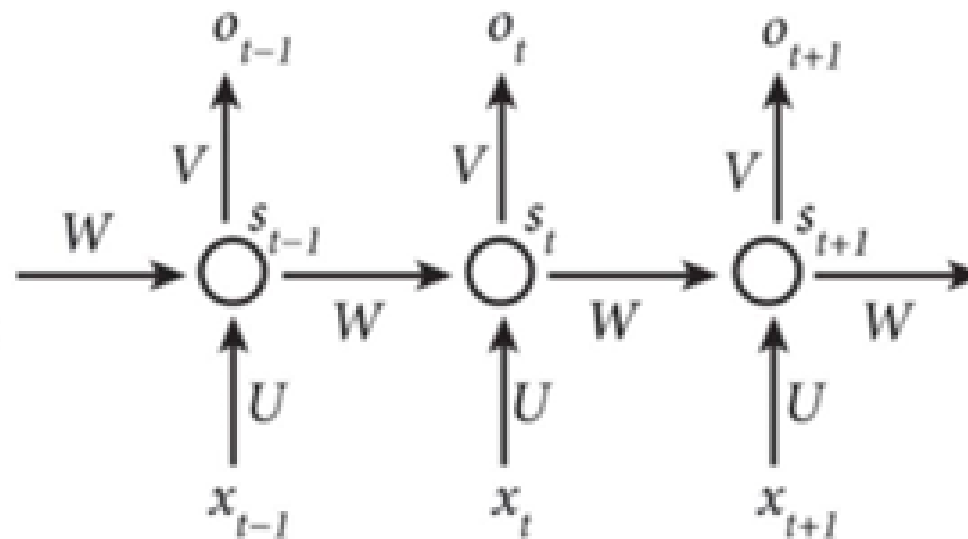


- 去掉有 W 的带箭头的圈，它就变成普通的全连接神经网络
- x 是一个向量，它表示输入层的值
- s 是一个向量，它表示隐藏层的值， s 取决于当前这次的输入和上一次隐藏层
- o 也是一个向量，它表示输出层的值
- U 是输入层到隐藏层的权重矩阵
- V 是隐藏层到输出层的权重矩阵
- W 是隐藏层上一次的值作为这一次的输入的权重矩阵

循环神经网络---计算流程



展开循环层
→



- 网络在 t 时刻接收到输入 x_t 之后，隐藏层的值是 s_t ，输出值是 o_t 。 s_t 的值不仅仅取决于 x_t ，还取决于 s_{t-1} 。

循环神经网络的计算：

$$o_t = g(Vs_t) \quad (\text{式1})$$

$$s_t = f(Ux_t + Ws_{t-1}) \quad (\text{式2})$$

循环神经网络---计算流程

循环神经网络可以往前看任意多个输入值的原因：

$$o_t = g(Vs_t) \quad (\text{式1})$$

$$s_t = f(Ux_t + Ws_{t-1}) \quad (\text{式2})$$

如果反复将（式2）带入（式1），我们将得到

$$\begin{aligned} o_t &= g(Vs_t) \\ &= g(Vf(Ux_t + Ws_{t-1})) \\ &= g(Vf(Ux_t + Wf(Ux_{t-1} + Ws_{t-2}))) \\ &= g(Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Ws_{t-3})))) \\ &= g(Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Wf(Ux_{t-3} + \dots))))) \end{aligned}$$

即输出值 o_t ，受前面历次输入值 $x_t, x_{t-1}, x_{t-2}, x_{t-3} \dots$ 的影响。

循环神经网络---训练方法

- BPTT (Back-Propagation Through Time) 算法是常用的训练RNN 的方法， 因为RNN处理时间序列数据， 所以要基于时间反向传播， 故称作**随时间反向传播**。
- BPTT的中心思想是沿着需要优化的参数的负梯度方向不断寻找更优的点直至收敛（和BP算法相同）。
- BPTT算法本质上还是BP算法， BP算法本质是梯度下降法， 那么求各个参数的梯度便成了此算法的核心。

循环神经网络---处理自然语言的例子

- 以自然语言例子来解释循环神经网络是如何处理时序数据

将自然语言句子进行分类，分类为饮食、体育、文化等不同类别。

(a) 首先，对文本语句进行分词成单词；

(b) 读入每个单词后，产生对应单词的隐式编码，并利用其来参与后续单词的隐式编码；

(c) 得到了每个单词的隐式编码后，通常可以通过如下方法得到一个句子整体的表示向量：

- 考虑到句子中所有单词的信息均被后向传递，因此将最后一个单词的输出作为整个句子向量编码表示；
- 将每个单词的隐式编码进行加权平均，将加权平均结果作为句子的向量编码表示；

(d) 最后，就可以利用某个分类器，对句子的类别进行判断，从而识别出句子的主题是什么。

循环神经网络---处理自然语言的例子

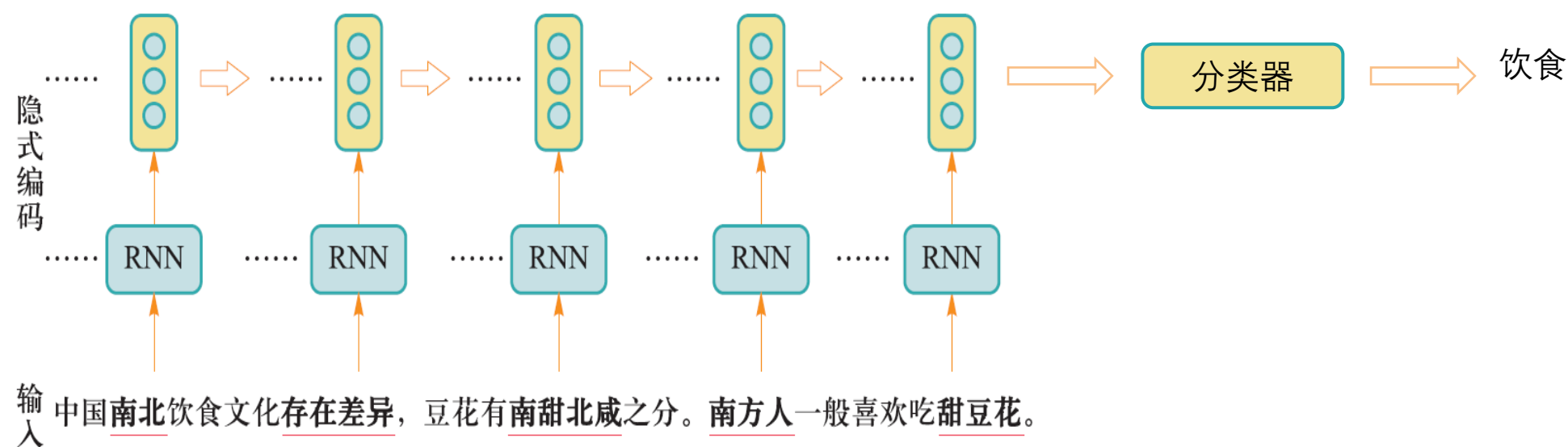


图 5.16 循环神经网络的应用示例

“南北” “存在差异” “南甜北咸” “南方人” “甜豆花” 等单词之间存在前后依赖，这种依赖会被有效利用起来，得到各单词的隐式编码及句子的向量表达。

循环神经网络---应用模式

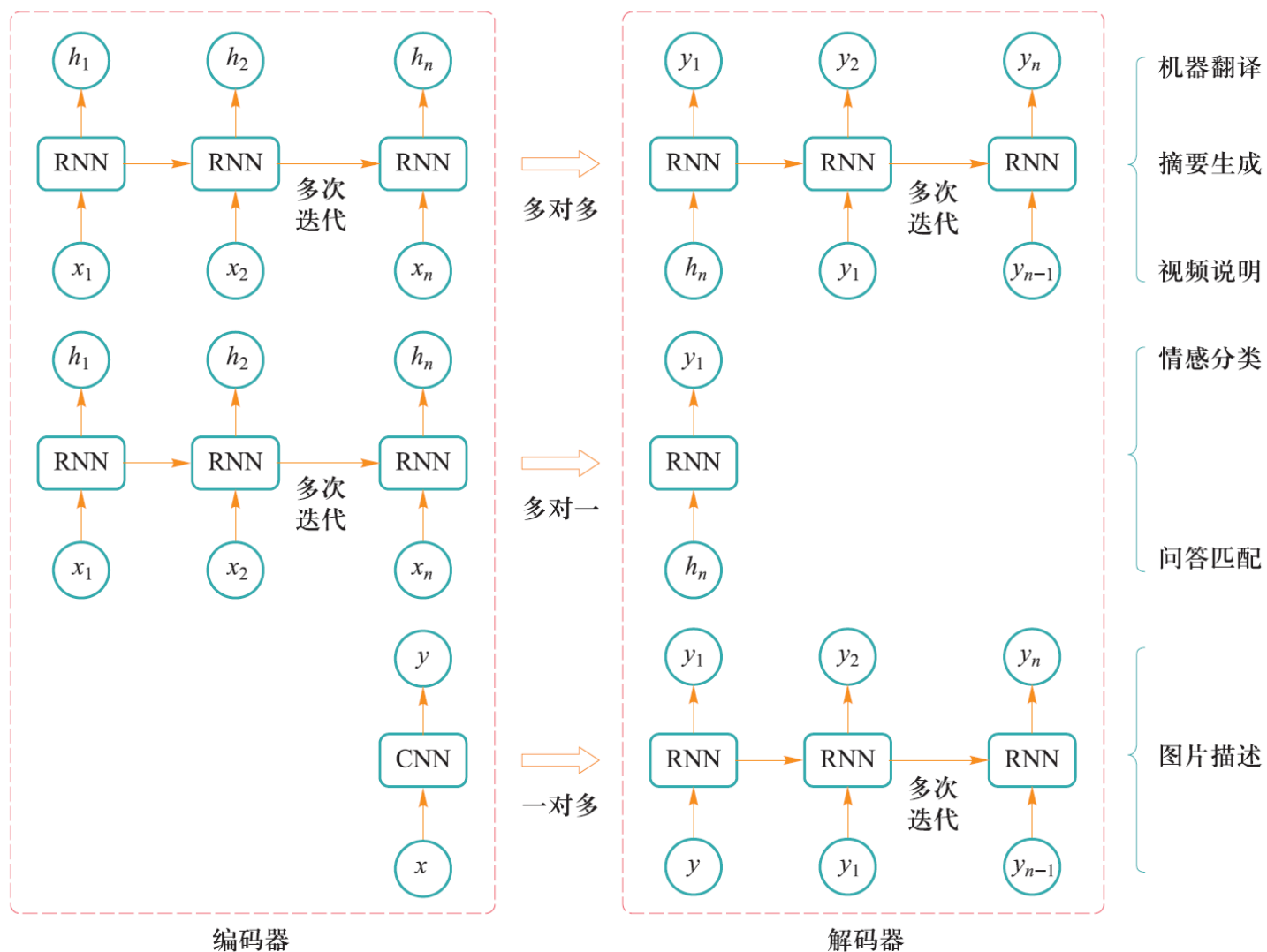


图 5.17 输入输出不同情况下循环神经网络结构示例

循环神经网络---梯度传递

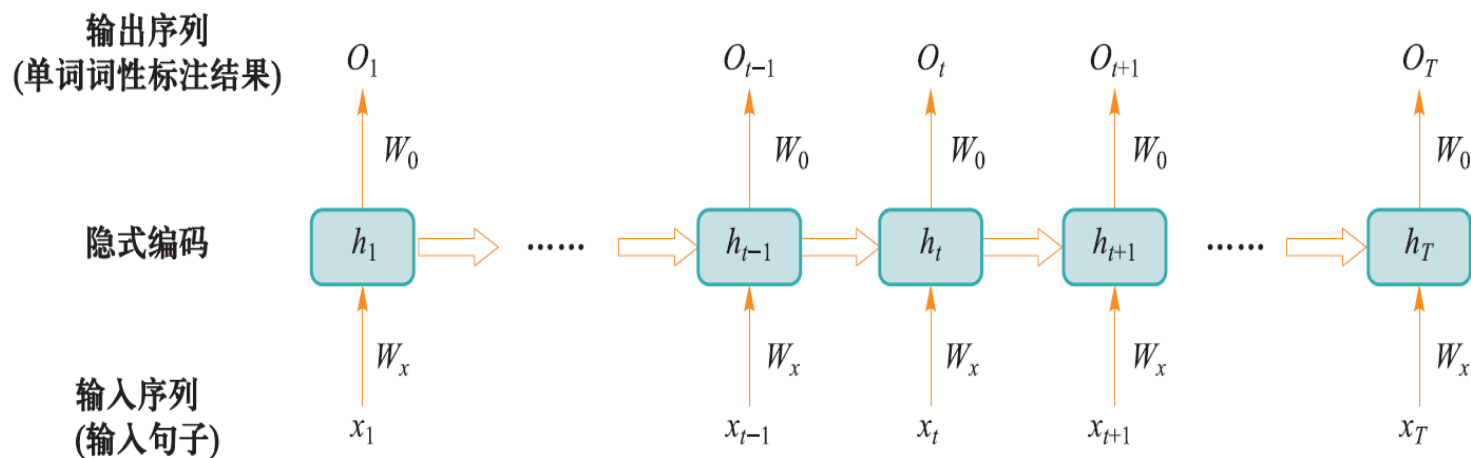


图 5.18 循环神经网络中梯度传递示意

- 图5.18以词性标注为例子给出了一个输入序列数据 $(x_1, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_T)$ 被循环神经网络处理的示意图。时刻 t 输入数据 x_t （文本中的单词）的隐式编码为 h_t 、其真实输出为 y_t （单词词性）、模型预测 x_t 的词性是 O_t 。参数 W_x 将 x_t 映射为隐式编码 h_t 、参数 W_0 将 h_t 映射为预测输出 O_t 、 h_{t-1} 通过参数 W_h 参与 h_t 的生成。图中 W_x 、 W_0 和 W_h 是复用参数。
- 假设时刻 t 隐式编码如下得到： $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ 。使用交叉熵损失函数计算时刻 t 预测输出与实际输出的误差 E_t 。显然，整个序列产生的误差为 $E = \frac{1}{2} \sum_{t=1}^T E_t$ 。

循环神经网络---梯度传递

下面介绍如何根据时刻 t 所得误差来更新参数 W_x 。在时刻 t 计算所得 O_t 不仅涉及到了时刻 t 的 W_x ，而且也涉及了前面所有时刻的 W_x ，按照链式求导法则， E_t 在对 W_x 求导时候，也需要对前面时刻的 W_x 依次求导，然后再将求导结果进行累加，即：

$$\frac{\partial E_t}{\partial W_x} = \sum_{i=1}^t \frac{\partial E_t}{\partial O_t} \frac{\partial O_t}{\partial h_t} \left(\prod_{j=i+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_i}{\partial W_x}$$

$$\text{其中} \prod_{j=i+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=i+1}^t \tanh' \times W_h$$

令 $t = 3$ ，则有：

$$\frac{\partial E_3}{\partial W_x} = \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial W_x} + \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_x} + \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_x}$$

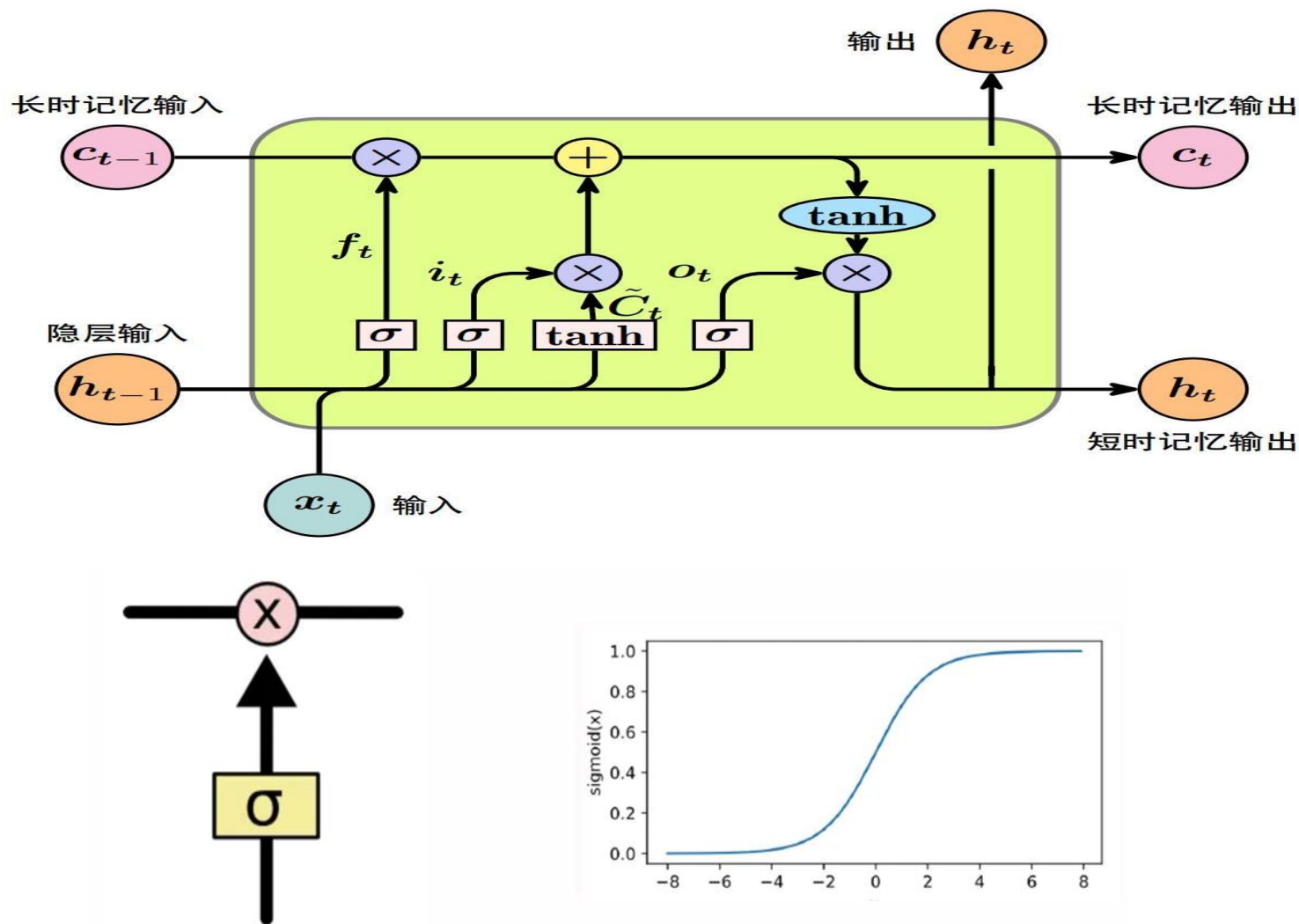
由于 $\tanh$ 函数的导数取值位于0到1区间，对于长序列而言，若干多个0到1区间的小数相乘，会使得参数求导结果很小，引发梯度消失问题。 E_t 对 W_h 的求导类似，这里就不列出了。为了缓解梯度消失问题，长短时记忆模型（Long Short-Term Memory, LSTM）被提出。

循环神经网络---长短时记忆模型

- 与简单的循环神经网络结构不同，长短时记忆网络中引入了内部记忆单元 (internal memory cell) 和门 (gates) 结构来对当前时刻输入信息以及前序时刻所生成信息进行整合和传递。在这里，内部记忆单元中信息可视为对“历史信息”的累积。
- LSTM模型中有输入门(input gate)、遗忘门(forget gate)和输出门(output gate)三种门结构。对于给定的当前时刻输入数据 x_t 和前一时刻隐式编码 h_{t-1} ，输入门、遗忘门和输出门通过各自参数对其编码，分别得到三种门结构的输出 i_t 、 f_t 和 o_t 。
- 在此基础上，再进一步结合前一时刻内部记忆单元信息 c_{t-1} 来更新当前时刻内部记忆单元信息 c_t ，最终得到当前时刻的隐式编码 h_t

循环神经网络---长短时记忆模型

LSTM 通过精心设计的称之为“门” (gate) 的结构来去除或者增加单元状态中的信息。门是一种让信息选择式通过的方法。



此门包含一个sigmoid 神经网络层和一个pointwise 乘法操作

循环神经网络---长短时记忆模型

- LSTM用两个门来控制单元状态 c 的内容
 - **遗忘门** (forget gate) , 它决定了上一时刻的单元状态 c_{t-1} 有多少保留到当前时刻 c_t ;
 - **输入门** (input gate) , 它决定了当前时刻网络的输入 x_t 有多少保存到单元状态 c_t 。
- LSTM用**输出门** (output gate) 来控制单元状态 c_t 有多少输出LSTM的当前输出值 h_t

循环神经网络---长短时记忆模型

- Long Short-Term Memory, LSTM
 - 记忆单元 (Cell State) : 存储和访问信息, 并通过门控机制 来控制信息的遗忘和记忆
 - 遗忘门、输入门和输出门

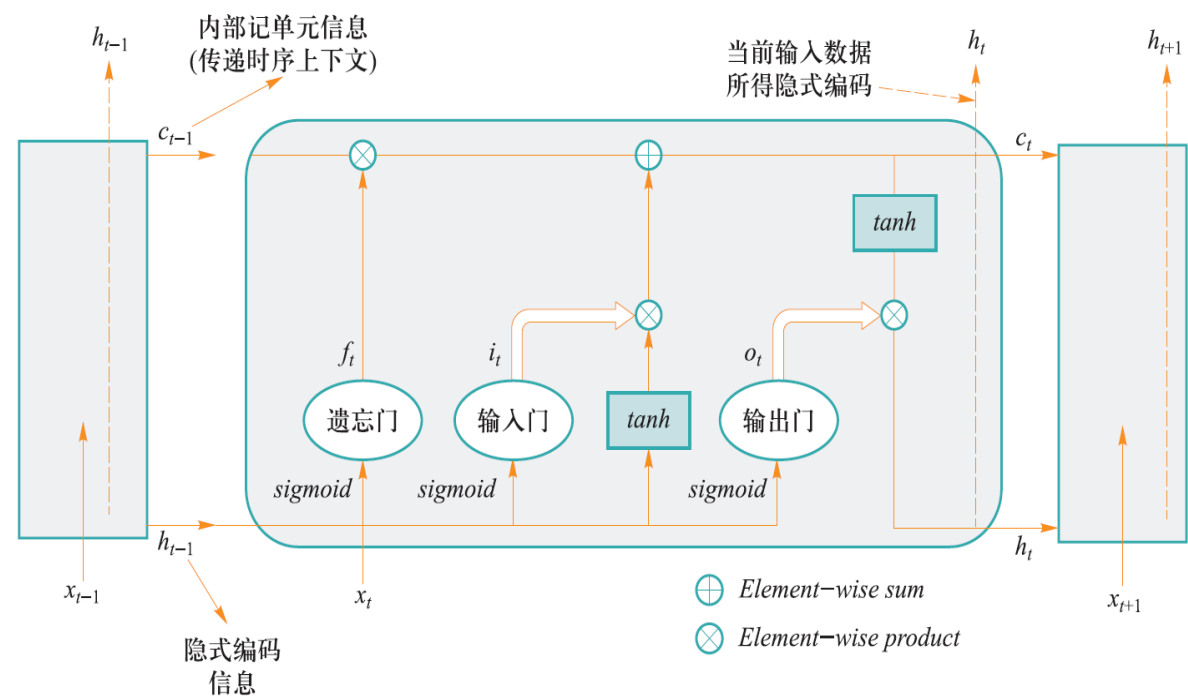
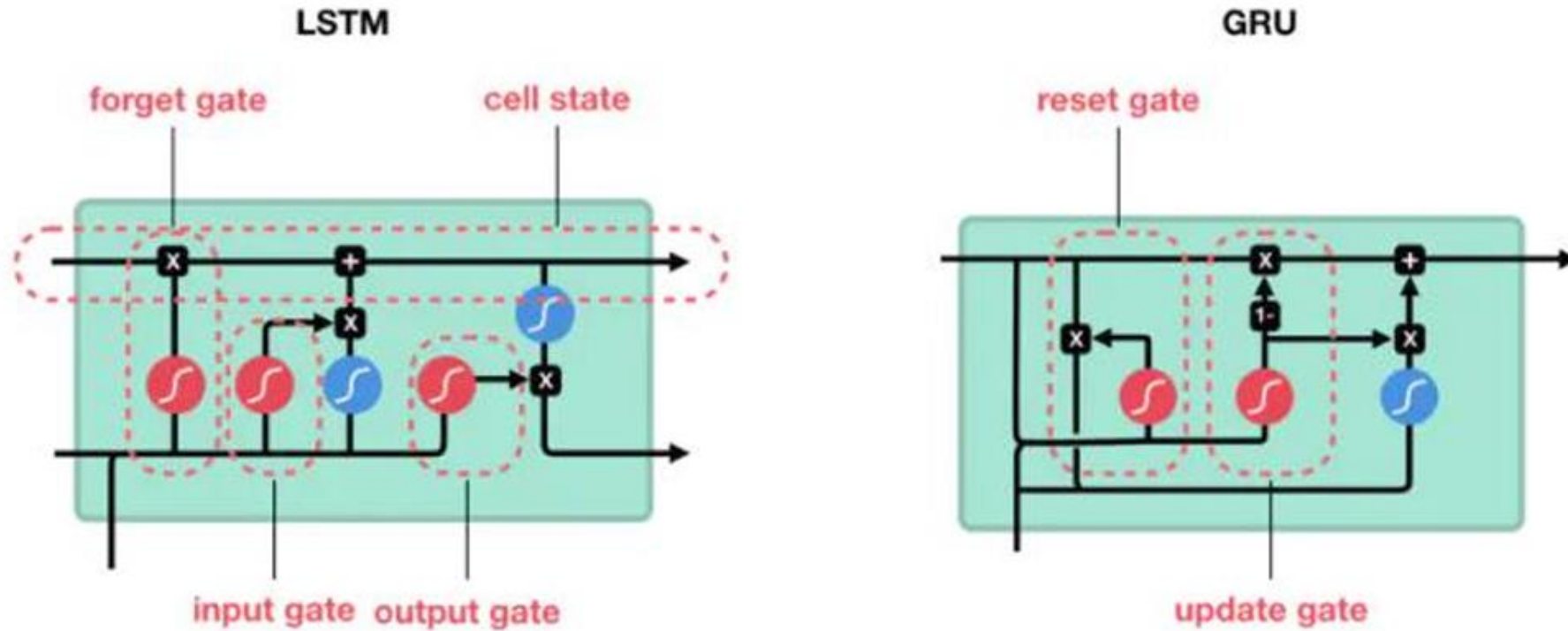


图 5.19 长短时记忆网络模型

(注: 在每个时刻 t , 只有内部记忆单元信息 c_t 和隐式编码 h_t 这两种信息起到了对序列信息进行传递的作用; 只有 h_t 作为本时刻的输出用于分类等处理)

$$\begin{aligned} \mathbf{i}_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), & \tilde{\mathbf{c}}_t &= \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \\ \mathbf{f}_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), & \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{o}_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \end{aligned}$$

循环神经网络---门控循环神经单元 (GRU)



GRU是LSTM的一种变体，它较LSTM网络的结构更加简单，而且效果也很好。

LSTM引入了三个门函数：输入门、遗忘门和输出门来控制输入值、记忆值和输出值。而在GRU模型中只有两个门，分别是更新门和重置门。

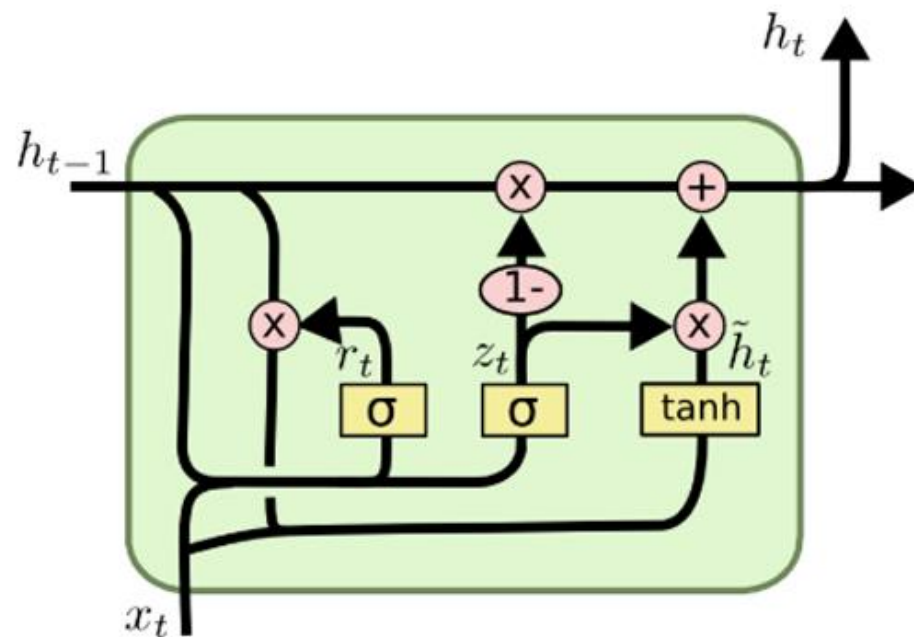
另外，GRU将单元状态与隐藏状态合并为一个状态 h 。

循环神经网络---门控循环神经单元 (GRU)

图中的 z_t 和 r_t 分别表示更新门和重置门。

更新门用于控制前一时刻的状态信息被带入到当前状态中的程度，更新门的值越大说明前一时刻的状态信息带入越多。

重置门控制前一状态有多少信息被写入到当前的候选集 $\tilde{h}_t$ 上，重置门越小，前一状态的信息被写入的越少。



计算流程：

$$\begin{aligned} z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\ r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\ \tilde{h}_t &= \tanh(W \cdot [r_t * h_{t-1}, x_t]) \\ h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \end{aligned}$$

注意力机制与正则化---大脑中的注意力

- 人脑每个时刻接收的外界输入信息非常多，包括来源于视觉、听觉、触觉的各种各样的信息。
- 但就视觉来说，眼睛每秒钟都会发送千万比特的信息给视觉神经系统。
- 人脑通过**注意力**来解决**信息超载**问题。

注意力机制与正则化---注意力示例



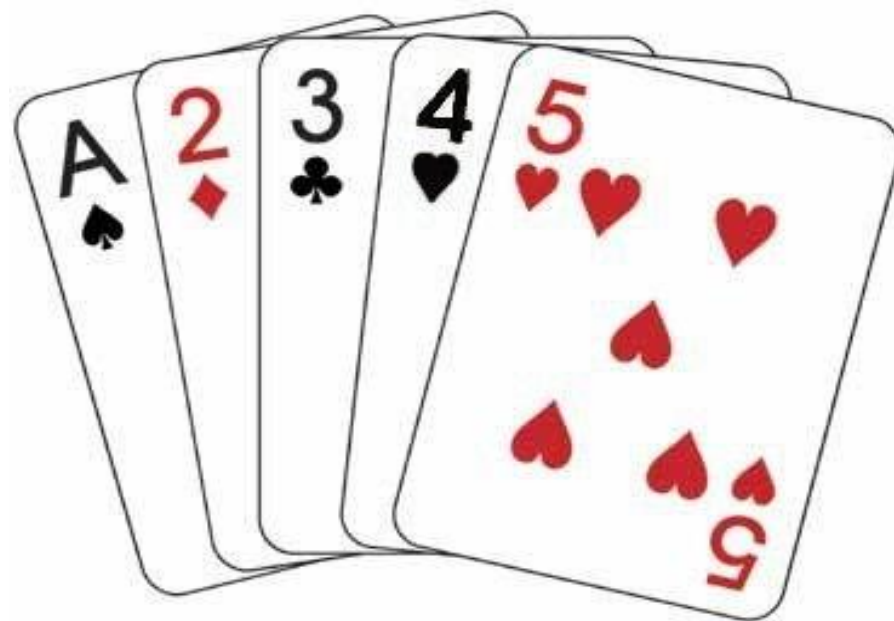
注意力机制与正则化---注意力示例

- 当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰 很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音。
- 同时，如果未注意到的背景声中有重要的词（比如他的名字）， 他会马上注意到。

鸡尾酒会效应

考试？

注意力机制与正则化---注意力示例



所有黑色牌的数字之和为多少？

注意力机制与正则化---生物学中的注意力

- 非自主性提示：基于环境中物体的突出性和易见性
- 自主性提示：认知和意识的控制

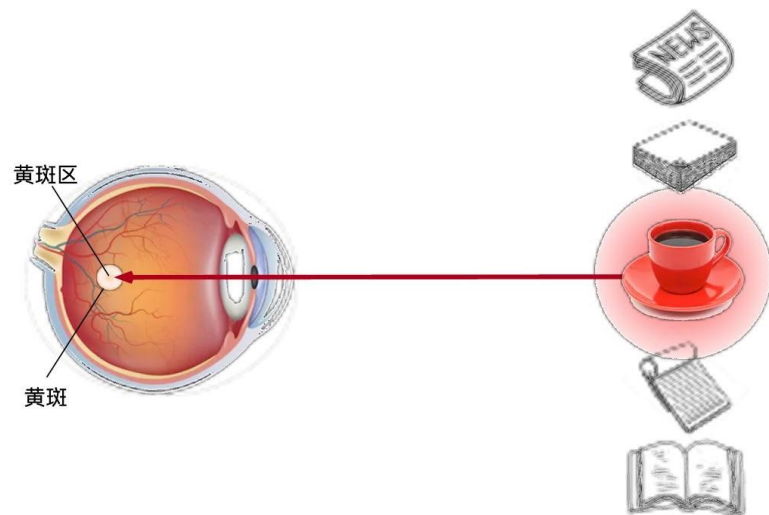


图10.1.1: 由于突出性的非自主性提示（红杯子），注意力不自主地指向了咖啡杯

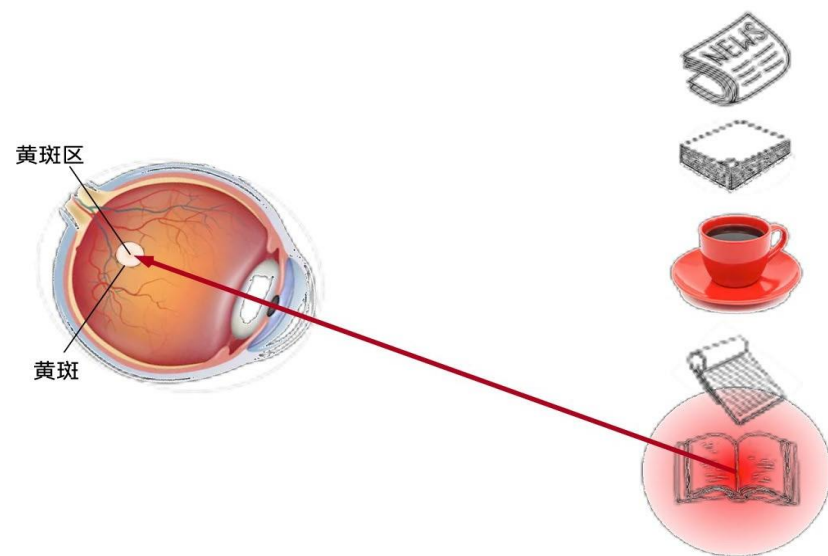


图10.1.2: 依赖于任务的意志提示（想读一本书），注意力被自主引导到书上

汇聚（pooling）

自下而上

聚焦（focus）

自上而下

MaxPooling?

注意力机制与正则化---人工神经网络中的注意力机制

- 注意力, “全局信息查询”

Transformer vs. RNN

- Transformer: 一眼看见所有单词
- RNN: 序列读入一个一个单词

I arrived at the **bank** after crossing thestreet? ...river?
What does **bank** mean in this sentence?



I've no idea: let's wait
until I read the end

RNNs

$O(N)$ steps to process a
sentence with length N

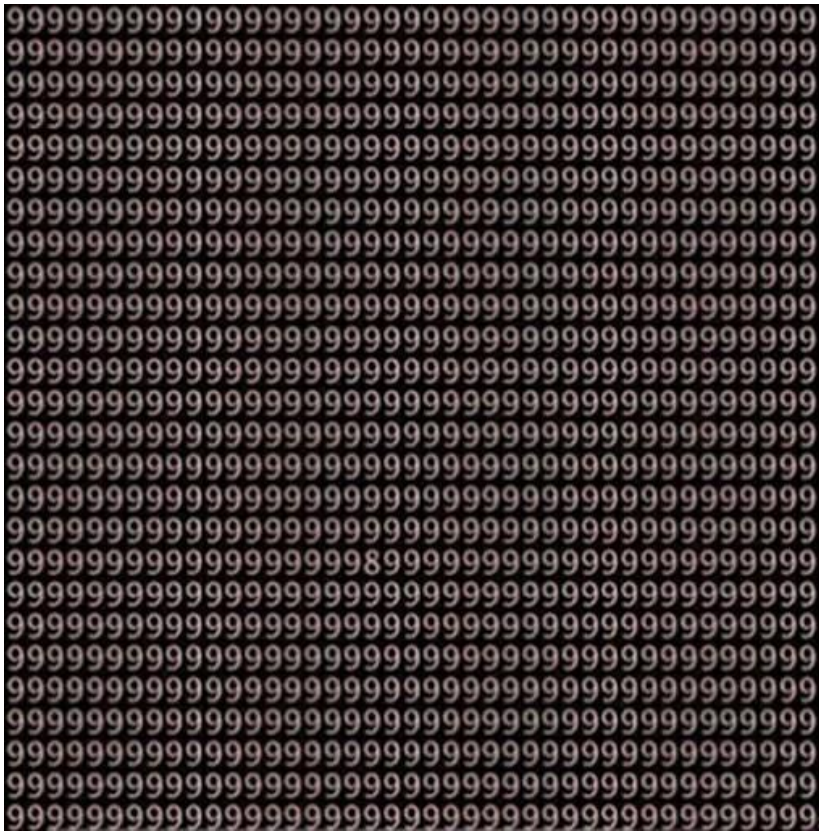


I don't need to wait - I
see all words at once!

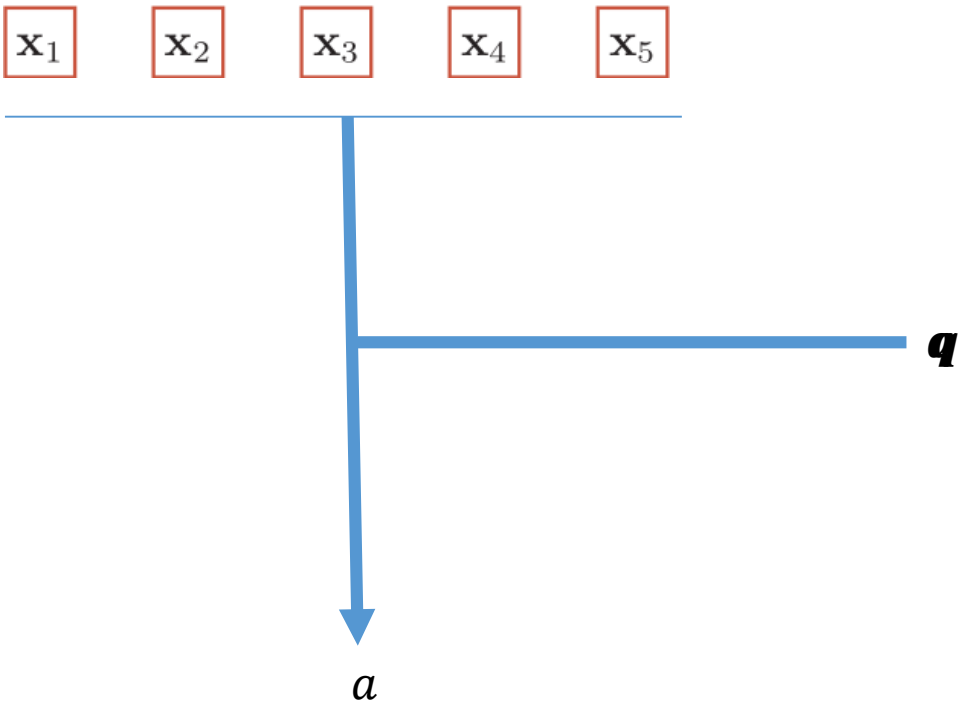
Transformer

Constant number of steps
to process any sentence

注意力机制与正则化---如何实现注意力机制？



Try to find 8 less than 8 seconds



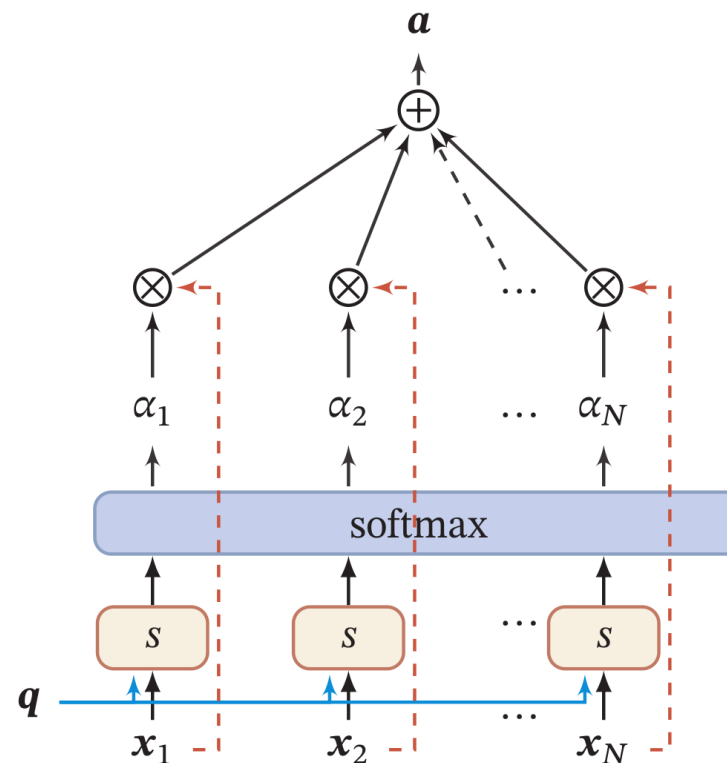
注意力机制与正则化---注意力机制

- 软性注意力机制 (soft attention mechanism)
- 注意力机制可以分为两步
 - 计算注意力分布 α

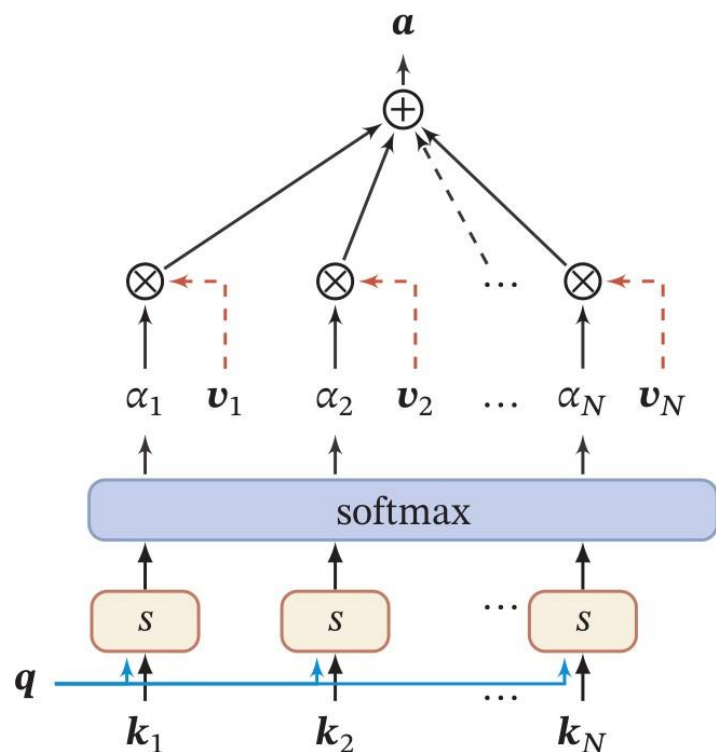
$$\begin{aligned}\alpha_n &= p(z = n | \mathbf{X}, \mathbf{q}) \\ &= \text{softmax}(s(\mathbf{x}_n, \mathbf{q})) \\ &= \frac{\exp(s(\mathbf{x}_n, \mathbf{q}))}{\sum_{j=1}^N \exp(s(\mathbf{x}_j, \mathbf{q}))}\end{aligned}\quad \begin{aligned}\text{att}(\mathbf{X}, \mathbf{q}) &= \sum_{n=1}^N \alpha_n \mathbf{x}_n, \\ &= \mathbb{E}_{z \sim p(z | \mathbf{X}, \mathbf{q})}[\mathbf{x}_z]\end{aligned}$$

$s(\mathbf{x}_n, \mathbf{q})$ 打分函数, $\mathbf{q}$ 是 query 输入一个查询

- 根据 α 来计算输入信息的加权平均。



注意力机制与正则化---注意力机制



用 $(K, V) = [(k_1, v_1), \dots, (k_N, v_N)]$ 表示N个输入信息

$$\begin{aligned} \text{att}((K, V), q) &= \sum_{n=1}^N \alpha_n v_n, \\ &= \sum_{n=1}^N \frac{\exp(s(k_n, q))}{\sum_j \exp(s(k_j, q))} v_n \end{aligned}$$

- Q是查询(query), 表示要寻找的目标 (搜索物品单词), K是键(key), 表示与查询匹配, 确定相关性 (被搜索物品的描述信息), V是值 (value) 表示与键对应的实际数据内容 (搜索物品的语义内容)
- 把Q和K各建模成一个向量, Q和K计算一个相似度 (比如向量内积) $s(k_n, q)$ 相似度函数

注意力机制与正则化---Transformer自注意力

- 首先生成每个单词的内嵌向量（包含单词在句子中位置编码向量信息），记为 w_i
- 生成每个单词 w_i 的查询向量 q_i (query)、键向量 k_i (key)和值向量 v_i (value):

$q_i = W^q \times w_i$; $k_i = W^k \times w_i$; $v_i = W^v \times w_i$; W^q 、 W^k 和 W^v 三个映射矩阵是自注意力模型需要训练的全部参数。

- 计算给定单词和句子中其他单词之间归一化关联度（softmax()函数的输出），利用关联度大小来对每个单词的自身内容调整和加权取和，求出每个单词在句子中因为上下文语义而与其他单词之间的自注意力。

注意力机制与正则化---Transformer自注意力

1、计算 w_3 所对应查询向量与其他单词键向量之间的关联度（点积）：

$$\alpha_{3i} = q_3 \cdot k_i$$

2、对 α_{3i} 通过softmax函数进行归一化，得到 α'_{3i} ， α'_{3i} 为“意在”单词与其他单词所具有的重要程度（即注意力）。

3、 α'_{3i} 乘以每个单词 w_i 所对应的值向量 v_i ，即 $\alpha'_{3i} \times v_i$ 。

4、对上一步结果进行 $\sum_j \alpha'_{3i} \times v_i$ ，这个结果作为在当前句子语境下单词 w_3 “意在”与其他单词的关联程度（self-attention）。

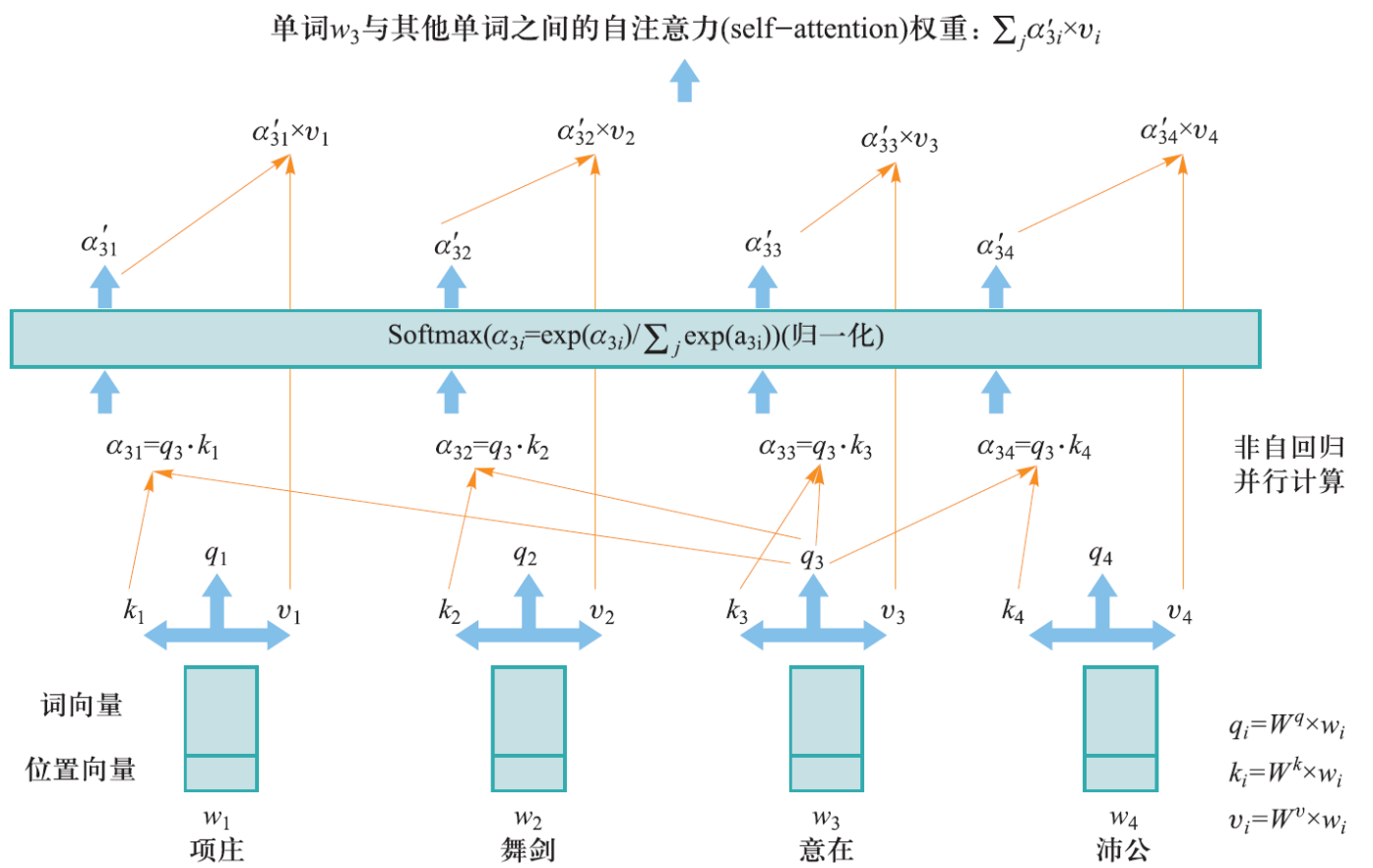


图 5.21 句子中每个单词与其他单词自注意力取值计算示意（以单词 w_3 为例）

注意力机制与正则化---正则化

深度神经网络结构复杂、参数众多，很容易造成过拟合（over-fitting）。为了缓解神经网络在训练过程中出现的过拟合现象，需要采取一些正则化技术来提升神经网络的泛化能力（generalization）。

- ◆ **Dropout**：指在训练神经网络的过程中随机丢掉（屏蔽）一部分神经元来降低神经网络的复杂度，从而防止过拟合。
- ◆ **批归一化**：批归一化（batch normalization）就是通过规范化手段，把神经网络每层中任意神经元的输入值分布改变成均值为0、方差为1的标准正态分布，把偏移较大的分布强制映射为标准正态分布。经过批归一化处理，激活函数的输入值被映射到非线性函数梯度较大的区域，使得梯度变大从而克服梯度消失问题，进而加快收敛速度。

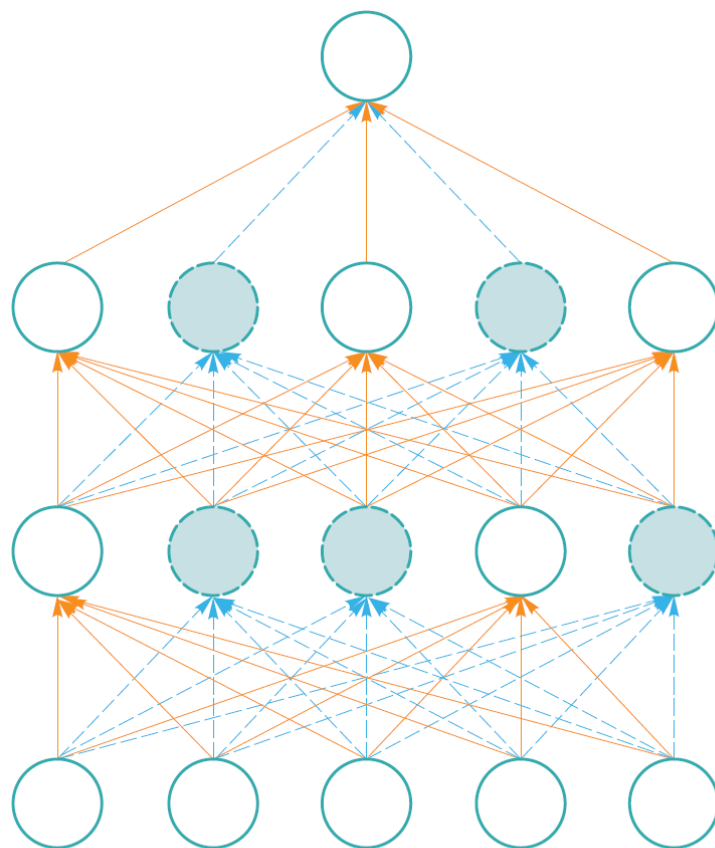


图 5.22 使用 Dropout 的神经网络模型

注意力机制与正则化---正则化

L_1 和 L_2 正则化: 对于具有 n 个训练数据 $\{(x_1, y_1), \dots, (x_n, y_n)\}$ 的神经网络, 其中 $(x_i, y_i) (1 \leq i \leq n)$ 分别为输入样本及其对应的标签。加入正则化项后, 神经网络的损失函数一般可如下表示:

$$\min \frac{1}{n} \sum_{i=1}^n \underbrace{\text{Loss}(y_i, f(\mathbf{W}, x_i))}_{\text{损失函数}} + \underbrace{\lambda}_{\text{正则化权重}} \times \underbrace{\Phi(\mathbf{W})}_{\text{正则化项}}$$

其中, $f(\mathbf{W}, x_i)$ 表示参数为 $\mathbf{W}$ 的神经网络对输入 x_i 的预测、 $\Phi(\mathbf{W})$ 为正则化项 (又称惩罚项)、 λ 为正则项权重。

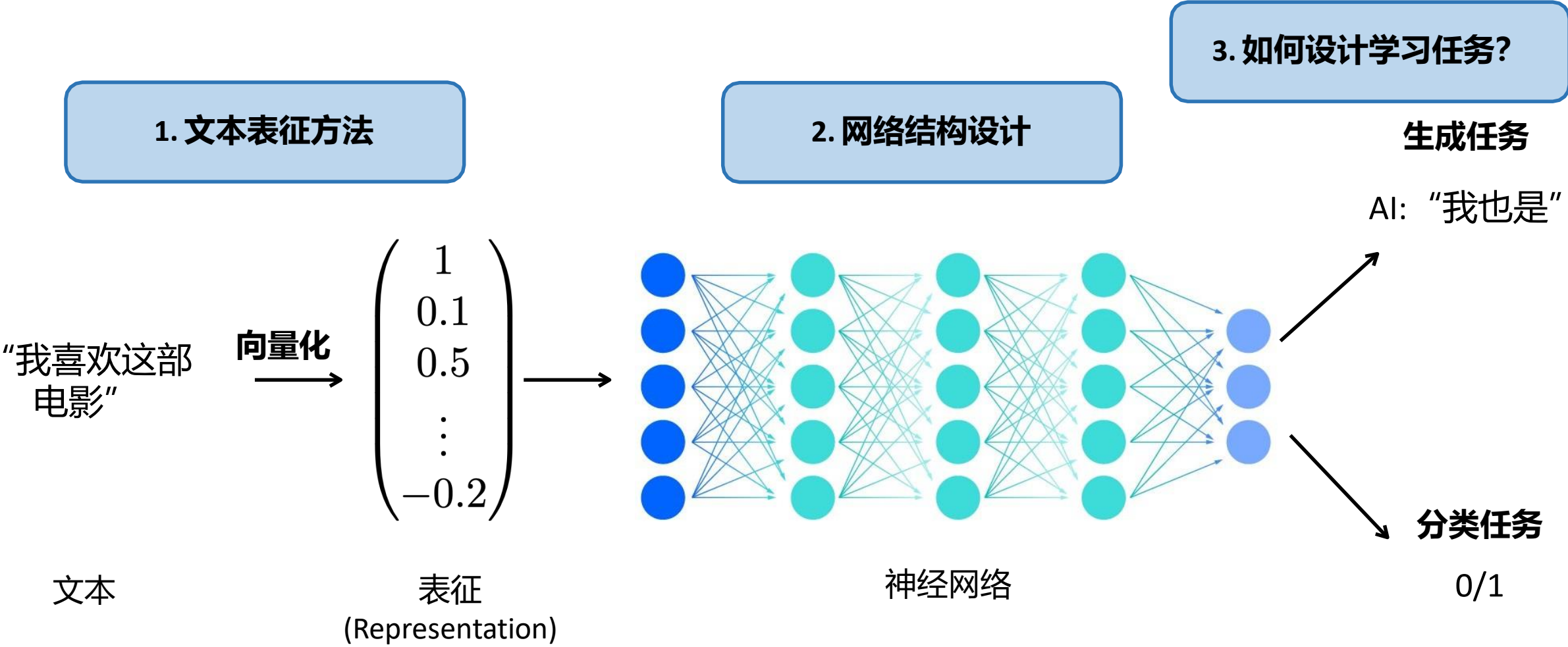
正则化项 $\Phi(\mathbf{W})$ 一般用模型参数 $\mathbf{W}$ 的范数形式来表示。假设神经网络模型中参数数目为 N , 则参数的范数形式主要有:

L_0 范数: 数学表示为 $\|\mathbf{W}\|_0 = \sum_{i=1}^N \mathbb{I}[w_i \neq 0]$, 其中 $\mathbb{I}$ 是指示函数, 若括号内表达式为真, 函数值为1, 反之为0。所以 L_0 范数指模型参数 $\mathbf{W}$ 中非零元素个数。 L_0 可实现模型参数的稀疏化。但是, 由于 L_0 范数正则化是一个NP Hard问题, 难以求解, 因此一般不用 L_0 范数。

L_1 范数: 数学表示为 $\|\mathbf{W}\|_1 = \sum_{i=1}^N |w_i|$, 指模型参数 $\mathbf{W}$ 中各个元素的绝对值之和。 L_1 范数也被称为“稀疏规则算子”(Lasso regularization)。

L_2 范数: 数学表示为 $\|\mathbf{W}\|_2 = \sqrt{\sum_{i=1}^N w_i^2}$, 指模型参数 $\mathbf{W}$ 中各个元素平方和的开方。

自然语言和计算机视觉应用---词向量生成



自然语言和计算机视觉应用---词向量生成

词典中有 V 个不同的单词，如何生成第 k 个单词的 N 维词向量？

- 在词袋模型：一个单词按照词典序被表示为一个词典维数大小的向量（称为one-hot vector），向量中该单词所对应位置按照其在文档中出现与否取值为1或0。1表示该单词在文档中出现，0表示没有出现，导致单词间关联丢失
- 词向量（Word2Vec）模型：为了刻画不同单词之间的语义相关性，利用深度学习模型，将每个单词表征为 N 维实数值的分布式向量

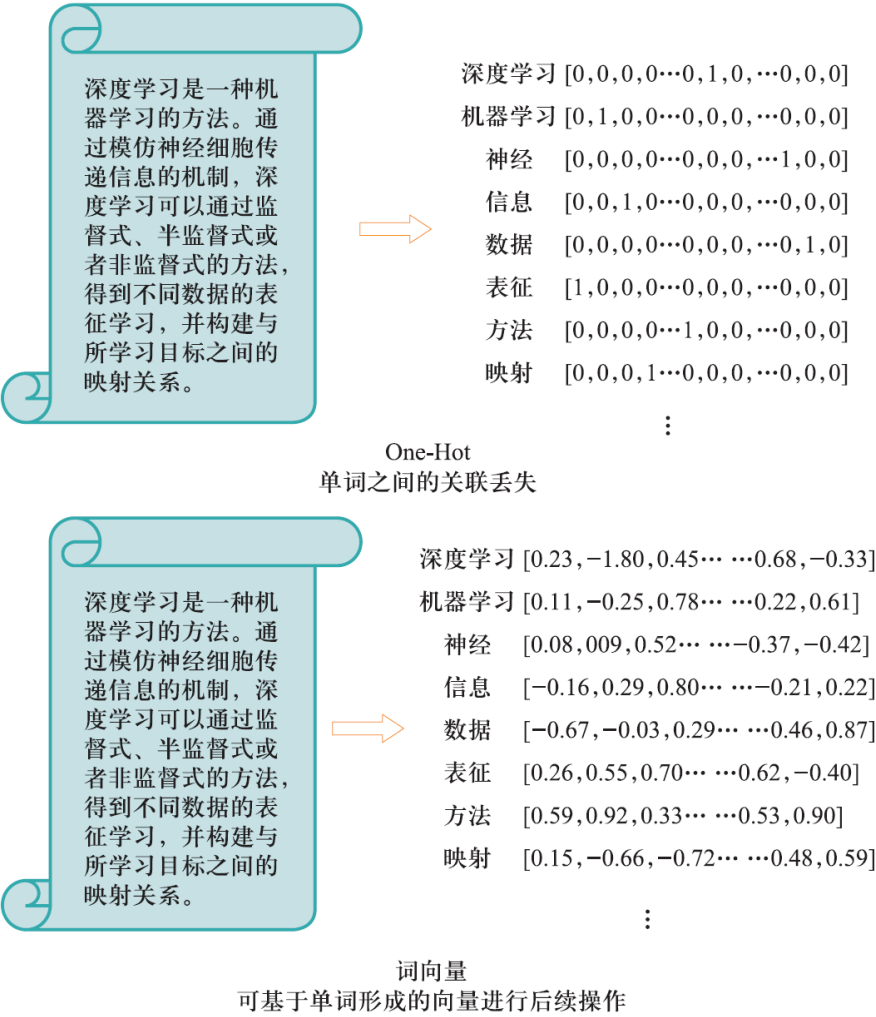


图 5.23 词典模型与词向量模型表示文本

自然语言和计算机视觉应用---词向量生成

词典中有 V 个不同的单词，如何生成第 k 个单词的 N 维词向量？

- 采用词袋模型法将该单词表示成 V 维one-hot 向量 X ，向量 X 中第 k 个位置取值为1、其余位置取值均为0。隐藏层神经元大小为 N ，每个神经元记为 $h_i(1 \leq i \leq N)$ 。向量 X 中每个 $x_i(1 \leq i \leq V)$ 与隐藏层神经元是全连接，连接权重矩阵为 $W_{V \times N}$ 。所得到的隐藏层结果与输出层是全连接，全连接权重矩阵为 $W'_{N \times V}$ ，输出层是 V 维归一化的概率值，其中 y_k 对应第 k 个单词归一化的概率值。
- 这里 $W_{V \times N}$ 和 $W'_{N \times V}$ 为模型参数。一旦训练得到了图5.24的神经网络，就可以将输入层中 x_k 与隐藏层连接权重为 $\{w_{k1}, w_{k2}, \dots, w_{kN}\}$ 作为第 k 个单词 N 维词向量(word vector)。

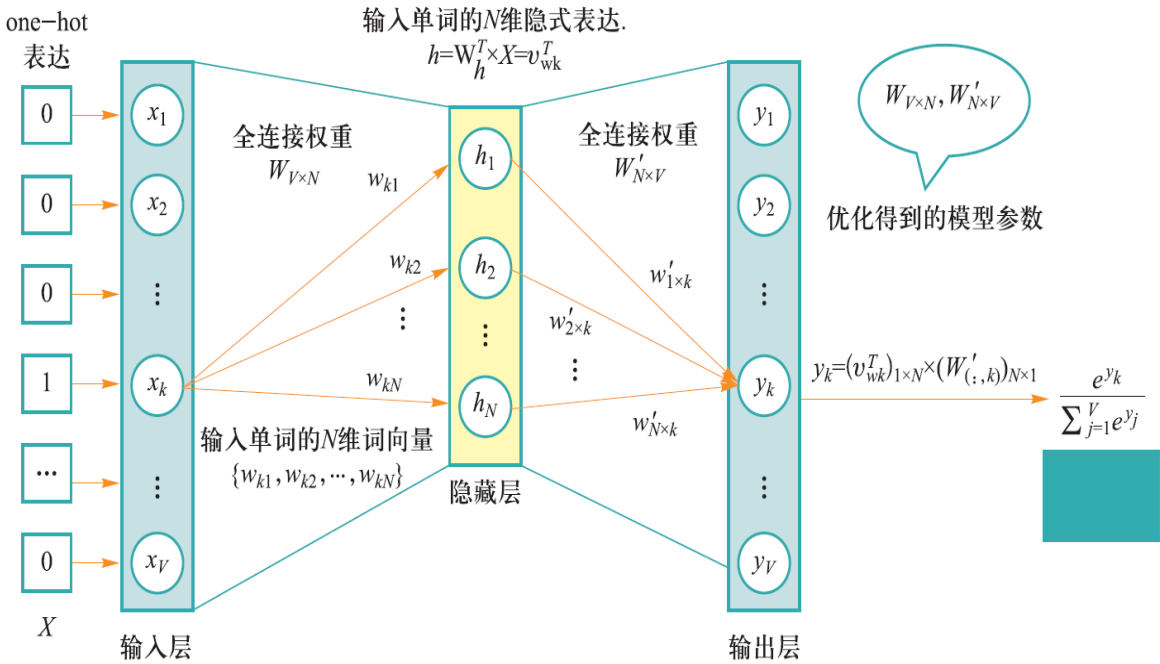


图 5.24 单个单词的词向量生成示意

自然语言和计算机视觉应用---图像分类与目标定位

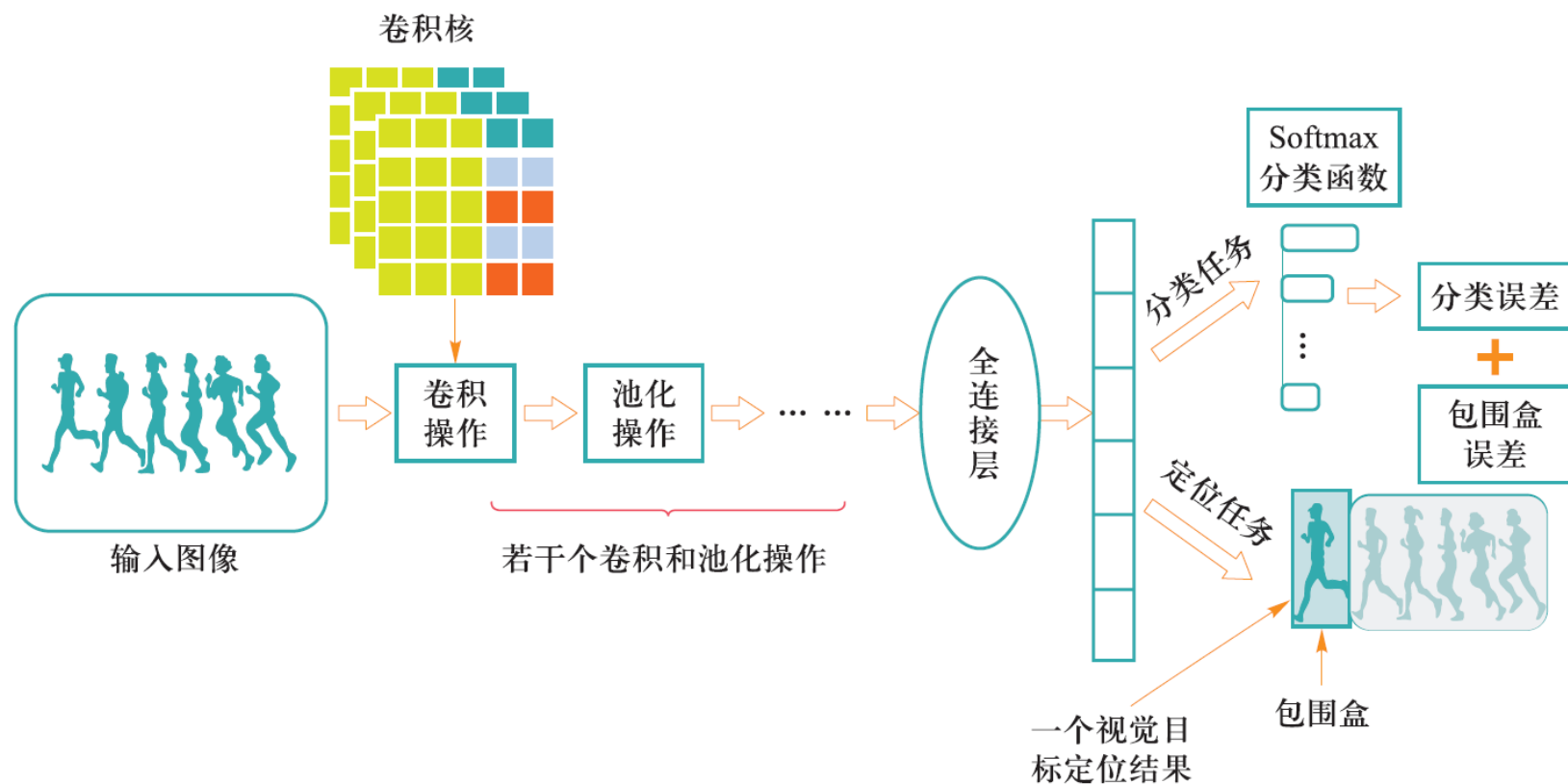


图 5.26 利用卷积神经网络进行图像分类与目标定位示意

a) 分类任务。Softmax 分类函数将全连接层所得特征向量映射为归一化概率值，用来表示每个概念在输入图像中出现的概率。在训练中，可用交叉熵等损失函数来计算模型学习的误差。

b) 目标定位。将全连接层输出向量转换为一个四维向量，**四维向量分别代表一个包围盒的左上角横坐标和纵坐标以及该矩形框宽与高取值**，并使用均方差损失函数计算模型的误差。最终，将两个任务所得误差相加，作为整个模型的误差

本章习题

1、2、3、4、5、8